

6 第六章 穷理尽微：云原生核心原理

第五章以游戏服务的部署与运维为主线，说明游戏服务为什么需要容器、编排、弹性伸缩与 Serverless，以及开发者如何通过 Docker、Kubernetes、HPA 等平台接口声明和使用这些能力。通过第五章，读者主要掌握相关技术的应用场景、需求来源、操作对象与配置方法。

本章进一步将关注点下沉到平台内部，分析这些能力的实现机制与工程代价：隔离边界如何建立，数据与状态如何流动，控制决策如何形成，时间、资源与管理开销产生于哪些环节，以及各项机制具有怎样的适用条件与能力边界。仅会配置和调用平台接口，并不足以支撑复杂系统的诊断与优化。当系统出现卡顿、排队、冷启动或资源泄漏时，开发者需要沿平台内部的数据路径和控制路径定位问题。例如，战斗服延迟升高，既可能源于容器资源限额，也可能与网络虚拟化路径有关；HPA 扩容不及时，既可能由监控指标滞后引起，也可能受制于 Pod 的创建与启动过程；Serverless 首次响应较慢，则需要进一步拆解实例分配、运行环境初始化、代码加载与应用启动等冷启动环节。

因此，第五章重点回答“这些能力适用于什么场景，以及如何声明和使用”，本章则重点回答“平台如何实现这些能力，以及实现过程中存在哪些限制与代价”。全章依次分析容器隔离、容器网络、弹性控制与快速启动机制，最后以面向 Agent 的高并发 Sandbox 为综合案例，考察这些云计算机制如何在高并发、强隔离、短生命周期和动态负载等新型工作负载下重新组合。

6.1 容器的本质：操作系统级虚拟化的三块基石

本节讨论的“容器”，指的是 Linux 上的**操作系统级虚拟化**。虚拟机通常在硬件之上再“虚拟出一台机器”，每台虚拟机有自己的客户机内核；容器采用另一条路径：多个应用进程**共享同一个宿主机内核**，同时让每一组进程拥有独立的系统视图。从结果看，容器也能做到“互不干扰”，它依靠的是内核提供的三类机制：**Namespaces** 用来隔离“看见什么”，**Cgroups** 用来限制“能用多少”，**rootfs 与镜像存储机制**用来构造“看到的文件系统长什么样”。在常见的 Linux 容器实现中，OverlayFS 经常用于将共享的只读镜像层与容器私有可写层合并为统一视图。容器运行时的职责，就是在启动应用进程之前，把这三类内核机制按同一份配置组合好：准备文件系统视图、创建命名空间、再把进程加入对应控制组。这样，一个普通进程才会在进入用户态后看到一套受限、独立、可复制的运行环境。

6.1.1 Namespaces：让进程拥有“独立的系统视图”

Namespace 是容器操作系统级虚拟化的第一类基石机制。读者第一次接触容器时，最容易观察到的差异往往来自进程视图：在宿主机上执行 `ps aux` 能看到很多进程，在容器里执行同样的命令，却通常只看到容器自己的那几个进程。这个现象直接指向 Namespace 要解决的问题：同一个内核怎样向不同进程组呈现不同的系统视图。图 77 展示了同一内核如何向不同进程组呈现不同视图：容器里并没有多出一个 Linux 内核，只是不同进程组看到的系统视图不同。

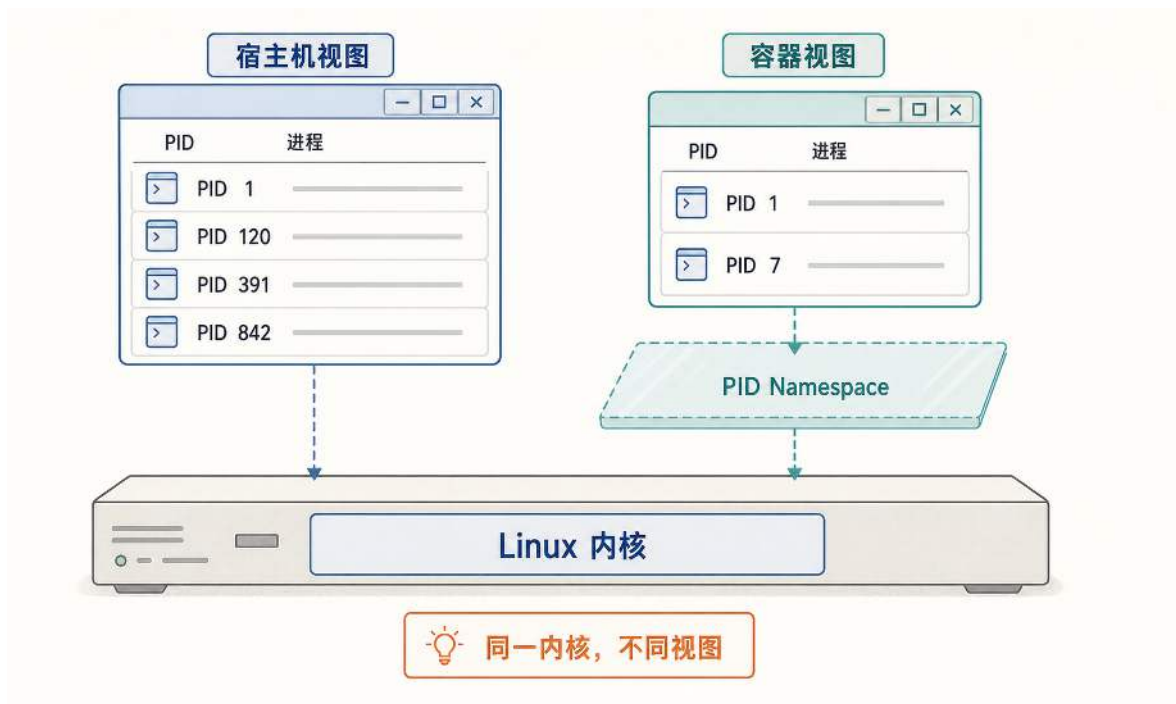


图 77: Namespace 改变进程看到的系统视图：宿主机能看到全局进程，容器只看到本命名空间内的进程。

更准确地说，宿主机处在默认的 root PID Namespace 中，容器处在运行时为它创建的 PID Namespace 中。宿主机视图中的 PID 1 是系统的 init 进程（通常是 systemd），负责启动和管理所有系统服务；容器视图中的 PID 1 则是容器的入口进程，例如游戏服务的主程序。两个 PID 1 是完全不同的进程，只是各自在所属的编号空间中被分配了相同的编号。但编号相同也意味着一些运行时责任相似：在 Linux 中，PID 1 需要承担回收孤儿子进程等职责，并且具有特殊的信号处理语义。宿主机的 PID 1 由 systemd 担任；容器内如果入口进程本身不能正确回收子进程、处理终止信号并转发给子进程，容器运行时通常会引入一个轻量的 init 进程（例如 `tini`）来承担这些工作，避免容器内子进程退出后无法被正确回收。图中宿主机看到四个进程（PID 1、120、391、842），容器只看到两个（PID 1、PID 7），因为内核为每个命名空间独立维护进程编号，容器内的程序通过 `getpid()` 得到的是命名空间内的编号，通过 `/proc` 看到的也只有同一命名空间中的进程。

由此可以得到 Namespace 的第一条边界：它解决的是“进程能看到什么”；“进程能使用多少资源”要交给后面的 Cgroups 处理。

在没有隔离机制时，Linux 系统中的许多关键资源在逻辑上是“全局共享”的。例如，系统只有一套进程号的编号规则，只有一棵统一的挂载目录树，只有一套网络协议栈，也只有一个主机名。进程之所以能“看见”这些资源，是因为它通过系统调用向内核查询或操作这些全局对象：例如 `getpid()` 返回进程号，`hostname` 查询主机名，`mount` 查询挂载信息，`ip addr` 查询网络接口等。由于这些对象在默认情况下是全局的，所以任何进程看到的结果基本一致。

Namespace（命名空间） 是 Linux 内核提供了一种隔离机制，它的思想可以用一句通俗的

话概括：同一类系统资源可以从“全局唯一的一份”，划分成多份；每个进程只属于其中一份，因此它只能看见那一份资源。Namespace 的作用可以表述为：把“全局对象”变成“按组划分的对象”，从而让不同进程组拥有不同的系统视图。

容器常用的几类 Namespace 分别隔离不同维度的视图。PID Namespace 隔离进程号空间；Mount Namespace 隔离挂载树；Network Namespace 隔离网络栈；UTS Namespace 隔离主机名。它们叠加在一起，就形成了容器最直观的效果：容器内运行 `ps` 时只看到容器自己的进程；容器内查看网络接口时，只看到属于容器的接口（最初往往只有回环接口 `lo`）；容器内的主机名可以与宿主机不同；容器内的目录结构也可以与宿主机不同。

需要特别强调的一点是：Namespace 生效在内核层，用户态工具只是读取内核提供的视图。例如，容器内执行 `ps` 看不到宿主机进程，不是“内核自动把 `/proc` 改了”，而是容器运行时通常会在新的 Mount Namespace 下挂载新的 `/proc` 并配合 PID Namespace 只暴露该命名空间中的进程。同理，`ip addr` 显示的网络接口来自内核网络子系统；Network Namespace 生效后，该命名空间拥有独立的网络接口集合，命令读到的也就只剩下容器自己的那一份。这个例子揭示了容器隔离的层次：隔离发生在内核层，用户态工具只是如实读取内核提供的视图。

从“如何创建”角度看，Namespace 是内核为进程提供的一个属性。容器运行时在启动子进程时，通过 `clone` 系统调用的标志位告诉内核需要哪些独立的命名空间。下面的片段保留了这一过程的核心结构：

用 `clone` 标志位为子进程创建独立命名空间（伪代码）

```
1 cmd := exec.Command("/bin/sh")
2 cmd.SysProcAttr = &syscall.SysProcAttr{
3     Cloneflags: syscall.CLONE_NEWPID | // 独立的进程编号空间
4             syscall.CLONE_NEWNS | // 独立的挂载树
5             syscall.CLONE_NEWNET | // 独立的网络栈
6             syscall.CLONE_NEWUTS, // 独立的主机名
7 }
8 cmd.Start()
9 defer cmd.Wait()
```

每个 `CLONE_NEW*` 标志对应前面介绍的一类 Namespace。当内核执行这段代码时，子进程 `/bin/sh` 进入四个新的命名空间：它新的 PID Namespace 中成为 PID 1；新的 UTS Namespace 初始继承父命名空间的主机名，但之后可以独立修改；新的 Network Namespace 最初只有回环接口 `lo`；新的 Mount Namespace 以父进程挂载列表的副本作为初始视图。除了在创建子进程时指定标志位，进程也可以通过 `unshare` 系统调用为自己或后续子进程创建新的命名空间；如果要让进程加入一个已经存在的命名空间，则通常使用 `setns` 系统调用。PID Namespace 是一个例外：调用进程执行 `unshare(CLONE_NEWPID)` 后，新的 PID Namespace 主要对后续创建的子进程生效，调用进程本身不会立刻变成新命名空间里的 PID 1。

但仅靠这几个标志位，子进程得到的还只是尚未完成容器化配置的系统视图：网络栈没有外部连接，文件系统以父进程挂载列表的副本为起点，资源使用也没有任何限制。Mount Namespace

隔离的是挂载列表；后续挂载事件是否会跨命名空间传播，还取决于挂载点被配置为共享、私有还是从属（shared/private/slave），容器运行时通常会进一步调整传播关系。容器运行时在创建命名空间之后，还要完成三件事才能把一个普通进程变成可用的容器：用 Cgroups 限制它能使用多少 CPU 和内存，用 rootfs 为它准备独立的文件系统，再配置网络使它能与外部通信。前两件事分别是接下来两节的主题。

综合以上机制，Namespace 的作用可以概括为：**Namespace 把原本全局共享的系统资源按组切分，并为进程提供一套“只属于它那一组”的系统视图**。有了这层视图隔离，容器才具备“看起来像一台独立机器”的第一步。

6.1.2 Cgroups：把资源使用变成“可计量、可限额、可统计”的对象

如果说 Namespace 解决的是“容器看见什么”，那么 Cgroups 解决的就是“容器最多能用多少”。仅靠视图隔离并不能避免资源争用：某个容器即使看不见其他容器的进程，也仍然可以把 CPU 长时间跑满，或者把内存申请到系统紧张，最终拖慢整台机器上的所有服务。因此，一个真正可用的容器体系必须提供**资源控制**，让每个容器的资源使用在可预期范围内。

Linux 的 **Cgroups (Control Groups, 控制组)** 的基本思想是：先把一批相关进程归为一个控制组，再由内核对该组的资源使用进行限制和统计。上一节的代码为子进程创建了命名空间，但没有限制它能使用多少资源。接下来在那段代码的基础上，为同一个子进程加上 Cgroups 限额：

为子进程创建控制组并设置 CPU 和内存限额（伪代码）

```
1 // 在 /sys/fs/cgroup 下创建一个控制组目录
2 cgroupPath := "/sys/fs/cgroup/game-battle"
3 os.MkdirAll(cgroupPath, 0755)
4
5 // CPU 限额：示例写法为每 1s 周期内最多使用 500ms CPU 时间
6 os.WriteFile(cgroupPath+"/cpu.max", []byte("500000 1000000"), 0644)
7
8 // 内存上限：最多使用 512MB
9 os.WriteFile(cgroupPath+"/memory.max", []byte("536870912"), 0644)
10
11 // 将子进程加入该控制组
12 pid := strconv.Itoa(cmd.Process.Pid)
13 os.WriteFile(cgroupPath+"/cgroup.procs", []byte(pid), 0644)
```

这段代码揭示了 Cgroups 的实现本质：控制组在文件系统中表现为 `/sys/fs/cgroup` 下的一个目录，资源限额通过写入该目录中的控制文件来设置，将进程加入控制组也是一次文件写入。Kubernetes 中的 CPU limit 和 memory limit 最终也会映射为这类控制文件中的数值。真实容器运行时还需要处理 cgroup v2 的挂载方式、写入权限、controller 是否已启用，以及进程在执行应用代码前完成归组等细节；这里的伪代码只保留资源限额的核心路径。

图 78 将 Cgroups 的资源控制链路按执行顺序展开：进程先被归入控制组，控制组再获得 CPU 与内存限额，内核随后在运行时统计用量并执行限制。理解这条链路之后，`cpu.max` 与 `memory.max` 就不只是两个配置项，而是控制组约束资源使用的入口。

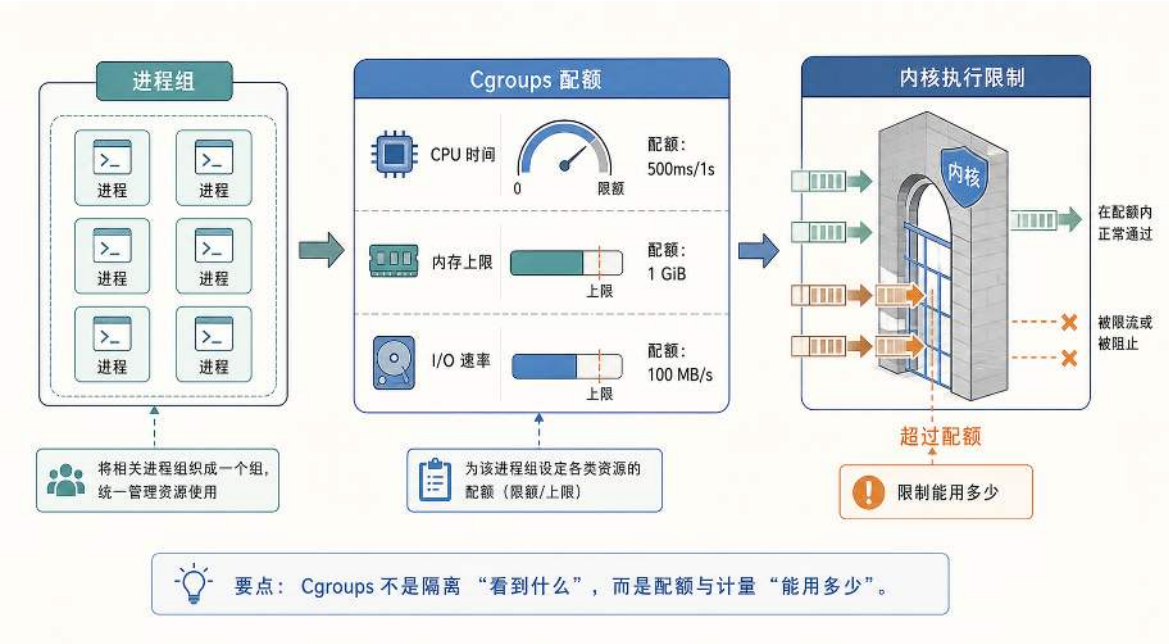


图 78: Cgroups 将进程组织成控制组，并对 CPU、内存和 I/O 等资源进行统计与限额。

代码中 `cpu.max` 写入的 `500000 1000000` 直接对应 CPU 限制的核心机制。这里将周期写成 1 秒，主要是教学演示；在许多实现里也常见 100 毫秒量级的配额周期（100000 微秒）。其含义是：内核每个周期为该控制组发放 500000 微秒（500ms）的 CPU 运行预算。组内所有线程在运行时共同消耗这份预算；一旦 500ms 用完，控制组内的所有线程会被暂停（throttled），直到下一个 1s 周期开始才重新获得预算。由此可见，CPU 限制关注的是“一段时间里累计可运行的总时长”，而不是固定分配某个核心。

这个机制会带来一个初学者容易误解的现象：**同样的配额，在单线程与多线程场景下表现完全不同**。预算按控制组整体计算，而不是按线程分别计算。如果战斗服容器内同时有 4 个线程在不同核心上并行运行，500ms 预算会以约 4 倍速消耗，大约 125ms 墙钟时间后整个控制组就会被暂停。宿主机还有空闲 CPU 核心，但容器内所有线程都停下来等待下一个周期。第 3.3.4 节用 CPU 占用和 P99 延迟来判断单机瓶颈；到了容器环境，这些指标还要和 Cgroups 配额一起解读，才能判断应该优化代码、放宽配额还是增加副本。

同样的思想也适用于内存控制。代码中 `memory.max` 设置的 512MB 是该控制组的硬上限。`memory.high` 是内存压力阈值：使用量超过该值后，组内进程会被节流并承受更强的页面回收压力，但不会仅因超过该阈值触发 OOM。当使用量达到 `memory.max`，且无法通过回收将其降低时，才进入该控制组的 OOM 处理。是否杀掉整个控制组相关进程，取决于 `memory.oom.group` 的配置；否则内核通常只在组内选择牺牲进程。无论哪种策略，内存限制目标是把局部失效限制

在可控单元内，避免一个容器的内存泄漏扩散为整台机器故障。

从 Namespace 到 Cgroups，容器运行时完成了两件事：隔离视图和限制资源。但容器内的程序仍然需要一套独立的文件系统才能运行：rootfs 决定进程把哪棵目录树作为根目录，镜像存储机制则负责组织和复用其中的文件内容。

6.1.3 rootfs 与 OverlayFS：镜像分层与容器写入的机制

容器内的程序需要可执行文件、动态库和配置文件才能启动，这些都存放在文件系统中。如果没有文件系统层面的隔离，容器内的进程会直接读写宿主机的目录树，无法形成独立的运行环境。这就引出文件系统层面需要解决的三个问题：**容器里看到的文件系统从哪里来？多个容器能否共享同一份基础环境？容器运行时产生的文件存到哪里？**

Linux 进程运行时，需要读取可执行文件、动态库和配置文件，这些文件都通过路径访问，而路径是相对于“根目录 /”展开的。例如，进程加载 `/bin/sh` 时，内核从当前进程看到的根目录开始查找 `bin/sh`。如果容器进程与宿主机共享同一个根目录，它读到的 `/bin`、`/lib`、`/etc` 就是宿主机的目录内容，无法构成独立环境。解决办法是：为容器进程指定一个不同的目录作为它的根目录 `/`。这个被指定为容器根目录的目录树，就是 **rootfs (root filesystem)**。当 rootfs 包含一套完整的目录结构时，容器内的程序通过 `/bin` 找到可执行文件，通过 `/lib` 找到动态库，通过 `/etc` 找到配置，rootfs 覆盖的这些普通文件路径就与宿主机目录树隔离开来。不过，`/proc`、`/sys` 等伪文件系统仍由宿主机内核提供，容器也可以通过 `bind mount` 或外部卷显式挂入宿主机路径。容器镜像的核心作用，就是为 rootfs 提供内容：一份可以被用作根目录的文件系统快照。

如果每个容器都需要一份完整的 rootfs，那么在同一台机器上启动多个容器时，磁盘空间会被大量重复文件占满。实际上，同一批容器往往共享相同的基础环境：同一个 Linux 发行版、同一套运行时库、同一套依赖包。更理想的做法是：**基础环境只保存一份并且只读共享，每个容器只保存自己的改动**。下文以采用 **OverlayFS (叠加文件系统)** 的常见 Linux 容器实现为例，说明镜像层共享和容器可写层的实现机制。

OverlayFS 的做法是让基础环境只读、共享，各容器的改动私有。图 79 展示了这一分层结构：底部若干层是只读镜像层，自底向上叠出基础目录、运行时库和应用文件，多个容器共用同一批而不各自复制；顶部是每个容器私有的可写层，未挂载外部卷的路径一旦写入，改动就落到这里。容器进程看到的不是某一层，而是各层合并后的**合并视图**：同一路径只出现一次，内核自上而下查找，优先返回可写层里的版本。这样，共享的只读层省下了磁盘，私有的可写层又守住了隔离。

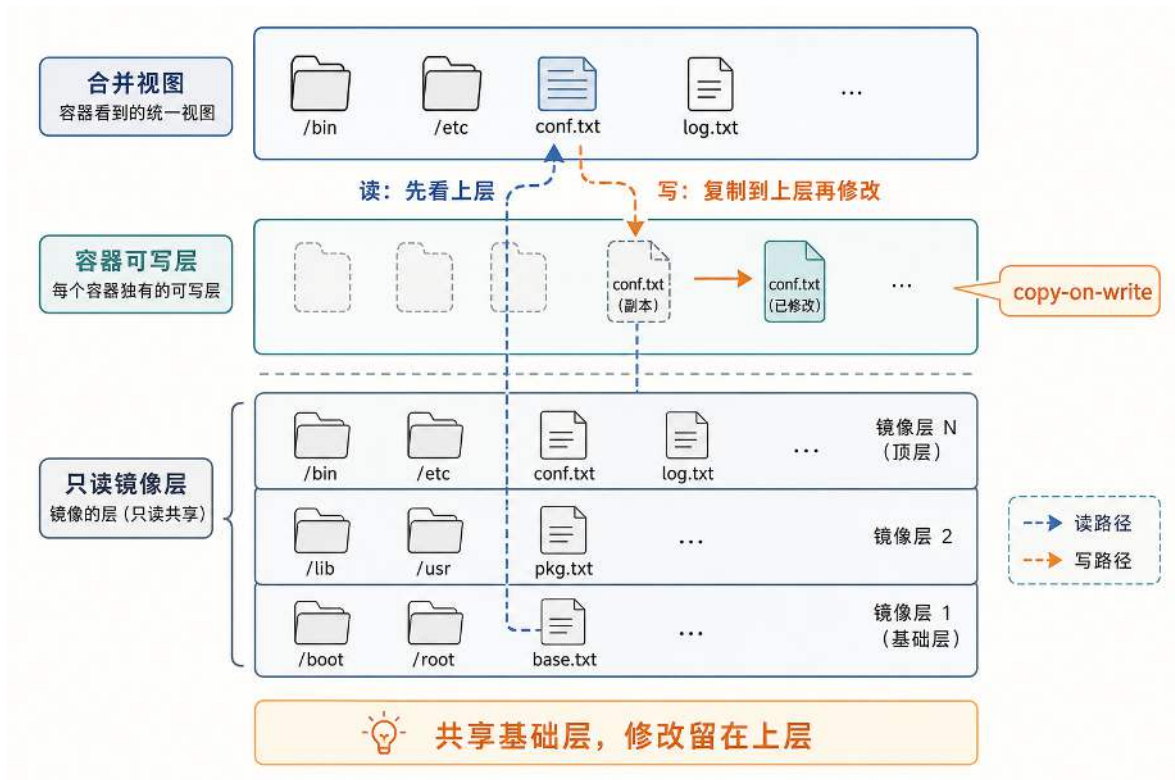


图 79: OverlayFS 将多层只读镜像层与容器可写层合并为统一视图, 写入时通过 copy-on-write 把文件复制到上层再修改。

写入路径上还藏着一个关键细节。当容器需要修改一个只存在于只读层的文件(如 `conf.txt`)时, 内核不会直接改动只读层, 而是先将该文件复制到可写层, 再在可写层中修改副本。这就是 **copy-on-write (写时复制)**: 只有实际发生写入时才触发复制, 复制粒度是整个文件 (而非块存储系统中常见的块级复制); 平时读取只读层中的文件不会触发 copy-up, 只承担合并视图查找带来的少量元数据开销。此后, 合并视图中该路径指向的就是可写层中的新版本, 只读层中的原始文件仍然完好。如果容器删除一个只读层中的文件, OverlayFS 不会修改只读层, 而是在可写层中创建一个特殊的 **whiteout 文件**, 标记该路径已被删除; 合并视图在遍历到 whiteout 标记时会跳过对应的只读层文件, 使其从容器视角消失。

至此, 前面提出的三个问题都有了明确的答案。容器看到的文件系统来自 **rootfs**: 一个被指定为根目录的目录树。多个容器通过共享只读镜像层来复用基础环境, OverlayFS 保证共享的同时互不干扰。在未挂载外部卷、bind mount 或 tmpfs 的路径上, 容器运行时产生的文件, 无论是新创建的还是对已有文件的修改副本, 都存储在该容器独有的可写层中。可写层的生命周期与容器绑定: 容器被删除时, 可写层随之清除, 只读镜像层不受影响。

这一机制也解释了一个常见现象: “删除容器后数据丢失”。运行时的改动保存在可写层中, 容器被删除时可写层一并清理, 镜像层提供的环境仍然保留, 但可写层中的运行数据已经不存在。如果希望数据不随容器消失, 就需要把数据放到不属于可写层的地方 (例如挂载宿主机目录或使用独立卷), 这就是容器持久化存储的动机。

综合来看, **rootfs** 解决的是 “容器把哪一份目录树当作 /”; **OverlayFS** 解决的是 “这份目

录树如何既能共享基础环境，又能允许每个容器独立写入”。理解了这两点，“镜像分层”“容器可写”“删除容器数据消失”等现象就可以从机制层面统一解释。

从工程视角看，这也正是容器解决“依赖环境复杂”的关键。游戏服务往往同时依赖特定版本的运行时与系统库：例如某个组件需要特定版本的 `glibc/libstdc++`，另一个组件依赖某套语言运行时与第三方包；如果这些依赖直接安装在宿主机上，就容易出现版本漂移与相互污染，导致同一份代码在不同机器上表现不一致。镜像提供的 `rootfs` 本质上是可复制的用户态依赖环境：可执行文件、动态库、配置目录都放进同一套目录树，容器启动时再通过挂载变成进程看到的根目录 `/`，从而让 `/bin`、`/lib`、`/usr` 的视图与宿主机解耦。

镜像分层与 OverlayFS 进一步把依赖环境的复用与隔离同时做到：基础层可以被多个服务共享（同一发行版与通用运行时只保存一份），服务层只叠加该服务独有的依赖与二进制；运行时写入则落在容器自己的可写层里，不会回写到基础层，也不会污染其他容器。这样，只要节点安装了容器运行时并能拉取同一份镜像，调度器把服务放到哪台机器上启动，依赖环境都是一致的。

6.1.4 容器的安全边界：共享内核带来的隔离限度

理解了 Namespaces、Cgroups、rootfs 与常见的 OverlayFS 实现之后，容器的大部分表面现象已经可以解释：它看起来像一台独立机器，有自己的进程列表、网络接口与目录结构，并且资源使用还能被限制。但在安全边界上，容器与虚拟机仍然存在本质差异。虚拟机的隔离来自**硬件虚拟化与 Hypervisor**：每个虚拟机通常运行自己的内核，内核之间由虚拟化层隔开。容器则不同：**所有容器共享同一个宿主机内核**。因此，容器的隔离更准确地说是“同一内核中的隔离”，而不是“不同内核之间的隔离”。

这一区别带来的直接后果是：Namespace 与 Cgroups 虽然能让容器“像是独立的”，但它们并不自动提供与虚拟机同等级别的安全边界。Namespace 主要改变的是**进程看到的系统视图**，例如看见哪些进程、哪些网络接口、哪些挂载点；Cgroups 主要控制的是**资源使用额度**，例如最多能用多少 CPU 时间、最多能占用多少内存。它们都不改变一个事实：容器内的进程执行系统调用时，最终仍然进入**同一个宿主机内核**。因此，一旦内核存在可被利用的漏洞，攻击者就可能借助漏洞突破隔离边界，从容器影响宿主机，这就是所谓“容器逃逸”的根源。

由于容器共享内核，工程实践通常会进一步收紧容器权限，以降低风险。例如，容器运行时会尽量减少进程拥有的特权能力（capabilities），并通过系统调用过滤（seccomp）限制可调用的内核接口；在需要更强策略控制时，还会配合 AppArmor/SELinux 等安全模块。若业务必须运行不可信代码（例如用户提交的脚本或插件），常见做法是引入额外隔离层：在容器与内核之间加入“系统调用拦截层”（如用户态内核方案），或者干脆把每个容器放入轻量虚拟机中运行，以换取更强的安全边界。选择哪种方案，本质上是在性能、隔离强度与运维复杂度之间做权衡。

本节的核心结论是：**容器之所以像独立系统，是因为 Namespace 改变了进程看到的系统视图，Cgroups 限制了进程组能使用的资源额度，rootfs 构造了容器看到的文件系统；OverlayFS 则是实现镜像层共享和容器独立写入的一种常见机制。而容器之所以不等于虚拟机，是因为它们仍然共享同一个内核，这决定了它们的安全边界与风险模型不同。**

到这里,单个容器已经具备隔离视图、受限资源和独立文件系统,但它的 Network Namespace 中只有一个回环接口 `lo`,既无法与同机的其他容器通信,也无法访问外部网络。这引出容器网络虚拟化的核心问题:如何在同一台宿主机上为多个隔离的网络命名空间搭建出一张互通的虚拟网络。

6.2 网络虚拟化原理:从隔离的网络命名空间到互通的容器网络

上一节说明, Network Namespace 为每个容器创建了一套独立的网络协议栈,包含独立的接口集合、路由表和端口空间。新创建的网络命名空间中,内核只提供一个回环接口 `lo`,没有外部链路,也没有可用的 IP 地址。容器要与其他容器或外部网络通信,必须由容器运行时在命名空间之间建立连接。

容器网络可以按两个维度分类:一是通信是否跨越宿主机,二是跨越宿主机边界时由哪一侧主动发起连接。本节以一条“典型 Linux 实现”为主线来讲解(veth、bridge、SNAT/DNAT 与 VXLAN),因为它把每一层要解决的问题都显式暴露了出来,便于逐步拆解。需要提醒的是,这只是众多实现中的一种:实际的 CNI 还可能采用三层路由、eBPF、云厂商 VPC 原生网络或硬件卸载,其数据路径不一定包含 bridge、iptables 或 VXLAN。读者应把下面的主线理解为“一种把问题讲清楚的典型路径”,而非唯一路径。本节围绕四个递进的问题展开。第一,容器如何获得一块可以收发数据包的网络接口和一个 IP 地址?第二,同一台宿主机上的多个容器如何相互通信?第三,容器主动访问外网与外部主动访问容器时,宿主机分别如何转发数据包?第四,不同宿主机上的 Pod 如何互通?这四个问题分别对应 veth、bridge、SNAT/DNAT 和跨主机路由或 Overlay 网络,不能把它们混为同一组候选方案。

6.2.1 容器的第一块网络接口: veth 对与命名空间互联

在讨论容器网络之前,需要先明确一个基础概念。Linux 中通常所说的“网卡”,更准确的名称是**网络接口 (network interface)**。网络接口不一定对应真实的硬件电路板,也可以是内核创建的软件对象。无论是物理接口(例如连接网线的以太网口)还是虚拟接口,它们对上层协议栈的作用相同:为内核协议栈提供一个收发网络数据的出入口。对于以太网接口和 veth 这类设备来说,这个出入口在链路层表现为二层以太网帧的收发。

上一节提到,新创建的 Network Namespace 中只有一个回环接口 `lo`。`lo` 是 Linux 的**回环接口 (loopback interface)**,从它发出的数据包不会到达任何外部链路,而是直接回到本机协议栈的接收路径。回环接口的作用是让同一命名空间内的进程通过 TCP/IP 互相通信(例如访问 `127.0.0.1`)。需要注意,`localhost` 并不等于 `lo`: `localhost` 是主机名解析层面的约定(通常在 `/etc/hosts` 中映射到 `127.0.0.1`),而 `lo` 是内核层的网络接口对象。程序访问 `127.0.0.1` 时,内核将回环地址的流量交给 `lo` 处理。然而,`lo` 无法承载与宿主机或其他容器之间的流量。

要让容器与外部通信,必须为它的网络命名空间添加一个可以连接到宿主机网络栈的接口。Linux 提供的基础机制是 **veth (virtual ethernet) 对**。veth 总是成对创建,形成两个网络接口对象,它们之间满足一条稳定性质:从一端发出的二层以太网帧,会出现在另一端的接收方向;

反过来亦然。veth 对在逻辑上等价于一条点对点的二层链路，常用于连接两个不同的网络命名空间。

在容器网络中，veth 对的典型用法是：把一端放入容器的 Network Namespace 并命名为 `eth0`，另一端留在宿主机的网络命名空间。`eth0` 历史上表示“第一块以太网接口”；现代 Linux 发行版的物理网卡可能采用可预测命名（如 `ens33`、`enp0s3`），但容器运行时通常仍将容器内的主接口命名为 `eth0`，与宿主机上的物理网卡无关。配置完成后，容器通过 `eth0` 发出的数据包会经 veth 对到达宿主机网络栈；宿主机发往该容器的数据包也会通过 veth 对进入容器的 `eth0`。至此，容器拥有了一个可以连接外部的网络出入口。

veth 对为容器提供了一条点对点的二层链路，但单独一对 veth 只能在直连的两端之间搬运以太网帧。要让容器能够向任意 IP 地址发送数据包，还需要两项三层配置：一个 IP 地址作为容器在网络中的身份标识，以及一条默认路由告诉内核将非本地网段的流量交给哪一个下一跳。容器运行时通常会完成这些配置：为容器分配一个私网 IP，并设置默认路由指向宿主机侧的网关地址（这个网关地址位于容器网段内，由容器运行时预先配置）。配置完成后，单个容器具备了发送和接收 IP 数据包的完整条件：有接口、有地址、有路由。但如果宿主机上运行了多个容器，它们各自的 veth 宿主机端仍然是彼此独立的接口，数据包并不会自动在这些接口之间转发。换言之，每个容器都已具备一条独立的二层链路，但这些链路的宿主机端彼此隔离，尚未接入同一台交换设备。

6.2.2 同宿主机容器互联与 bridge 组网

在实际部署中，同一台宿主机上往往运行多个容器，例如战斗服容器、匹配服务容器和配置中心容器。这些容器之间经常需要通过网络交换数据：战斗服启动时要从配置中心拉取房间参数，匹配完成后要通知战斗服创建房间。上一节已经为每个容器建立了通往宿主机的 veth 链路，但这些链路的宿主机端彼此独立，容器 A 发出的帧不会自动到达容器 B。问题因此变为：如何把多条独立的 veth 链路汇聚起来，使同宿主机上的容器像处于同一个局域网中一样互通。

解决思路与物理网络相同：多台主机各自有网线，要让它们互通，需要把网线插入同一台交换机。在容器场景中，Linux 内核提供的 **bridge（网桥）** 承担了这个角色。bridge 是一台纯软件实现的**二层交换机**，容器运行时将每个容器的 veth 宿主机端绑定到 bridge 的一个端口上，所有容器就处于同一个二层广播域中。图 80 展示了同宿主机容器通过 bridge 互联的拓扑。图中以 Docker 默认创建的 `docker0` 网桥为例：两个容器各自通过 `eth0` 和 veth 对连接到 bridge，形成二层局域网。

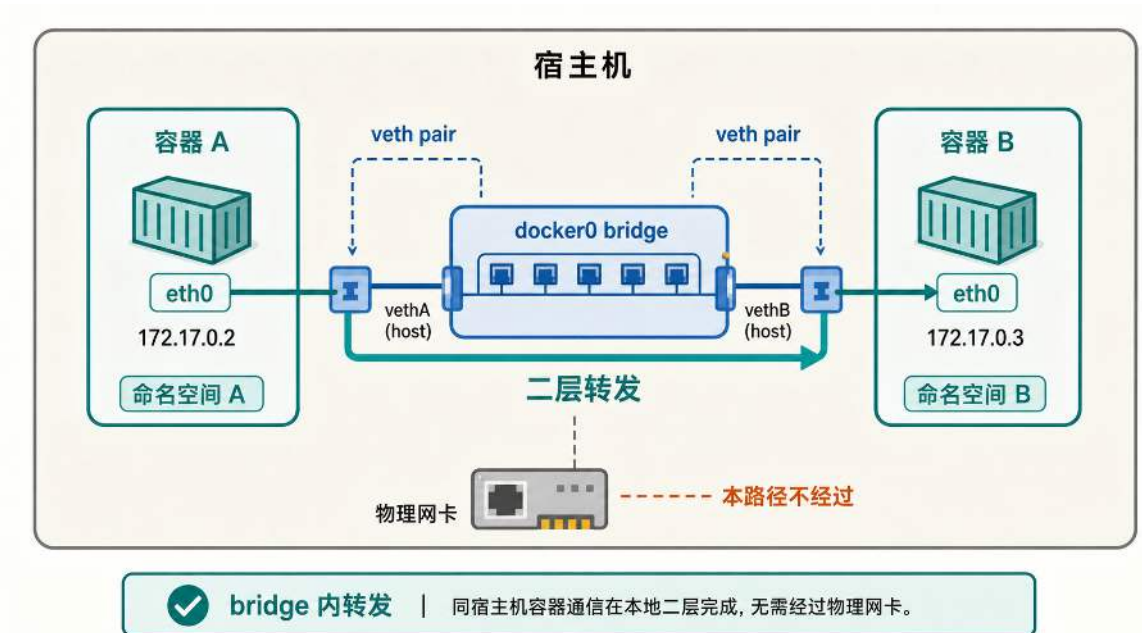


图 80: 同宿主机容器通过 veth 接入同一个 bridge，形成二层局域网。bridge 在端口之间转发以太网帧，使容器间可以按 IP 地址互相访问。

bridge 的转发机制决定了容器间通信的具体路径。bridge 内部维护一张 MAC 地址表，记录每个 MAC 地址最近一次从哪个端口出现。当某个端口收到以太网帧时，bridge 根据帧头中的目的 MAC 地址查表：如果目的 MAC 已有记录，帧只被转发到对应端口；如果尚未记录，bridge 将帧广播到除入端口外的所有端口。

以图中的容器间通信为例。容器 A (172.17.0.2) 要向同子网的容器 B (172.17.0.3) 发送 IP 数据包。IP 层判断目的地址 172.17.0.3 在本地网段内，不需要经过网关，但发送以太网帧需要知道目的 IP 对应的 MAC 地址。容器 A 通过 **ARP (Address Resolution Protocol, 地址解析协议)** 获取这一映射：A 发出一个广播帧，请求目的 IP 的持有者回复其 MAC 地址。该广播帧从 A 的 eth0 经 veth 进入 bridge，bridge 将其广播到除入端口外的所有端口，容器 B 收到后回复自己的 MAC 地址。A 获得 B 的 MAC 地址后，将数据帧的目的 MAC 填入帧头，从 eth0 发出。帧经 veth 到达 bridge，bridge 查表找到 B 对应的端口并转发；帧随后通过 B 的 veth 进入容器 B 的 eth0，交由 B 的协议栈处理。整个过程都在宿主机内核中完成，数据包直接在 bridge 内部完成二层转发，无需经过宿主机的物理网卡。

至此，同宿主机上的容器互联问题已经解决：veth 对为每个容器提供一条通往宿主机的二层链路，bridge 将这些链路汇聚为一张局域网并负责帧转发。在 veth 对已接入同一 bridge 且容器的 IP 地址配置在同一网段的前提下，它们即可通过二层转发直接互通。但容器通常还需要访问宿主机之外的服务，这就引出下一个问题：容器的数据包如何离开这张局域网，到达外部网络。

6.2.3 容器访问外网：默认网关、SNAT 与连接跟踪

上一节解决了同宿主机容器之间的互通，但容器通常还需要访问宿主机之外的服务，例如拉取依赖包、调用第三方 API 或连接外部数据库。此时容器面临的问题是：它的 IP 数据包如何离开由 bridge 组成的局域网，到达外部网络。

上一节已经说明，容器的默认路由会将非本地网段的流量指向宿主机侧的网关地址。现在需要回答的问题是：数据包到达网关后，如何继续走向外部网络，并让返回包最终回到容器？

在容器网络中，这个网关通常是 bridge 接口在容器网段中持有的 IP 地址。容器向外发送数据包时，先通过 ARP 获取网关的 MAC 地址，然后将帧从 eth0 经 veth 送入 bridge，进入宿主的网络栈。宿主机随后根据自己的路由表，将数据包从 bridge 接口转发到通往外部网络的物理网卡接口（这要求宿主机内核已启用 IP 转发功能）。至此，数据包离开了容器局域网。

但数据包能够发出，并不意味着通信能够完成。容器使用的是私网地址（例如 10.0.0.0/8、172.16.0.0/12、192.168.0.0/16），这些地址在公网或未配置容器网段路由的外部网络中不可路由：外部网络不会为它们维护路由条目，返回包无法按原路送回容器。因此，仅靠路由转发不足以完成外网访问，还需要在数据包离开宿主机时进行地址改写。

这一步由 **SNAT (Source Network Address Translation, 源地址转换)** 完成。宿主机在出口处将数据包的源 IP 从容器的私网地址改写为宿主机出口接口在外部网络中可达的地址。外部服务收到的请求看起来来自宿主机，因此返回包会先回到宿主机。Linux 提供了一种常用的 SNAT 规则形式：如果出口地址可能由接口动态决定，就不必在规则里写死一个源地址，而是让内核在转发时使用出口接口当前的地址完成改写，这种规则称为 **MASQUERADE**。在 Docker bridge 这类默认容器网络中，容器出网通常就是通过 iptables 或 nftables 中的 MASQUERADE 规则完成的。这条规则要回答三个问题：哪些包需要改写，改写成什么地址，以及改写后如何让返回包找到原来的容器。MASQUERADE 的核心动作如下：

MASQUERADE 的核心动作

```
1 func MasqueradeOutbound(pkt *Packet, out Iface) {
2     if !ContainerCIDR.Contains(pkt.SrcIP) {
3         return
4     }
5     if out.Name != HostExternalIface.Name {
6         return
7     }
8
9     original := pkt.FiveTuple()
10    pkt.SrcIP = out.CurrentIP()
11    pkt.SrcPort = NAT.AllocatePortIfNeeded(pkt.SrcPort)
12    rewritten := pkt.FiveTuple()
13
14    Conntrack.Remember(original, rewritten)
15 }
```

这段伪代码中的前两个判断说明，只有来自容器网段、并且即将从宿主机外部接口离开的包才需要被改写；中间两行改写源地址和必要的源端口，使外部服务看到的是宿主机出口接口；最后一行记录改写前后的五元组。这条记录正是返回路径成立的前提：宿主机收到返回包后，需要将目的地址从宿主机 IP 还原为发起请求的容器 IP，并通过 bridge 和 veth 将包送回正确的容器。这一还原过程依赖 **连接跟踪（connection tracking, conntrack）** 机制。内核在每次出向连接建立时记录一条映射：原始的容器 IP 与端口、改写后的宿主机 IP 与端口、以及对端的 IP 与端口。返回包到达宿主机时，内核在连接跟踪表中查找匹配条目，将目的地址还原后转发给对应的容器。SNAT 在改写源地址时将映射写入连接跟踪表，返回包到达宿主机时依据同一条目完成地址还原，二者共同完成双向通信。

这一机制也解释了容器网络中的一个常见现象：容器可以主动访问外网，但外部网络默认无法主动访问容器。出向 SNAT 只为容器发起的连接建立映射，外部网络不知道容器的私网地址；即使外部流量到达宿主机，宿主机内核的连接跟踪表中也没有对应条目可以匹配，这些数据包会被丢弃。如果需要从外部访问容器内的服务，必须通过额外的机制（例如端口映射或显式路由）为外部流量提供入口。

概括来说，容器访问外网依赖两层机制的组合：**路由转发**将数据包从容器局域网送往宿主机的出口接口，**SNAT 与连接跟踪**在出口处完成地址改写与还原，使外部网络能够将返回包送回宿主机，再由宿主机交还给正确的容器。

6.2.4 外部访问容器：端口映射与 DNAT

SNAT 只为容器主动发起的连接建立映射，尚未解决相反方向的问题：外部用户如何主动访问容器内的服务。外部流量要到达容器，面临两个障碍：容器使用的私网 IP 在外部网络中不可

路由，外部网络即使能到达宿主机也无法确定应将流量送给哪个容器。端口映射通过在宿主机入口处建立明确的转发规则来解决这两个问题。

端口映射的机制可以分为三步。第一步是**入口定位**：外部用户向宿主机的某个对外端口发起连接，例如访问 宿主机 IP:8080。宿主机的对外 IP 是公网可路由的，因此外部网络能够将数据包送达宿主机。第二步是**目的地址转换 (DNAT, Destination NAT)**：宿主机收到入向数据包后，在网络栈入口处将目的地址从 宿主机 IP:8080 改写为 容器 IP:80（假设容器服务监听 80 端口）。第三步是**转发到容器**：改写后的数据包经 bridge 与 veth 到达容器的 eth0，交给容器内的服务进程处理。

SNAT 与 DNAT 都发生在宿主机中，也都依赖连接跟踪完成往返，但二者不能混为同一条路径。图 81 把它们放在同一坐标系中比较：上半部分由容器主动发起连接，宿主机改写源地址；下半部分由外部用户主动发起连接，宿主机改写目的地址。两条返回路径都依据连接跟踪中保存的映射还原地址。

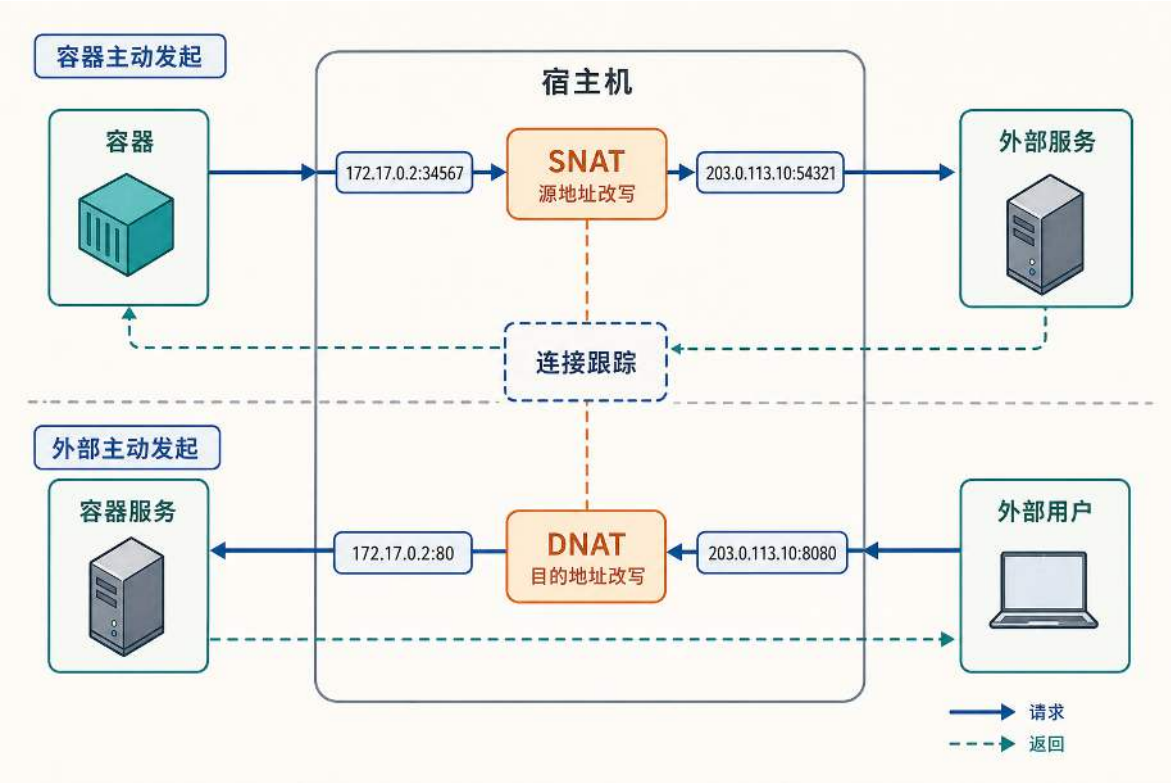


图 81: SNAT 与 DNAT 的方向对照：容器主动出网时改写源地址，外部主动入网时改写目的地址；两类连接都由宿主机记录映射，并在返回方向依据连接跟踪还原地址。

端口映射是双向的会话级转发，而不只是入向的一次改写。DNAT 建立后，宿主机通过连接跟踪记录这条连接的映射关系。返回数据包离开宿主机时，源地址需要从 容器 IP:80 还原为 宿主机 IP:8080，否则外部客户端会收到来自未知地址的响应，导致连接失败。因此，端口映射的完整语义是：**入向改写目的地址并送入容器，返回改写源地址并送回外部**。

这一机制也解释了几个常见问题。端口映射必须在宿主机上完成，因为外部流量首先到达宿

主机的对外 IP，容器的私网 IP 对外不可达，容器本身无法接收外部的入向数据包。映射以端口为单位，因为 TCP/UDP 连接通过四元组（源 IP、源端口、目的 IP、目的端口）标识，宿主机正是通过不同的对外端口将流量分流到不同容器。在简单的一对一端口映射中，同一个宿主机端口不能同时映射给多个容器，因为相同的入口地址对应不同的转发目标时，宿主机无法确定数据包应送往哪个容器（若需将同一入口端口对应多个后端，则需要额外引入负载均衡或服务代理层）。

概括来说，端口映射的作用是：**容器仍处于私网中，外部网络只能到达宿主机；宿主机通过 DNAT 与连接跟踪，将指定端口上的入向连接转发给目标容器，并将响应以宿主机身份送回外部。**

第五章介绍的 Kubernetes Service 位于更高的抽象层：它为一组会变化的 Pod 提供稳定地址，并按照后端列表选择目标实例。默认的 ClusterIP 主要服务集群内部访问；需要从集群外接入时，还要通过 NodePort、云负载均衡器、Ingress 或 Gateway 提供外部入口。流量进入节点后，可以由地址转换、路由、代理或其他数据面实现送往目标 Pod。因此，Service 解决的是“调用方通过哪个稳定入口找到一组后端”，DNAT 解释的则是其中一种底层地址转换机制，二者不能直接等同。

到这里，单台宿主机的两种边界通信已经完整：容器主动出网由 SNAT 建立返回路径，外部主动入网由 DNAT 建立明确入口。接下来讨论另一类问题：通信双方都位于集群内部，但 Pod 分布在不同宿主机上，此时需要解决 Pod 地址如何跨越物理网络。

6.2.5 跨宿主机的 Pod 互通：Overlay 网络与 VXLAN 封装

前几节解决了同宿主机容器互通以及容器网络的出入方向。但在 Kubernetes 集群中，服务通常分布在多台宿主机上，不同宿主机上的 Pod 之间同样需要按 IP 地址直接通信。此时的核心问题是：**不同宿主机上的 Pod 地址如何彼此可达。**

单机内部的 bridge 和 ARP 广播域仅限于本机。一旦跨出宿主机，数据包进入机房的三层物理网络，由路由器按宿主机 IP 转发。物理网络通常不会为每个 Pod 的地址维护路由条目，也不会将 Pod 网段的二层广播跨宿主机扩散。因此，Node1 上的 PodA 发往 Node2 上 PodB 的数据包，仅靠本机的 veth 和 bridge 无法送达对端。

一种直接的思路是将每个节点的 Pod 子网作为路由前缀发布到机房物理网络中，使路由器直接转发 Pod 地址。这在原理上可行，但在通用集群中会带来工程代价：如果按节点级 Pod 子网发布路由，节点加入、退出或 Pod 子网重新分配时，底层网络需要同步更新；如果进一步细化到每个 Pod 或更小粒度的路由，Pod 迁移和扩缩容又会把更高频的虚拟端点变化暴露给物理网络。物理设备的转发表容量、路由收敛速度和运维边界都不适合无限承接这类变化。另一种常见的工程取舍是：**物理网络只负责转发宿主机之间的三层流量，Pod 的互通由节点上的软件在边界处完成。**

这一取舍对应云原生网络中的两个概念。在本节采用的 VXLAN Overlay 模型中，**Underlay** 是机房的底层网络，只需要转发节点或 VTEP 地址，无需维护 Pod 地址的转发状态。**Overlay** 则是在 Underlay 之上构造的一张逻辑网络，使 Pod 的地址在集群范围内可达。VXLAN 是常见的 Overlay 实现之一。

VXLAN 的核心机制是封装与解封装，它将两层网络的职责分开：**物理网络中只出现节点到节点的流量，通信两端仍保持 Pod 到 Pod 的地址语义**。每个节点上都会配置一个 VXLAN 隧道端点，称为 **VTEP (VXLAN Tunnel Endpoint)**；它可以由内核 VXLAN 设备、虚拟交换机或网络插件配置出来，不一定是一个独立运行的用户态进程。当本机某个 Pod 发出的数据包的目的地址不在本机时，数据包不会直接交给物理网卡，而是被路由到本机的 VTEP。VTEP 把 Pod 之间完整的**内层以太网帧**整体作为载荷，外面再套上一层“节点 IP + UDP + VXLAN”报头（其中外层 IP 以节点地址为源和目的），使物理网络能够按宿主机路由转发。对端节点的 VTEP 收到后执行解封装，剥掉外层报头，还原出原始的 Pod 间以太网帧，再交给本机的 bridge 和 veth 送入目标 Pod。

图 82 把这一分工具体化：外层用节点地址供物理网络转发，内层保留 Pod 地址在两端不变。数据包因此在物理网络里以节点身份穿行，到两端再还原成 Pod 到 Pod 的通信，物理网络自始至终看不到 Pod 地址。

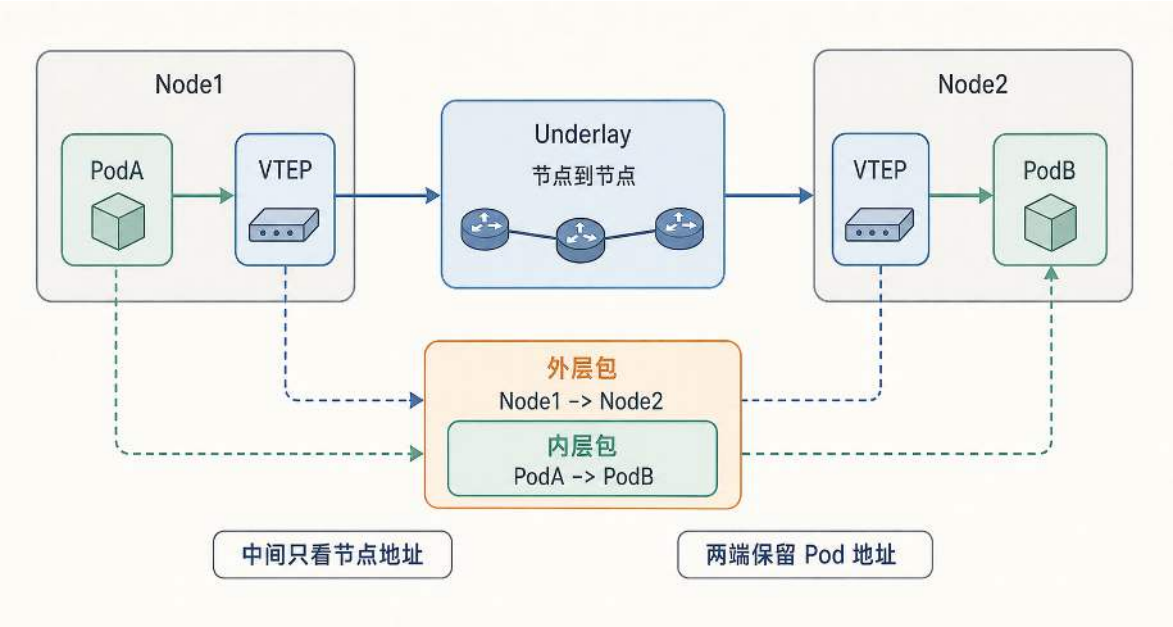


图 82: VXLAN 封装结构：内层保留 Pod 到 Pod 的地址语义，外层使用节点到节点的地址，使物理网络无需感知 Pod 地址。

以一次具体的跨节点通信为例。PodA 位于 Node1，PodB 位于 Node2。PodA 发往 PodB 的数据包从 PodA 的 `eth0` 经 veth 进入 Node1 的网络栈。Node1 的路由判断目的 Pod 不在本机子网内，将数据包交给 VTEP。VTEP 不改变内层以太网帧所承载的 PodA 到 PodB 的 IP 包，而是在整个内层以太网帧外添加以 Node1 为源、Node2 为目的的外层报头。封装完成后，物理网络将其作为一个普通的节点间报文转发到 Node2。Node2 的 VTEP 执行解封装，去掉外层报头，还原出原始的内层以太网帧。此后回到单机路径：帧进入 Node2 的 bridge，经 veth 送入 PodB 的 `eth0`，再由 PodB 的协议栈处理其中的 IP 包。从端到端看，PodA 与 PodB 始终按 Pod 地址通信；从物理网络看，跨宿主机这一跳只是节点到节点的常规转发。**主机内互通由**

veth 和 bridge 完成，跨主机这一跳由 VXLAN 的封装与解封装补齐。

VTEP 在封装时需要知道目的 Pod 所在的节点地址。这依赖一张映射表：在 Kubernetes CNI 的教材化理解中，可以把它看成“Pod 地址段到节点/VTEP 地址”的对应关系；在更底层的 VXLAN 语义中，它也会涉及内层二层标识、VNI 与远端 VTEP 地址的对应。这里最关键的是：远端地址不是公网意义上的“外部 IP”，而是该节点在 Underlay 网络中可达的节点地址或 VTEP 地址。集群的控制面（例如 Kubernetes 的网络插件）负责维护和分发这类映射，VTEP 按表完成封装。**控制面维护并分发映射表，数据面按表完成封装、转发与解封装。**

封装还会带来一个附加影响：同样的业务数据在外层报头的包裹下，在链路上占用更多字节。链路一次能够承载的最大报文长度称为 MTU (Maximum Transmission Unit, 最大传输单元)。如果封装后的报文长度超过物理链路的 MTU，系统可能需要将报文拆成更小的片段转发，或者在某些配置下直接丢弃。因此，使用 Overlay 网络时通常需要适当调低容器侧的 MTU，为外层报头预留空间。

至此，容器网络的主机内互通、出向访问、入向访问和跨主机互通已经分别建立。它们解决的是不同通信范围和发起方向的问题，不能把 SNAT、DNAT 与 VXLAN 当作同一目的下的替代方案。

6.2.6 低开销的容器网络：Overlay 封装的代价与前沿优化

前述 Overlay 网络用 VXLAN 把每一个跨宿主机的数据包重新封装一次，让分布在多台机器上的容器像在同一张二层网络里一样互通。这层封装换来了可移植性，但会带来额外开销：每个数据包在发送端要被封装、在接收端要被解封装，封装后的包还要在宿主机内核的网络协议栈里多走一遍。当容器间通信频繁、吞吐要求高时，这部分固定开销会直接体现在延迟、吞吐和 CPU 占用上。

具体而言，许多典型 Overlay 实现会在内核里走**逐包转换**：对每个包做连接跟踪、包过滤、路由查表，再套上外层 VXLAN 头。结果是同一个包要在发送端和接收端各自的内核网络栈里穿行两次——一次处理容器看到的内层地址，一次处理宿主机之间传输用的外层地址。对一条已经建立的连接来说，这些处理每个包都要重复执行，即使结果变化很小。针对这种固定开销，研究者给出了两条思路，都可以落回本书关心的高频交互面。

第一条思路是把虚拟化从“每个包”提升到“每条连接”。Slim³⁰的做法是：只在连接建立时改写一次连接级的元数据，之后这条连接上的数据包就直接走宿主机的网络栈，不再逐包封装，每个包只穿越内核网络栈一次而不是两次。论文报告，在内存键值存储场景下，这一改动把吞吐提升约 66%、延迟降低约 42%、CPU 占用降低约 54%。代价是它工作在连接级，需要专门的组件介入 socket 建连过程，改变了标准 socket 的部分行为。

连接级改写还引入了一个更直接的新代价：建立连接时，收发双方要额外做一轮往返，交换宿主机 socket 与容器 socket 的映射信息，连接建立因此变慢。对需要高速处理大量短连接的应

³⁰Danyang Zhuo 等, *Slim: OS Kernel Support for a Low-Overhead Container Overlay Network*, NSDI 2019, pp. 331–344. <https://www.usenix.org/conference/nsdi19/presentation/zhuo>

用（如代理、缓存），这笔建连开销会盖过数据传输上省下的收益。SlimFast³¹正是针对这一点，在保留 Slim 低数据开销的同时优化了建连流程，让短连接也能受益；论文报告，相较 Slim，在短连接实验中，SlimFast 将 Nginx 代理吞吐提高约 89%，并将 Memcached 响应率提高约 219%。可以看到，连接级这条路线内部也在推进：先用 Slim 消除逐包封装，再用 SlimFast 补上它带来的建连开销。

第二条思路是保留标准 Overlay 的数据路径，但把其中“每个包都重算、结果却不变”的部分缓存下来。ONCache³²观察到：一条连接存活期间，连接跟踪结果、包过滤结果、主机内路由决策和外层封装头这几项在正常情况下都是不变的。于是它用 eBPF 在收发路径两端各挂一段程序，第一个包正常走完整路径并把这些结果缓存起来，后续包命中缓存后直接套用缓存的外层头、按缓存的路由转发，跳过重复处理。整套快速路径只约 524 行 eBPF 代码，可以作为 Antrea、Flannel 等标准 Overlay 的插件叠加上去；一旦缓存不可用或网络发生变化（容器删除、迁移、过滤规则变更），流量自动回退到标准路径，保证正确性。论文报告 TCP 吞吐和请求-响应事务率分别提升约 12% 和 36%，同时显著降低每包 CPU 开销。

图 83 将三种路径并列起来：标准 Overlay 把封装、路由和过滤留在逐包路径上，Slim/SlimFast 将改写提升到连接建立阶段，ONCache 则让首包建立流级缓存并让后续包复用结果。三者的比较重点在于重复处理被移出逐包数据路径的方式：连接级改写更深地介入内核与 socket 路径，缓存旁路更接近对标准 Overlay 的加速插件，二者对应不同的兼容性和适用场景。

³¹Fusheng Lin 等, *Slim and Fast: Low-Overhead Container Overlay Network With Fast Connection Setup*, IEEE Transactions on Cloud Computing, Vol. 12, 2024, pp. 1–12, DOI:10.1109/TCC.2023.3282238. <https://ieeexplore.ieee.org/document/10143220>

³²Shengkai Lin 等, *ONCache: A Cache-Based Low-Overhead Container Overlay Network*, NSDI 2025, arXiv:2305.05455. <https://arxiv.org/abs/2305.05455>

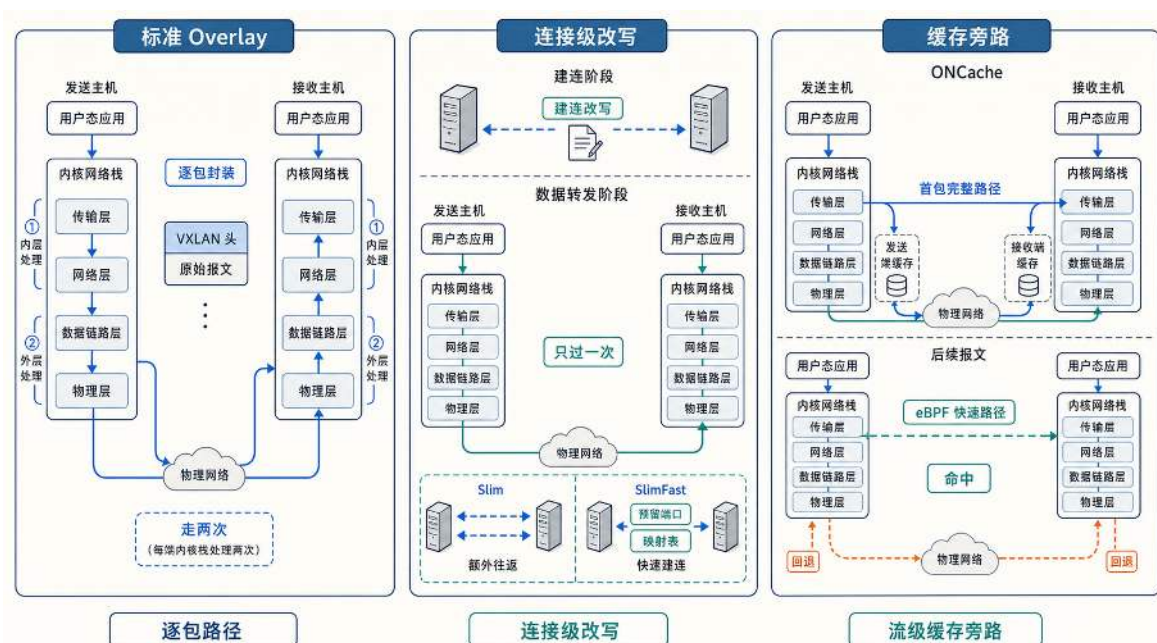


图 83: 三种容器 Overlay 网络处理路径的对比：标准 Overlay 将封装、路由和过滤留在逐包路径上，Slim 与 SlimFast 将虚拟化改写提升到连接建立阶段，使数据转发阶段减少重复的内核栈处理，ONCache 则缓存每流不变量，让后续包复用首包得到的处理结果。图揭示的是两类降低固定开销的思路：连接级改写与缓存旁路都在把重复处理从逐包数据路径中移出，但二者在侵入性、兼容性和适用场景上不同。

这两条优化思路都可以回收到本章的交互面：当大量隔离环境持续交换动作、观察和同步消息时，跨主机通信中的固定开销会被交互频率放大。Overlay 的通用封装换来了可移植性；连接级改写和缓存旁路则把部分通用处理移出高频路径，代价是更强的内核依赖和兼容性约束。后文讨论 Agent sandbox 时仍会遇到同一类取舍：交互越密集，越需要判断哪些通用层次可以保留，哪些固定开销必须下压。

6.2.7 小结：网络虚拟化的层次结构

容器网络需要依次解决四个递进问题：隔离各自的网络栈、建立主机内互通、打通外网出入口、实现跨宿主主机通信。对应这四个问题，网络虚拟化可以归纳为四个层次。

第一层是**分栈**：Network Namespace 将网络协议栈切分为多份，同一 Network Namespace 内的进程共享一组独立的接口集合、地址、路由与端口空间。分栈之后，每个网络栈最初只有回环接口 lo，隔离并不会自动产生互联。

第二层是**连线与组网**：veth 提供点对点的二层连接，将不同命名空间的接口端点接回宿主机；bridge 将多条二层连接汇聚为一张虚拟局域网，使同局域网内的容器通过 ARP 与 MAC 地址表完成直接互通。这一层完成的是主机内部的局域网虚拟化。

第三层是**出入口控制**：宿主机在虚拟局域网与外部网络的边界处承担路由转发角色，通过

SNAT 与连接跟踪使容器能以宿主机身份访问外网，通过 DNAT 与端口映射为外部流量提供入口。

第四层是**跨主机互通**：当容器分布在多台宿主机上时，Overlay 网络（如 VXLAN）通过封装与解封装，在物理网络之上构造一张逻辑网络，使不同宿主机上的容器地址（在 Kubernetes 中对应 Pod 地址）在集群范围内可达。

在此基础上，针对 Overlay 封装带来的固定开销，研究者还提出了连接级改写与缓存旁路等优化思路，它们不增加新的网络层次，而是把重复处理移出高频数据路径。

网络虚拟化通过隔离网络栈、构造二层连接、提供二层汇聚、在边界做三层转发与地址转换、在集群范围内做隧道封装，让同一台或多台物理主机上出现多个逻辑独立且可控互联的网络环境。

6.3 弹性伸缩的应用基础：无状态与有状态

上一节的网络虚拟化解决了实例之间如何互通，但实例能够互通，并不意味着应用能够安全地增减实例。第五章已经从业务场景说明了为什么需要弹性伸缩，并介绍了 HPA、Cluster Autoscaler 和 Serverless 等使用方式。本节进一步讨论应用侧的前提：平台增加一个副本后，这个副本怎样真正承担业务负载；平台回收或替换副本时，应用又怎样保持业务正确性。

副本数增加并不等于有效容量增加。平台创建和回收的是执行实例，而应用实际分配和保护的是队列分片、房间、会话等状态单元；只有当状态单元的处理权与新实例对应起来时，新增副本才真正分担负载。一个新实例要走到这一步，至少要通过三道门。第一道是**资源与实例就绪**：平台已经为实例分配计算资源，应用也完成了配置加载、依赖连接和缓存预热。第二道是**流量接入**：服务发现和请求路由已经把新实例纳入可用路径。第三道是**状态接管**：如果请求绑定队列分片、房间或其他状态单元，新实例还要获得相应的处理权。实例只通过第一道门，只能说明进程已经运行；通过前两道门，才可能收到请求；三道门全部通过，有状态服务的新增副本才会形成有效容量。

这三道门主要回答扩容时新实例如何进入服务。安全伸缩还要处理实例退出的过程：计划缩容时有排空机会，故障时则可能突然中断。此外，一次请求在结果不明确时还可能被重复投递，应用必须保证重试不会破坏业务正确性。因此，弹性伸缩不仅要让新实例顺利接入，还要定义旧实例怎样安全退出、故障后怎样恢复、以及重复请求怎样保持幂等。下面先从不需要交接业务状态的可替换实例开始，再讨论有状态服务怎样跨过第三道门。

6.3.1 无状态与可替换实例：安全水平伸缩的六项条件

到达的请求如果可以交给任意一个可用实例处理，不依赖某个实例独占的本地业务状态，这类服务通常称为**无状态服务**。无状态并不意味着服务完全不使用状态，而是指需要保留的状态不绑定在特定实例的本地内存或文件系统中。实例因此可以被替换，不需要先把一份业务状态的修改权交给新实例。对无状态服务而言，第三道门被简化了，扩容主要取决于新实例能否完成初始化并进入流量路径。

让无状态服务能够安全地水平伸缩，需要满足六项设计条件。需要先厘清的是，真正定义“无状态”的只有状态外置一项：它决定实例是否可以被任意替换。其余五项并不属于无状态的定义，而是让水平伸缩能够安全进行的配套条件。为便于理解，可以把这六项按解决的问题分成三组。

第一组是**状态与实例**，它决定服务本身是否真正无状态。

状态外置。所有需要在实例重启或迁移后仍然保留的数据，必须写入外部存储（数据库、缓存或对象存储），不能仅存于进程内存。例如玩家签到记录、订单信息和持久化配置都应写入 PostgreSQL 或 Redis；实例本地内存中只允许保留可重建的缓存或临时计算结果。只有做到这一点，任意实例才能接手任意请求，实例才成为可替换的单元。

有些系统会让负载均衡器把同一用户长期送往同一实例，以便继续使用本地会话。这种做法称为**会话粘性**。它可以减少会话读取开销，却没有消除状态与实例的绑定：实例故障后，本地状态仍会丢失；实例负载不均时，请求也不能自由转移。会话粘性可以作为性能优化，但不能替代状态外置或状态恢复协议。状态外置同样不是没有代价，它会把一部分本地访问变成远程访问，并使数据库或缓存成为共享依赖；设计者还要为外部存储准备容量、故障恢复和一致性机制。

第二组是**流量接入**，它决定新增或替换的实例能否被正确地接入流量。

稳定入口。客户端始终连接一个固定地址（域名、虚拟 IP 或网关），由入口层根据后端实例列表动态路由。实例增减时，客户端不需要更换目标地址。Kubernetes Service 或云负载均衡器承担的就是这一角色。

服务发现。入口层需要及时获得当前可用的后端实例集合。成员信息既可以由实例主动注册，也可以由平台控制器根据实例状态维护；在 Kubernetes 中，平台根据 Pod 的标签与就绪状态更新 Service 的后端集合。没有服务发现，新扩容的实例无法承接流量，已经失效或尚未就绪的实例也可能继续收到请求。

就绪检查。实例启动后并非立刻可用：可能需要加载配置、预热缓存、建立数据库连接。平台通过就绪探针确认实例真正具备处理能力后，才将其加入服务发现列表。跳过就绪检查会导致请求被发给尚未准备好的实例，引发超时或错误。

第三组是**重试与生命周期**，它保证在重复投递和实例退出时业务仍然正确。

幂等性。同一请求被多次处理时，结果应当与处理一次相同。平台在故障恢复或网络重试时可能重复投递请求；如果服务不具备幂等保护，重复请求就会变成重复发奖或重复扣款。典型的幂等实现是为每次业务操作生成唯一键，并用数据库唯一约束或条件写入把“检查是否已处理”和“记录已处理”合并为原子动作。

优雅下线。实例终止前需要先停止接收新请求，把已接收的请求处理到可安全中断的位置，再退出进程。对短请求服务，这意味着完成存量请求；对长连接服务，这意味着等待连接自然关闭或完成会话迁移。平台提供终止信号和超时窗口，但何时能够安全退出由应用定义。

这六项条件并不是同一层面的属性，而是分别作用于实例生命周期的不同阶段。状态外置使实例成为可替换单元；就绪检查保证实例通过第一道门，稳定入口与服务发现使它通过第二道门；幂等性处理结果不明确后的重复请求；优雅下线则保证实例退出时先处理存量工作。一个没有为伸缩设计过的应用，即使能够被平台复制，也不一定能形成有效容量；即使状态已经外置，缺少

幂等保护或优雅下线，重试和缩容仍可能破坏业务正确性。

6.3.2 有状态服务：状态归属决定伸缩方式

并非所有服务都能做成无状态。游戏战斗服维护的对局状态、匹配队列中的等待关系、实时协作系统正在编辑的文档，都要求系统持续保存并协调其变化。这类服务称为**有状态服务**。平台可以为它们创建新实例，但新实例能否形成容量，取决于应用能否把状态单元及其处理权交给新实例。

分析有状态服务时，不应先根据服务名称判断它“属于哪一类”，而应围绕权威来源、修改权、状态粒度、生命周期和变化处理五个问题，逐步确定状态怎样随实例交接。这五个问题并非彼此孤立，可以沿着“正确性依据、分配方式、变化处理”的顺序展开。

首先要确定系统依据什么维持正确性。发生分歧或故障恢复时，哪一份记录是权威来源；系统正常运行时，又由一个实例独占修改权，还是允许多个实例并发修改。前者决定恢复时以什么为准，后者决定并发更新和所有权转移需要采用什么机制。若某一时刻只能有一个所有者，转移修改权时还必须阻止旧所有者继续写入。

确定正确性依据后，还要说明状态怎样分配给实例。系统分配和迁移的是一名用户、一场对局、一个队列分片，还是整个数据库分片；粒度越小，负载越容易均衡，但需要维护的归属记录和路由规则也越多。状态的生命周期又决定了这种分配能否及时调整：短请求产生的临时状态可能很快释放，一场对局或一次协作会话则会持续较长时间，实例退出时未必能够等待它自然结束。

最后才能选择实例变化时的处理方式。扩容时，新实例是重新计算状态，还是接管已有状态；计划缩容时，存量状态是等待排空，还是在线迁移；实例故障时，又要从哪一份持久记录恢复。不同选择会改变服务中断时间、实现复杂度和资源开销，也决定新增实例能否真正形成容量、退出实例会不会破坏业务正确性。

根据这些问题的主要回答，工程上可以归纳出四种常见的状态特征。它们帮助设计者识别主要风险，但并不是互斥的服务分类；同一份状态可能同时具有其中多种特征。

可重建缓存：例如从数据库加载的玩家基础信息缓存、热点商品列表。这类状态丢失后可以从持久存储重新加载，只是会短暂增加延迟。伸缩时不需要专门迁移，新实例启动后自行重建即可。

连接与会话状态：例如 WebSocket 连接与玩家会话的对应关系、正在进行的对局上下文。这类状态如果丢失，相关连接或会话会被直接中断。伸缩时需要决定：等待存量会话自然结束，还是把会话状态迁移到新实例；前者实现简单但下线较慢，后者切换更快但需要恢复连接和状态的协议。

共享协调状态：例如匹配队列、协作文档和房间归属表。它们会被多个用户或实例共同读写，主要问题是规定更新顺序、处理并发冲突和转移修改权。单纯把数据存入外部数据库并不能自动解决这些问题，应用仍需选择锁、事务、版本号或共识等协调机制。

权威持久状态：例如账户余额、已完成订单和角色资产。这类状态是后续业务判断的依据，主要问题是已经确认的结果不能丢失，恢复后也不能退回更早的值。它通常由数据库或分布式存储系统保管；系统是否允许副本暂时出现差异、读操作能看到多新的值，则由所选一致性模型决

定。

共享协调与权威持久描述的是两个不同维度。前者关注“并发修改时怎样形成唯一有效的顺序或所有者”，后者关注“系统恢复后以什么记录作为事实”。匹配队列在运行时需要协调多个玩家和处理实例，它记录的“玩家是否仍在等待”也可能是匹配阶段的权威事实；账户余额必须持久保存，同时也需要事务或锁来协调并发扣减。因此，四种特征不能代替前面的五个问题。

有状态服务的伸缩还要区分两个对象。**伸缩单元**是平台创建和回收的实例，**状态单元**是应用分配和保护的话、房间、任务或分片。增加一台战斗实例，只是增加了一个伸缩单元；只有把新的对局或可迁移的既有对局分配给它，系统才增加了实际对局容量。状态单元的粒度、所有权和生命周期，决定了伸缩单元怎样通过第三道门。

当状态量大到单台机器无法承载时，系统需要**分片**（sharding）：把状态按某个键（如玩家ID、房间号、地理区域）划分到不同实例。分片使每个实例只负责一部分状态，从而让总容量随实例数增长。但分片也带来新的约束：请求必须按分片键路由到正确实例；扩容时需要**数据重平衡**，把部分分片从旧实例迁移到新实例；缩容时则需要把被回收实例上的分片迁走。

如果状态数据保存在原实例本地，数据重平衡通常要经过四个步骤。第一步复制一份**基础快照**，让目标实例获得某个时刻的完整起点；如果没有这一步，目标实例就只能从空状态开始。第二步追平复制快照期间产生的**增量更新**，把新旧实例之间的差距缩小到可以切换的范围。第三步切换**状态归属与请求路由**，使后续请求到达新所有者。这一步对应的正是状态分析五问中的第二问——修改权的转移：为防止旧所有者在网络延迟或暂停恢复后继续写入，可以为每次归属分配一个单调递增的版本号，存储端只接受当前版本的写入；这种用新版本阻挡旧所有者的编号称为**隔离令牌**（fencing token）。第四步验证新实例能够稳定处理读写，再清理旧副本；过早清理会使切换失败后失去回退路径。

并非每次状态接管都要复制大量数据。如果匹配玩家已经保存在共享队列中，扩容时可以只转移队列分片的处理权、更新隔离令牌和路由；如果数据只存在原实例本地，才需要走完快照、增量追平和切换过程。只有当迁移同时改变某个共识组的副本集合时，才需要使用第四章介绍的Raft 成员变更机制。有状态伸缩的共同点不是采用同一种迁移算法，而是让状态数据、修改权和请求路径完成一致的交接。

6.3.3 案例：游戏服务端的状态架构与弹性伸缩

前两小节分别建立了有效容量的三道门和状态分析五问。下面沿“登录与大厅、匹配、战斗、结算”这条业务路径，观察同一个游戏服务端怎样处理扩容、缩容和故障替换。

先观察扩缩容为何失效 活动开始后，在线人数快速增加，匹配队列长度和等待时间持续上升。平台根据负载指标增加了大厅、匹配和战斗实例。新增大厅实例完成初始化并进入服务发现列表后，能够分担登录和查询请求；新增匹配与战斗实例虽然已经运行，匹配吞吐却没有同步提高，新实例的资源利用率也一直很低。原因是应用仍使用部署时写定的匹配分片归属表和房间分配规则：队列分片仍由原有匹配实例处理，新对局也仍被分配给原有战斗实例。新增副本通过了资源与实例就绪这道门，却没有完成状态接管，因而没有形成有效容量。

另一次流量回落后的缩容又产生了另一类故障。平台选择一台战斗实例并终止进程，但应用没有先停止分配新对局，也没有等待正在进行的对局结束。该实例上的玩家连接随进程一起断开，对局上下文又只存在本地内存，替代实例无法恢复。这说明缩容不是扩容的逆操作：扩容关注新实例如何接管工作，缩容必须先使旧实例失去新工作的入口，再处理它已经拥有的状态单元。

用状态分析五问拆解业务路径 玩家资料的权威来源是数据库，大厅实例只保存可重新加载的缓存，缓存的生命周期可以与实例一致。登录连接由当前网关持有，连接断开后可以由客户端重新建立，但用户会话和目标战斗服地址要保存在外部存储，不能只存在网关内存中。

匹配阶段的状态单元是队列分片。共享队列记录哪些玩家仍在等待，分片归属记录当前哪个匹配实例拥有处理权；它们既涉及并发协调，也是匹配阶段作出后续判断的依据。战斗阶段的状态单元则是一场对局。对局从房间创建开始，到结算结果确认后结束；在这段时间内，某个战斗实例负责推进状态。结算结果和角色资产由数据库作为权威来源，并以“对局标识与玩家标识”等唯一业务键识别重复提交。

经过这一步，系统不再按“大厅服无状态、战斗服有状态”给组件贴标签，而是明确了每份状态的权威来源、当前所有者、分配粒度、生命周期和实例变化时的处理方式。

让新增实例获得匹配分片 系统按照游戏模式和玩家所在区域等稳定字段划分匹配队列，把待匹配玩家写入按分片组织的共享队列，并用一份共享的归属记录说明每个分片当前由哪个实例处理。归属调整不能只看实例数量，还要观察各分片的等待人数、请求到达速率和处理速率，再选择需要转移的分片，使新旧实例承担的工作回到各自可处理的范围内。

增加匹配实例时，新实例先完成初始化并声明就绪，请求入口随后才能把它纳入可用成员集合。负责调整归属的控制逻辑再把部分分片交给新实例，并为新的归属生成更高版本的隔离令牌。旧实例停止处理这些分片，共享队列或它前面的写入接口拒绝携带旧令牌的请求；路由更新后，新请求才到达新所有者。至此，新实例依次通过资源与实例就绪、流量接入和状态接管三道门，获得了可以实际处理的队列分片。

让新增战斗实例承接新对局 战斗服务不必为普通扩容迁移正在进行的对局。创建房间时，匹配服务从已经就绪且未进入排空状态的战斗实例中选择目标，把房间与实例的对应关系写入共享的房间归属记录。扩容后，新实例进入候选集合，房间分配逻辑根据各实例尚可承载的对局数，把更多新对局送往负载较低的新增实例；已经开始的对局继续由原实例推进，直到完成结算。

这种方案把一场对局作为不可拆分的状态单元，优点是扩容不必暂停现有对局，也不必在高峰期额外传输大量状态。代价是容量生效速度受新对局产生速度影响：如果少数对局持续时间很长，旧实例可能长期保持较高负载。只有当对局持续时间超过可接受的排空期限，或者必须在限定时间内维护某台机器时，系统才需要考虑主动迁移。主动迁移要选择一致状态点生成快照、追平增量操作、切换房间所有权和路由，并恢复连接、定时器与未完成消息；这些开销可能进一步加重高峰压力，因此它是带条件的进阶方案，而不是常规扩容的默认步骤。

分别处理安全缩容与故障替换 计划缩容时，平台开始把实例移出新流量路径，同时发出终止通知并开始计算终止宽限期。应用立即把目标实例标记为排空状态；即使路由更新尚未传播完成，也不再接受新的状态单元。匹配控制逻辑停止向它分配新分片，房间分配逻辑也不再创建新对局；已有匹配分片完成所有权交接，正在进行的对局则等待自然结束。状态单元数量降为零后，应用主动退出；若直到宽限期结束仍未排空，平台会强制终止实例。因此，设计阶段必须根据状态单元的持续时间设置宽限期，并明确无法在期限内排空时，是迁移剩余状态还是中断超时任务。安全退出点由业务状态决定，宽限期则给这一过程设置了平台能够接受的时间上限。

故障替换没有这样的排空阶段。平台通过健康探测、节点状态或心跳发现实例失效并创建替代实例，但平台并不知道一场对局进行到了哪一步。状态控制逻辑必须先使旧实例的租约失效，或提升隔离令牌版本，防止暂时失联的旧实例恢复后继续写入。如果业务只承诺连接可以重建，不承诺对局恢复，那么故障会中断受影响的对局；如果业务要求继续对局，就必须事先保存快照或操作日志，由新实例恢复状态、取得更高版本的所有权，再更新房间路由并允许玩家重连。平台重新创建实例只能恢复一个可运行的执行环境，能否恢复业务取决于应用事先保留了什么状态和恢复协议。

用运行结果验证状态架构 改造完成后，不能只检查副本数是否达到期望值。扩容验证还要确认新实例是否通过就绪检查、是否收到请求、获得了多少队列分片和新对局，以及队列长度和等待时间是否随有效容量增加而下降。缩容验证要观察待回收实例上的分片数和进行中对局数是否持续降为零，并记录排空所需时间。结算链路还要检查重试后是否出现重复资产变更。这些是应用侧的业务验证指标，关注的是状态架构是否真正配合了扩缩动作，与第 5.3 节讨论的平台侧伸缩触发指标（如 CPU 利用率、请求速率）处在不同层面。只有业务状态和运行指标同时符合预期，才能说明新增副本转化成了有效容量，退出实例也没有破坏正确性。

这个案例得到的结论可以迁移到其他在线系统：**弹性伸缩不要求所有组件都无状态，关键是让状态单元的粒度、所有权和生命周期与实例的创建、接入、退出和替换过程相互配合。**

6.3.4 应用与平台的职责划分

上述案例说明，应用与平台的职责不能只按组件静态划分，而要在实例生命周期中相互衔接。

扩容阶段，平台根据容量决策创建实例、分配资源并提供服务发现机制；应用完成配置加载、依赖连接和缓存预热，通过就绪条件说明自己已经可以处理请求。平台把就绪实例加入通用流量入口后，应用或专用状态控制器还要分配队列分片、房间等状态单元，并更新相应的业务路由。平台能够决定“需要多少个实例”和“实例放在哪里”，应用则必须说明“新实例可以处理什么状态”。

缩容阶段，平台选择待回收实例，启动流量摘除并发出终止通知；应用随即进入排空状态，在路由更新传播期间也不再接收新的状态单元，转移可迁移的所有权，并等待其余存量工作到达安全退出点。平台负责提供生命周期通知、终止宽限期和最终回收资源的机制，应用负责用业务状态判断何时可以主动退出，并保证无法在期限内排空时有明确的处理策略。固定的等待时间只能提供上限，不能代替这个判断。

故障替换阶段，平台负责检测实例失效、隔离故障位置并创建替代实例；应用或专用控制器负责废止旧所有权、从权威来源恢复状态、执行幂等重试并更新业务路由。数据库、存储系统或 Operator 可以自动执行复制和重平衡，但它们仍要遵守应用定义的一致性要求、状态粒度和切换条件。

因此，平台提供的是实例生命周期和通用资源治理能力，应用提供的是状态语义和安全转换条件。任何一侧缺失，副本变化都可能停留在“进程数量改变”这一层，无法变成安全、有效的业务伸缩。**弹性伸缩必须进入应用架构设计本身：从设计之初就把实例的增加、退出和失败视为正常变化，并为这些变化准备状态接管、排空与恢复流程。**

接下来，在应用已经满足上述条件的前提下，平台内部如何把负载变化转换为调度决策，是下一节的主题。

6.4 弹性调度的内部机制：从反馈控制到按需执行

上一节说明应用需要满足哪些条件才能被安全伸缩。本节进入平台内部，分析控制器如何把负载变化转换为调度决策：观测哪些指标、如何抑制噪声、怎样分层执行、以及 Serverless 如何把同一套容量管理继续下沉到请求级执行环境。

6.4.1 调度对象与分层

弹性调度需要调节的对象不止一种。HPA 调节的是同类服务实例的数量；VPA 调节的是单个实例的资源规格；Cluster Autoscaler 调节的是集群节点数量；有状态系统的分片控制器调节的是分片或副本的分布；Serverless 调节的则是函数执行环境的创建与回收。不同对象对应不同层级的资源，也对应不同类型的延迟和约束。

这些调节器通常按分层结构组织。上层控制器（如 HPA）根据业务指标决定“需要多少容量”；中层调度器（如 Kubernetes Scheduler）根据资源约束决定“把新实例放到哪台机器上”；下层基础设施控制器（如 Cluster Autoscaler）根据节点资源余量决定“是否需要申请新机器”。上层决策向下层传导，下层的执行结果又通过指标反馈回上层，形成完整的控制链路。

分层的意义在于隔离决策频率和时延。HPA 的同步周期通常是十几秒，因为它只需要读取指标并计算一个数字；调度器在每次创建 Pod 时运行一次，决策时间通常是毫秒到秒级；Node 级扩容可能需要数分钟。如果把所有决策压在同一层，慢路径会拖快路径，快路径的频繁抖动又会干扰慢路径的稳定性。

6.4.2 反馈闭环的控制要素

无论调节对象是副本数、节点数还是函数实例，反馈闭环都包含四个共同要素：指标、目标、动作和校验。

指标是闭环的输入。它必须能够反映负载变化，但又不能太敏感。常见的做法是用滑动窗口或指数移动平均（EMA）对原始指标做平滑，避免单次尖峰触发不必要的扩缩。指标的选择也决定了闭环保护的对象：CPU 利用率保护的是计算资源余量，请求队列深度和等待时间保护的

才是用户体验。稳态下，Little 定律 $L = \lambda W$ 把平均积压量 L 、请求到达率 λ 和平均停留时间 W 联系起来；当到达率大致稳定而积压量持续上升时，等待时间也在变长，队列指标因此可以比 CPU 更早暴露容量不足。

目标是闭环希望维持的水位。例如“每个 Pod 的平均 CPU 维持在 60%”或“请求等待时间不超过 200 毫秒”。目标值不是越低越好：设得太低，资源利用率不足；设得太高，缓冲余量太小，偶发波动就容易突破容量上限。

动作是控制器根据偏差计算出的调节量。动作必须足够克制：设置最小规模防止低峰缩得过多，设置最大规模控制预算，限制单次增减幅度并设置冷却时间，避免在新实例尚未接管流量前连续过冲。

校验是闭环收敛的保障。控制器执行动作后，必须等待一定时间让动作生效，再读取新指标判断偏差是否缩小。若扩容后关键指标仍无改善，说明瓶颈可能不在容量扩展层。此时不应直接继续盲目扩容，而应把异常传递给上层排查链路。HPA 的职责是把容量决策标准化，而不是替代完整的根因归因；工程上通常配合限流、降级、队列治理等机制，避免误把非容量问题当成“再加更多副本”来处理。

这四个要素的共同挑战是**延迟**：指标恶化到被察觉需要时间，计算和决策需要时间，动作生效（镜像拉取、进程启动、预热）也需要时间。延迟越大，闭环越滞后；延迟不确定，控制器就越难设置合适的冷却时间。工程上通常通过容忍区间过滤噪声、通过稳定窗口抑制振荡、通过速率限制防止过冲，本质都是在“延迟不可避免”的前提下，让闭环尽可能稳健。

6.4.3 HPA 在做什么：闭环的工程落地

HPA (Horizontal Pod Autoscaler) 是 Kubernetes 中实现弹性伸缩闭环的标准控制器。它的职责可以归结为一个周期性动作：**读取当前指标，计算期望副本数，将结果写回工作负载的副本数配置**。写入完成后，Kubernetes 负责创建或回收对应的 Pod。

以一个大厅接口服务 `lobby-api` 为例（负责玩家上线后拉取公告、房间列表、好友状态等），这类服务请求短、状态少，适合水平伸缩。为它设定一个目标：每个 Pod 的平均 CPU 利用率维持在 60%，既不因过低而浪费资源，也不因过高而产生排队。这里的利用率是相对于容器 **CPU request** 而言的，而不是相对于宿主机总 CPU 或某种绝对占用；因此若容器没有配置 CPU request，这个百分比无法定义，HPA 也就无法依据该指标伸缩。副本数范围设为 2 到 10：下限保证低峰可用性，上限对应预算与集群资源约束。

图 84 把 HPA 控制器的职责边界具体化：它真正负责的只是读取指标、计算期望副本数、写回 Deployment 这一小段，写回之后的 Pod 调度、创建与回收都交给了 Kubernetes 控制平面。正是这条边界，决定了扩容不及预期时该往指标滞后、HPA 计算还是 Pod 调度上去查。

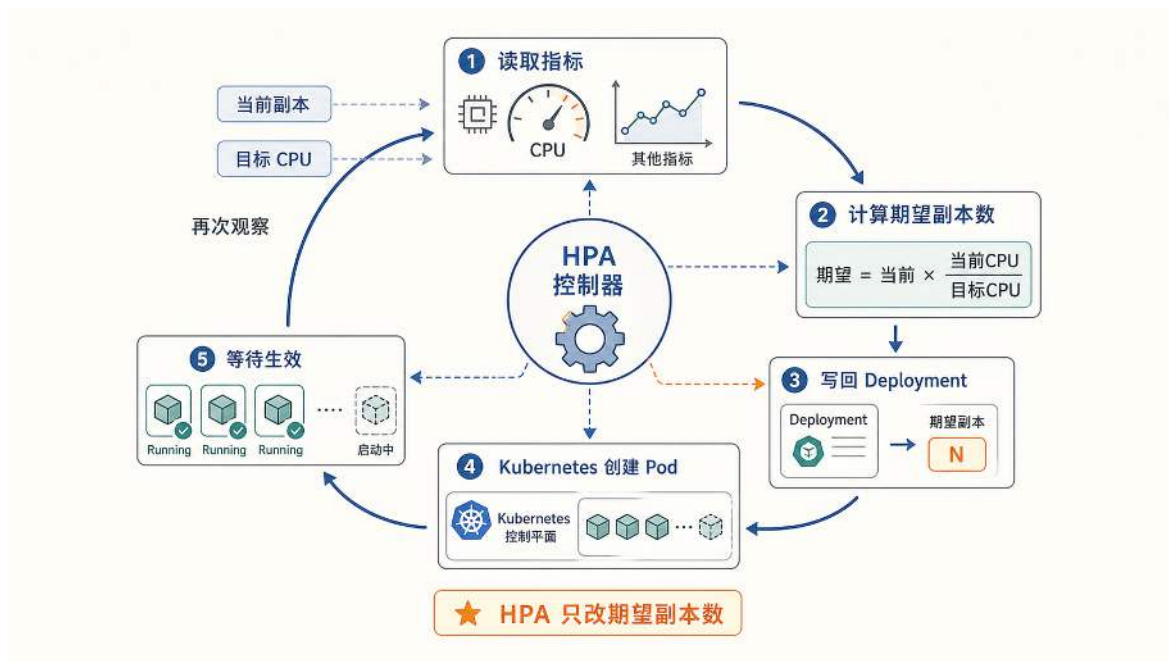


图 84: HPA 控制器周期性读取指标并计算期望副本数，随后把结果写回 Deployment，由 Kubernetes 完成 Pod 创建与回收。

因此，HPA 的输出是一个新的期望副本数；真正多出来的 Pod 还要等待 Kubernetes 调度、创建并通过就绪检查。

以晚间活动开始为例：玩家集中涌入后，最先变化的是**指标**而非副本数。lobby-api 的请求速率上升，单个 Pod 的 CPU 利用率从 30% 升至 90%。HPA 不把“达到 90%”作为一个单独的触发条件，而是在每个同步周期（默认 15 秒）计算当前指标与目标指标的比值，再据此得到期望副本数。控制器为这个指标比值设置了一个容忍区间（默认 10%）：如果比值仍落在目标值上下 10% 的范围内，控制器不会动作。扩容与缩容在响应速度上存在不对称：扩容默认没有稳定窗口，只要一次同步周期中的指标比值超出容忍区间，且计算出的期望副本数大于当前值，扩容就可以执行；而缩容则有默认 5 分钟的稳定窗口，只有较小的期望副本数在窗口内保持稳定，才会真正执行。这种不对称设计的逻辑是：扩容需要尽快响应以缓解排队，而缩容需要更多确认以避免震荡。

当一次同步得到的指标比值超出容忍区间后，HPA 进入计算期望容量阶段：将当前负载与目标负载做比例换算，得出期望副本数。其核心公式为

$$n_{\text{des}} \approx \left\lceil n \cdot \frac{\text{当前指标}}{\text{目标指标}} \right\rceil.$$

例如当前有 $n = 2$ 个 Pod，平均 CPU 为 90%，目标为 60%，则期望副本数为

$$n_{\text{des}} = \left\lceil 2 \cdot \frac{0.90}{0.60} \right\rceil = 3.$$

需要区分决策结果和执行结果：**HPA 的决策结果只是一个期望副本数**。它将 `lobby-api` 的副本数从 2 改写为 3，但不负责启动容器。随后 Kubernetes 才真正创建新 Pod、调度到节点、拉起进程并等待就绪检查通过。这就是前文提到的执行时延：从 HPA 写入期望副本数到新 Pod 实际承接流量，中间要经历镜像拉取、进程启动和预热等步骤，耗时可能从数秒到数十秒不等。

实际运行中，HPA 往往需要多轮迭代才能收敛。新 Pod 刚启动时，旧 Pod 仍然承受着全部负载，CPU 利用率可能仍然偏高。下一轮 HPA 再次读取指标，如果 3 个 Pod 时平均 CPU 仍在 80%，就会将期望副本数更新为 4；再过一轮，如果指标回落到 60% 附近，扩容停止。这种逐步逼近而非一次到位的行为，正是反馈闭环的特征：每一轮动作的效果成为下一轮决策的输入。

当活动结束、流量回落时，指标反向变化，HPA 开始计算更小的期望副本数。对 `lobby-api` 这类短请求无状态服务，减少副本的风险较低；但对长连接或强状态服务（例如战斗服、带会话的网关），直接回收 Pod 意味着正在进行的连接被强制中断。因此缩容策略通常比扩容更保守：HPA 默认使用 5 分钟的缩容稳定窗口，在窗口内保留近期计算出的期望副本数，并选择其中较保守的结果，避免短时低谷触发快速缩容；如果需要进一步限制单次缩减幅度，还可以通过 `scaleDown.policies` 显式配置缩容速率。同时，服务自身需要提供安全退出能力，例如收到终止信号后停止接入新会话，并为存量连接预留完成时间。两者配合，HPA 触发的副本减少才不会转化为用户可感知的中断。

HPA 并未改变弹性伸缩的本质，而是将反馈闭环封装为一个标准控制器，并把 Pod 的创建、回收和流量接入等执行细节交给 Kubernetes。第 6.4.2 节提出的指标、目标、动作和校验，在 HPA 中分别落实为指标采集、期望副本数、规模写回和下一轮观测；第 6.3 节提出的应用条件则决定缩容能否安全完成。

下一小节用一段 Go 伪代码实现一个最小化的 HPA 控制器，通过 Kubernetes API 执行一轮完整的 Reconciliation Loop：读取真实状态、与目标状态比较、将差异写回系统对象。

6.4.4 简化 HPA 控制器：用代码表达反馈闭环

上一节从外部视角描述了 HPA 的行为：周期性读取指标、计算期望副本数、写回工作负载对象。本节用一段 Go 伪代码将这个控制器的内部逻辑展开，使闭环的每一步对应到具体的 API 调用。

这个迷你控制器只针对一种场景：对某个 `Deployment` 自动调整副本数，使平均 CPU 利用率接近目标值（例如 60%）。这类目标适用于请求短、状态轻、多副本之间可自然分流的服务，如大厅接口或匹配入口。对于战斗服这种强状态组件，增加副本并不能分摊已有会话的负载，比例公式的前提不成立。这是第 6.3 节讨论的边界：弹性伸缩只对可分摊的负载有效。

控制器每一轮循环由三个步骤按因果顺序构成：先读取当前状态，才能计算差距；计算出期望副本数，才能写回系统。

读取状态。控制器需要两个输入：当前副本数（来自 `Deployment` 的 `/scale` 子资源）和当前负载水平（来自 `metrics-server` 提供的 `Metrics API`）。本例使用所有 Pod 的平均 CPU 利用率作为负载信号：平均 CPU 高于目标说明单个 Pod 处理能力不足，需要扩容；低于目标说明存在资源冗余，可以缩容。

计算期望副本数。沿用上一节的比例公式：

$$\text{des} = \left\lceil \text{cur} \cdot \frac{\text{avgCPU}}{\text{targetCPU}} \right\rceil.$$

计算结果需要经过边界约束：下限 `minReplicas` 保证低峰可用性，上限 `maxReplicas` 防止资源耗尽。

写回副本数。控制器将期望副本数写入 `Deployment` 的副本数字段，由 Kubernetes 负责实际的 Pod 创建或回收。写入后效果不会立即体现：新 Pod 需要经历调度、镜像拉取、进程启动和就绪检查。正因为存在这段执行时延，控制器在写入后必须等待一个周期再进入下一轮，避免在动作尚未生效时重复计算导致过冲。

以下伪代码保留了闭环的骨架，省略了稳态窗口、缩容排空和异常处理等工程细节：

迷你 HPA 控制器伪代码

```
1 for {
2     avgCPU := MetricsAvgCPU(ns, deploy) // 读平均 CPU
3     cur    := GetDeploymentReplicas(ns, deploy) // 读副本数
4
5     des := ceil(float(cur) * float(avgCPU) /
6                float(targetCPU)) // 比例计算
7     des = clamp(des, minReplicas,
8                maxReplicas) // 边界约束
9
10    if des != cur {
11        PatchDeploymentScale(ns, deploy, des) // PATCH .../deployments/{name}
12        }/scale
13    }
14    sleep(15 * second) // 按控制周期采样更新指标与状态
15 }
```

这段伪代码展示了 HPA 的核心结构：一个持续运行的控制环。工业级实现会在此骨架上补充容忍度与稳定窗口（避免对短暂波动做出反应）和异常处理（API 调用失败时的退避策略）。需要区分的是，HPA 控制器本身只负责计算并写入期望副本数；当 Kubernetes 因此回收 Pod 时，连接排空和存量会话完成属于 Pod 终止阶段的职责，由 `preStop` 钩子、优雅终止窗口和应用自身的退出逻辑来保障。两者配合，缩容动作才不会引发用户可感知的中断。

6.4.5 HPA 的边界：副本控制器不能解决所有问题

HPA 将副本数的增减自动化，但它的能力止步于此。一个常见的误解是：只要在 Kubernetes 上配置了 HPA，高并发和资源成本问题就自动解决。以下几个方向上的局限说明，自动化副本数只是弹性系统的一环，而非全部。

第一个局限来自**最小副本约束**。从成本角度看，凌晨无人时副本收缩是自然目标，但“直接收缩到 0”并非对所有服务都安全。若服务入口没有请求缓冲、预热机制或快速就绪路径，副本数为 0 会让首批请求等待更长或被拒绝。即使平台支持从零启动，新实例也要经历镜像拉取、进程启动、配置加载和数据库连接初始化，这些动作通常需要可观时间。对长连接或状态依赖强的组件，频繁从 0 拉起会放大尾延迟。工程实践通常会给这类服务保留非零底仓；当平台明确支持事件驱动、请求排队、快速预热和冷启动保护时，HPAScaleToZero 或类似策略才有机会安全落地。

第二个局限来自**指标滞后**。HPA 最常用的触发信号是 CPU 或内存利用率，它们获取方便但属于粗粒度的资源指标。在游戏匹配系统中，当大量玩家同时点击匹配时，最先恶化的是排队长度和等待时间；CPU 利用率要等到队列持续积压后才明显升高。**CPU 的上升常常滞后于玩家体验的恶化**。如果等 CPU 达到阈值才触发扩容，加上新 Pod 的启动时间，玩家已经经历了一段明显的等待。更极端的情况是，服务因数据库锁冲突而阻塞，此时 CPU 利用率很低，但服务实际上已不可用。解决方向是引入排队长度、业务并发数、响应时间等自定义指标，但这要求业务侧提供对应的埋点和监控接口。

第三个局限来自**容量参数与退出流程**。即使伸缩动作自动化了，开发者仍然需要回答一系列容量问题：每个 Pod 申请多少 CPU 和内存（CPU 配额过低会触发 throttling 导致延迟抖动，内存配额过低会触发 OOM 终止容器，过高则浪费配额）、目标利用率设为多少、最大副本数是否会超出下游连接池容量。对于包含有状态组件的游戏系统，问题更进一步：匹配入口这类无状态服务适合由 HPA 管理，但战斗服这类承载真实对局的强状态单元不能随意终止进程，需要专门的优雅退出、排空和状态迁移逻辑。HPA 并未消除基础设施的设计负担，而是将其从手工扩缩操作转化为资源配额、指标选择、伸缩边界和退出流程的设计问题。

第四个局限来自**不可水平扩展的瓶颈**。如果入口服务可以增加副本，而所有请求仍要经过同一个写锁、单分片数据库或串行协调器，那么新增副本只会更快地把压力送到这个单点。Amdahl 定律说明，系统中不能并行扩展的比例为 s 时，即使可扩展部分增加无限多副本，整体加速上限也只有 $1/s$ 。因此，弹性控制器只能调节已经具备并行扩展条件的部分，不能消除架构中的串行瓶颈。

这些局限的共同根源在于：**HPA 仍然以副本为中心**。开发者必须关心运行多少实例、每个实例分配多少资源、低峰至少保留几个、高峰最多允许多少。如果平台能够在请求到来时自动准备运行环境，在请求结束后回收环境，并将路由、资源分配和冷启动优化收敛到基础设施层，开发者就不再直接管理副本数。这正是 Serverless 要推进的方向：在 HPA 的基础上进一步下沉副本管理，同时需要评估它在游戏这类延迟敏感业务中的适用条件。

6.4.6 分层控制：从副本决策到节点放置

HPA 决定需要多少 Pod，但新 Pod 能否实际运行还取决于调度器和节点容量。完整的弹性控制链路可以概括为三层。

第一层是**副本决策**，由 HPA 完成：根据指标偏差计算期望副本数，写回 Deployment。这一层只关心“需要多少实例”，不关心实例放在哪里。

第二层是**放置决策**，由调度器完成。当 Deployment 的期望副本数增加后，Kubernetes 调度器为新 Pod 选择一台 Node。调度器的决策分两步：先按硬约束过滤掉不满足条件的 Node（资源不足、标签不匹配、存在亲和性冲突），再对候选 Node 按软目标打分（资源均衡、分散同类副本、数据本地性），选择得分最高的一台完成绑定。如果所有 Node 都无法通过过滤，Pod 会进入 Pending 状态，等待约束变化或由节点扩容器继续评估。

第三层是**基础设施决策**，由 Cluster Autoscaler 完成。它根据 Pending Pod 的资源与调度约束，判断预先配置的某个节点组新增机器后能否使 Pod 可调度；只有判断成立时，才向云平台申请新的虚拟机实例。资源不足通常可以通过增加合适规格的 Node 缓解，但不存在匹配标签、污点无法容忍或存储拓扑无法满足等约束，不会因为简单增加任意节点而消失。Node 级扩容还要经历虚拟机创建、系统初始化和节点注册，通常比 Pod 级扩容慢得多。

这三层控制器各自独立运行，通过对象状态间接协作：HPA 修改副本数字段，调度器读取 Pod 规格并写入 Node 绑定，Cluster Autoscaler 读取 Pending Pod 并修改节点组规模。任何一层的瓶颈都会传导到上层：调度器找不到合适 Node，HPA 的计算结果就无法落地；Node 扩容慢，调度器就只能让 Pod 持续 Pending。

图 85 把三层控制器的交接关系放在同一条链路中：上层先提出新的容量需求，中层尝试利用现有节点完成放置，下层只在新增节点能够解除调度约束时扩充基础设施。

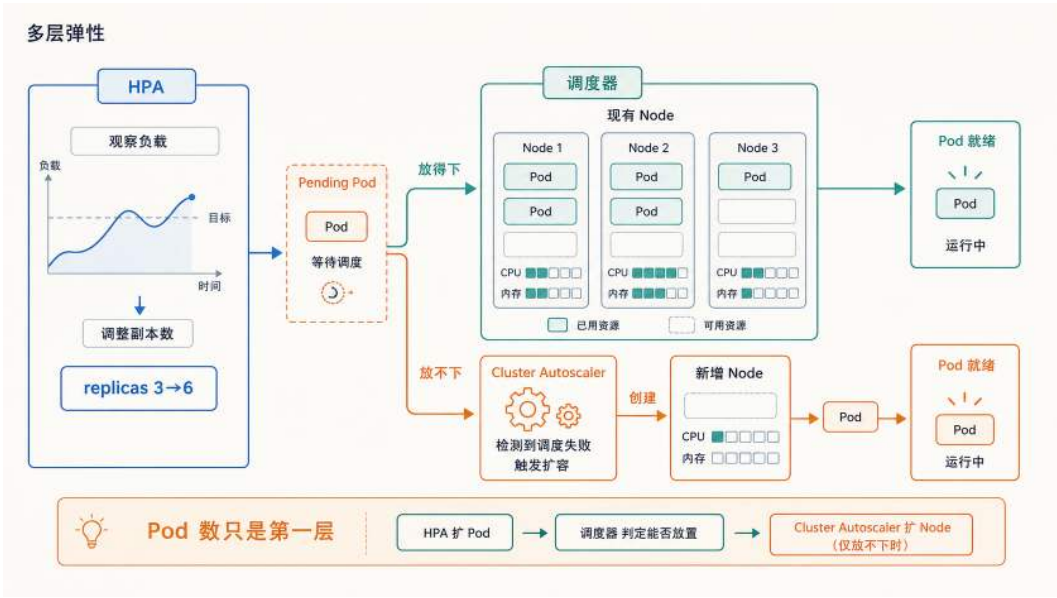


图 85: 分层弹性控制链路：HPA 产生新的 Pod 容量需求，调度器先在现有 Node 中寻找可行位置，节点扩容器再评估新增 Node 能否解除资源约束。

这张图也说明，HPA 已经写入期望副本数并不等于容量已经生效；只有放置、节点供给和就绪检查依次完成，新副本才真正进入服务路径。

6.4.7 多目标调度

第五章已经通过一次 battle 补位展示了“先过滤、再打分”的两步决策。本节进一步把打分环节形式化，以说明放置决策为何本质上是一个多目标优化问题：调度器不能只看“哪台机器

还有空闲 CPU”，还要同时考虑延迟、成本、故障隔离和数据位置等多个目标。

硬约束（必须满足）包括：Node 剩余资源是否大于 Pod 声明的 requests；Node 是否被标记为不可调度；Pod 是否声明了特殊的硬件需求（如 GPU）。调度器依次执行过滤规则，得到可行节点集合 $\mathcal{N}_{\text{feasible}}$ ；如果集合为空，本轮调度失败，Pod 返回队列等待后续重试。

软目标（综合权衡）包括：资源利用均衡（避免某些 Node 满载而其他 Node 空闲）、副本分散（同一服务的多个副本尽量放在不同 Node 或不同机架，降低单点故障影响范围）、数据本地性（把计算任务放到已缓存所需数据的 Node 附近，减少网络传输）、预热状态（优先选择已经缓存了对应镜像或预热了运行时环境的 Node，缩短启动时间）。

进入可行集合后，各项软目标先被转换到可比较的评分范围，再按权重合并。一个简化的评分函数可以写成

$$S(n) = w_r R(n) + w_f F(n) + w_d D(n) + w_p P(n), \quad n \in \mathcal{N}_{\text{feasible}},$$

其中 R 表示资源匹配程度， F 表示故障域分散程度， D 表示数据本地性， P 表示镜像或运行时预热程度， w_r, w_f, w_d, w_p 是各目标的权重。调度器选择得分最高的节点；若业务更重视故障隔离，就提高 w_f ，若任务需要读取大量本地数据，就提高 w_d 。

这类打分是在当前集群状态下对一个待调度 Pod 作出的在线决策，并不保证未来所有 Pod 的整体放置达到全局最优。不同目标的权重也因集群和业务而异。调度器的工程价值在于先用硬约束保证结果可执行，再用可配置的评分规则表达软目标，并把每一次选择限制在可解释、可复查的决策范围内。

6.4.8 Serverless 引擎：把副本管理继续下沉

HPA 将弹性粒度定在服务副本数：开发者仍然要设置最小副本数、最大副本数和指标阈值。Serverless 则把容量管理细化为按请求/事件驱动的执行环境分配：请求到来时优先复用空闲环境；没有可用实例时再按需创建；请求处理完后实例可回收或暂时保留。开发者交付的是一段业务逻辑代码（Function），平台负责为它分配执行环境并管理生命周期。这里的无服务器并非物理上不存在服务器，而是指开发者不再直接管理服务器和副本数。

粒度变化带来的第一个机制差异是**触发方式**。HPA 依赖周期性轮询（Polling）：控制器每隔固定时间读取指标，再计算期望副本数。Serverless 的入口更接近事件驱动（Event-Driven）：HTTP 请求、消息队列消息、对象存储更新等事件到达时，事件网关（Event Gateway）直接感知并分配给可用的函数实例。以游戏场景为例，玩家发起匹配请求、上传截图到对象存储、或消息队列中出现一封待发送的全服补偿邮件，都可以作为触发事件。

当事件瞬间爆发时，网关不需要等待 CPU 平均值升高再触发扩容。它直接判断当前是否有空闲函数实例：有则分配，无则在入口处短暂缓冲请求，同时通知调度器准备新的执行环境。环境就绪后，网关将请求转交执行。请求数量上升时，平台并行准备更多实例；请求处理完毕后，实例标记为空闲，空闲超时后回收。与 HPA 相比，容量分配从服务副本层向请求/事件层前移，响应路径也更靠近入口事件。

但事件驱动带来一个直接矛盾：容器从零启动需要时间（镜像拉取、文件系统准备、运行时初始化等），而 Serverless 又要求在请求到来时才准备执行环境。如果每次请求都从头启动，用户仍然会经历明显等待。因此 Serverless 引擎必须解决一个具体的底层问题：**如何将启动一个隔离执行环境的时间从秒级压缩到毫秒级。**

第 6.1 节拆解过容器的启动路径。冷启动慢的原因在于多个步骤串联在同一条请求路径上：镜像获取与解压、文件系统准备、网络配置、隔离环境创建、语言运行时加载、应用依赖初始化。将这些步骤按层次归类，冷启动时间可以分解为三部分：

$$T_{\text{cold}} \approx T_{\text{sandbox}} + T_{\text{runtime}} + T_{\text{app}}.$$

其中 T_{sandbox} 是环境准备与隔离创建的成本（镜像获取、文件系统、网络、沙箱权限边界等）； T_{runtime} 是语言运行时与框架初始化的成本（解释器/虚拟机启动、依赖装载、JIT 预编译等）； T_{app} 是应用层自带的启动工作（读配置、建连接池、拉取热数据、预热缓存等）。三项之和是**环境初始化成本**，它是冷启动优化的主要对象；但它并不等于用户感知到的完整首次响应时间——后者还可能叠加请求排队、准入判断、路由分发以及首个请求自身的执行时间。要将冷启动压到毫秒级，需要逐段压缩这里的初始化常数项，或将其中的大头从请求关键路径移到后台路径。

针对 T_{sandbox} ，工程上的代表方案是**极简微虚拟机 (MicroVM)**。容器依赖 Namespaces 与 Cgroups 实现视图隔离和资源限制，借助 OverlayFS 实现镜像分层与文件系统共享，进程共享宿主机内核，启动路径短；但共享内核意味着隔离边界依赖宿主机内核的正确性，多租户场景下故障影响范围较大。传统虚拟机每个实例有独立内核与地址空间，隔离边界更强；代价是完整的设备模型和引导流程将 T_{sandbox} 拉回秒级。MicroVM 在两者之间取舍：保留虚拟机的隔离边界（独立内核、硬件虚拟化），同时将虚拟机中不服务于云端工作负载的部分压到最少。以 Firecracker 为例，它使用轻量虚拟机管理器（VMM）与精简设备模型，只模拟网络和块存储等 Serverless 真正需要的虚拟硬件，不模拟键盘、声卡、显示器等无关外设。这样，启动一个强隔离执行环境可以被压到百毫秒量级。但 MicroVM 只解决沙箱层的启动速度， T_{runtime} 与 T_{app} 仍然需要其他手段。

针对 T_{runtime} 与 T_{app} ，代表方案是**内存快照与恢复 (Snapshot & Restore)**。即使沙箱启动压到百毫秒，语言运行时和应用初始化仍可能耗时几百毫秒到几秒（解释器启动、依赖加载、JIT 预热、连接池建立等）。快照的思路是：先在后台将执行环境启动到可接收请求的状态，暂停并保存当时的内存映像。下次请求到来时，系统将快照恢复到内存中，函数直接进入可用状态，跳过完整的初始化路径。原本昂贵的初始化被摊平到后台预热阶段，冷启动路径上只剩恢复与少量补齐的成本。

制作快照时，平台需要先把一个执行环境初始化到可接收请求的状态，再将其内存和运行状态保存下来；快照写入存储后，生成它的实例不必继续常驻。请求到来时，平台仍要支付快照读取、内存恢复和状态补齐的成本；数据库连接、网络会话、当前时间戳和随机数种子等外部或时效性状态也需要重新建立或校验。如果平台还要进一步压低恢复时间，可以另行维护预恢复的暖池，但这是附加的容量预留策略，并不是快照恢复本身的必要条件。

另一个方向是 **WebAssembly (Wasm)**，它将隔离边界下沉到指令集与运行时层面。Wasm 模块运行在受限的线性内存中，可用的系统交互由宿主能力接口（如 WASI）决定。由于不需要完整内核、用户态环境和设备模型，实例化一个 Wasm 模块的路径更短，很多场景下可以将启动开销压到毫秒级。Wasm 适合承接短小、轻量、对系统调用依赖少的计算单元，例如边缘侧逻辑、数据清洗或规则判定。其边界同样清晰：如果函数强依赖完整操作系统语义、需要大量原生扩展或长期维护复杂网络连接，迁移到 Wasm 需要重写运行时边界与依赖交付方式。工程上更常见的做法是将 Wasm 作为更轻的沙箱层：适合的部分用它换取更快启动与更小常驻开销，不适合的部分仍落在容器或 MicroVM 上。

这三类手段的共同目标，是降低请求关键路径上的初始化延迟，但采用的思路并不相同，可以归纳为三条：**减少必须完成的工作**——MicroVM 精简设备模型、Wasm 省去完整内核与用户态环境，让需要执行的初始化本身变少；**复用已经完成的工作**——快照恢复、Zygote fork、分层缓存都是把一次昂贵的初始化结果重复利用；**预留已经就绪的容量**——暖池、预创建实例和预恢复环境提前把工作做在后台，请求到来时直接取用。只有第三条才是严格意义上的“用常驻成本换首次延迟”：它必须持续占用内存和计算来维持就绪实例。前两条不一定伴随常驻暖实例，例如快照可以保存在存储中按需恢复，是否再叠加一个预恢复池是另一层独立的资源与延迟取舍。以游戏中低频的签到、运营活动入口为例：放到 Serverless 上可以显著降低低峰成本，但第一批请求更容易遇到短暂等待。引入预热池、快照恢复或 Wasm 执行层后，这个等待可以被压到用户几乎感知不到的范围；代价则依手段而异，可能是更多常驻资源，也可能是恢复时的补齐成本或运行时兼容性约束。

图 86 将完整冷启动与三类优化并排放在同一条时间尺度上：MicroVM 和 Wasm 通过精简执行环境减少必须完成的工作，Snapshot 通过恢复已保存的状态复用初始化结果，暖池则直接预留已经就绪的容量。这样才能区分某项技术究竟缩短了工作、复用了工作，还是提前支付了工作。

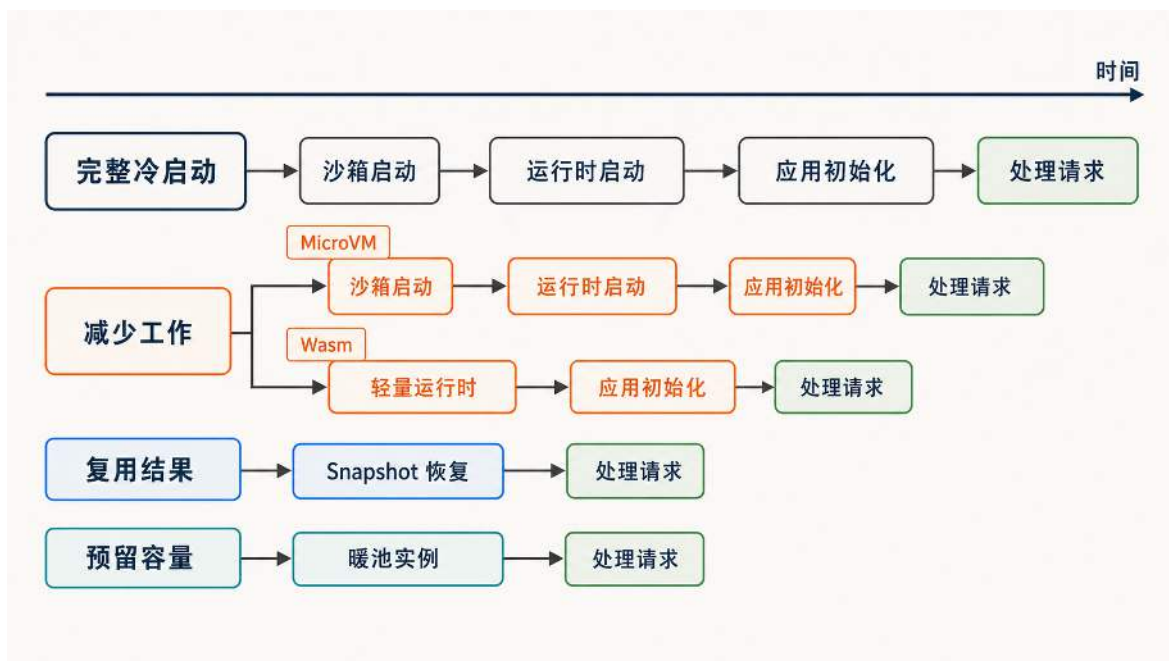


图 86: Serverless 冷启动的三类优化机制: MicroVM 和 Wasm 减少初始化工作, Snapshot 复用已经完成的初始化, 暖池预留可直接接收请求的容量。

Serverless 的目标并非让启动成本凭空消失, 而是通过更轻的沙箱、更快的恢复和更充分的后台预热, 将用户能感知到的那部分成本压低。

然而, 即使冷启动问题得到缓解, 以函数为交付单位的典型 Serverless 形态 (FaaS, Function as a Service) 仍然不适合所有游戏服务端。FaaS 通常要求函数按无状态方式设计, 并且不承诺某个执行环境会在整局对战期间持续存在、保持固定连接和会话归属; 即使环境可能在多次调用之间被复用, 应用也不能把这种复用当作生命周期保证。

从网络模型看, 无论是 MOBA 还是 FPS, 客户端与战斗服之间需要维持长达数十分钟的稳定 TCP 或 UDP 连接。典型 FaaS 不提供某个执行环境持续承载一局对战的生命周期保证, 因而无法直接承载长驻对局的连接模型。从执行模型看, 战斗服通常有一个持续运行的循环 (Game Loop), 以每秒 30 到 60 次的频率推进玩家位置、弹道计算和物理碰撞。典型 FaaS 以事件调用为基本执行单位, 调用结束后环境可能被冻结、复用或回收, 这种生命周期契约与持续运行的主循环不兼容。如果将每一帧同步拆成独立的函数调用, 产生的请求量、网关压力和外部状态访问反而破坏实时性。从状态模型看, 典型 FaaS 通常要求计算单元无状态, 所有持久数据落到外部存储。对于大厅查询、签到奖励或回合制游戏的单次结算, 这是可接受的。但在状态同步或帧同步的动作游戏中, 每次技能判定都要读写外部 Redis, 单次访问虽然只有毫秒级延迟, 但在每秒数十帧的循环中累积起来会严重影响体验。

上述生命周期约束解释了第五章表 8 的场景判断: 典型 FaaS 更适合大厅查询、商城请求、签到奖励、运营活动入口和异步任务处理等短请求或低频任务; 核心战斗服这类强状态、长连接、高频循环的组件, 仍然更适合运行在常驻进程或专门的房间服架构中。Serverless 的价值在于将适合事件驱动的部分从常驻资源中拆出来, 而非替代所有服务端形态。

从 HPA 到 Serverless，底层基础设施变得更复杂，但暴露给开发者的接口在变简单。复杂性没有消失，而是被平台收敛到调度器、网关、镜像系统、运行时、快照机制和预热池中。两者共享同一套基础问题：隔离环境如何创建、网络入口如何转发、请求如何排队、实例如何预热、状态如何恢复、失败如何被观察。差异仅在于这些问题由谁负责回答：HPA 模式下由开发者配置参数，Serverless 模式下由平台自动决策。

6.4.9 降低冷启动延迟：容器快速启动的两条研究路线

前述 MicroVM、内存快照和 Wasm 是工业界压缩冷启动的通用方案，它们分别针对 T_{sandbox} 、 T_{runtime} 和 T_{app} 下手。围绕同一个 T_{cold} 拆解，研究者还从两个更细的角度做了优化：一是追问容器启动本身为什么慢，二是追问预热成本能不能在更多函数之间摊薄。这两条路线对下一节要讨论的 Agent sandbox 尤其重要：那里需要在很短时间内批量创建大量隔离环境，启动路径上的每一毫秒都会被实例数量放大。

第一条路线是把容器启动路径本身做精简。SOCK³³ 先测量了通用容器的启动开销，发现瓶颈有两处：一是 Linux 的存储与网络命名空间等原语在创建和销毁时本身不可扩展，二是像 Python 这样的运行时在导入大量库时会额外增加约 100 毫秒。针对第一处，它构造**精简容器 (lean container)**：为 Serverless 函数量身裁剪文件系统、缓存 cgroup、跳过创建代价高的命名空间，只保留隔离真正需要的部分，仅此一项就比 Docker 快约 18 倍。针对第二处，它用**通用化 Zygote**：预先维护一批已经启动、且已导入常用库的模板进程，新函数实例直接从模板进程 fork 出来，跳过运行时初始化和库导入，再叠加约 3 倍加速。换句话说，SOCK 同时压低了 T_{sandbox} 和 T_{runtime} 。

第二条路线承认预热无法避免，转而让预热成本被更多函数复用。全量缓存整个容器最简单，但每个空闲容器都常驻一份完整内存，代价高；只缓存一部分或让多个函数共享容器又各有难处。RainbowCake³⁴ 的做法是按初始化阶段把容器拆成几层：Bare 层对应基础环境和工具，Lang 层对应语言运行时，User 层对应用户代码和依赖。User 容器只适合同一函数复用，Lang 容器可以在同语言函数之间复用，Bare 容器则可以服务任意函数。空闲容器超时后先清理用户代码并降级为 Lang，再释放运行时并降级为 Bare，于是平台可以保留更通用的下层、只在命中时补齐上层。它把用常驻成本换首次延迟这个取舍做得更细：常驻的不再是整只容器，而是更容易共享的初始化层。

图 87 对比了两条路线降低冷启动代价的不同方式：SOCK 缩短单次启动路径中的沙箱准备和运行时初始化，RainbowCake 则把容器状态拆成可复用的层，让命中的下层继续保留，只补齐请求缺失的上层。前者降低一次启动要付出的时间，后者降低预热状态长期占用的成本，二者可以叠加。

³³Edward Oakes 等, *SOCK: Rapid Task Provisioning with Serverless-Optimized Containers*, USENIX ATC 2018, pp. 57–70. <https://www.usenix.org/conference/atc18/presentation/oakes>

³⁴Hanfei Yu 等, *RainbowCake: Mitigating Cold-starts in Serverless with Layer-wise Container Caching and Sharing*, ASPLOS 2024, pp. 335–350, DOI:10.1145/3617232.3624871. <https://dl.acm.org/doi/10.1145/3617232.3624871>

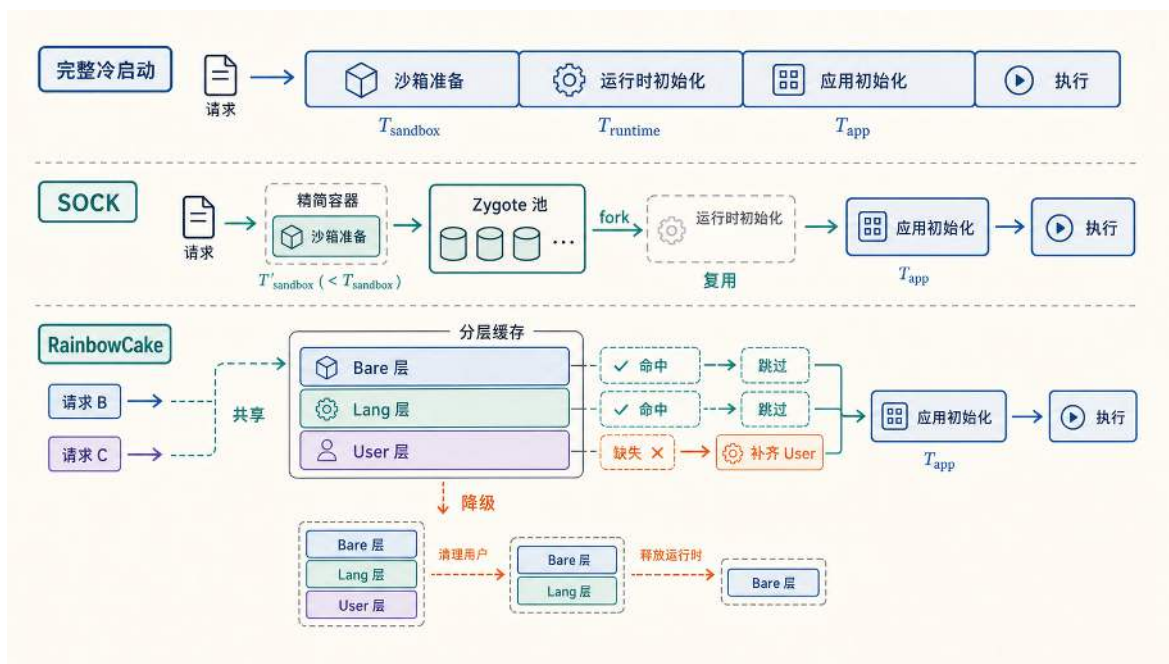


图 87: 容器快速启动的两条研究路线：SOCK 通过精简容器和通用化 Zygote 压低单次启动的 T_{sandbox} 与 T_{runtime} ，RainbowCake 将容器状态拆成 Bare、Lang、User 三层，使空闲容器可以逐层降级，并在后续请求命中时只补齐缺失的上层。图揭示了两条路线并不冲突：一条压低单次启动成本，一条摊薄常驻预热成本。

这两项工作和上面的工业方案指向同一个判断标准：冷启动不可能凭空消失，工程上能做的是把它的常数项压低，或把预热的常驻成本摊薄，问题只在于谁来付、以什么形式付。到了 Agent 场景，这个问题会以更极端的形式出现：需要在很短时间内创建大量可交互的隔离环境，快速启动不再是锦上添花，而是能否支撑训练与评测的前提。

6.5 面向 Agent 的高并发 Sandbox：从一次执行到经验生产

前面各节分别讨论了容器隔离、网络虚拟化、弹性调度和快速启动。Agent 训练与评测平台把这些能力组合到一个新的工作负载上：在短时间内批量创建大量可交互的隔离环境，让每个环境在模型多步动作之间持续存在，并把执行过程变成可验证、可复现的训练数据。

Agent 在本节中指的是一套闭环执行系统：模型观察环境状态，选择动作；动作通过工具作用到环境上（运行命令、修改文件、点击页面、安装依赖等），环境状态发生变化后将新的观察返回给模型。这个循环持续进行，直到任务完成、失败或达到步数上限。以修复代码仓库中的测试失败为例，模型需要运行测试、阅读报错、修改代码、再次运行测试，每一步动作都会改变后续环境状态。Agent 任务因此是一条连续展开的执行轨迹，而非单次函数调用。

训练数据的生产过程本身就是计算密集的：每个任务实例需要一个可交互的隔离环境，模型在其中执行多步动作；同一任务可能有多次尝试和多条路径；训练和评测随模型版本迭代反复执行。平台要承载的规模来自任务实例数、尝试次数、轨迹长度和交互频率的共同放大。EvoCUA

的技术报告和官方文章³⁵都把基础设施压力落在大规模 sandbox 承载、高频交互请求和分钟级批量创建能力上。这里不把不同发布口径中的请求吞吐写成单一固定数字，而用它说明同一个云计算问题：平台必须持续生产大量可验证的环境交互轨迹。DeepSeek-V4-Pro 的后训练框架³⁶同样将 Agentic Coding 和工具使用纳入能力培养，其基础设施需求也指向这一方向。

与游戏服务相比，Agent sandbox 的压力特征有所不同：游戏服务要求平台稳定承载玩家请求，Agent sandbox 要求平台批量承载会被模型不断修改的执行环境。两者都在考验隔离、调度、状态恢复和可观测性，但 Agent 场景额外要求环境可复位、可分叉、可批量创建。

图 88 把这些问题归到五个面向上，后面几节就沿这五个面向逐一展开：控制面负责任务准入与资源租约，执行面提供可隔离、可复位的运行环境，交互面承接模型动作与环境观察，状态面支持 checkpoint 与 rollback，验证面把执行结果变成可复查的数据。

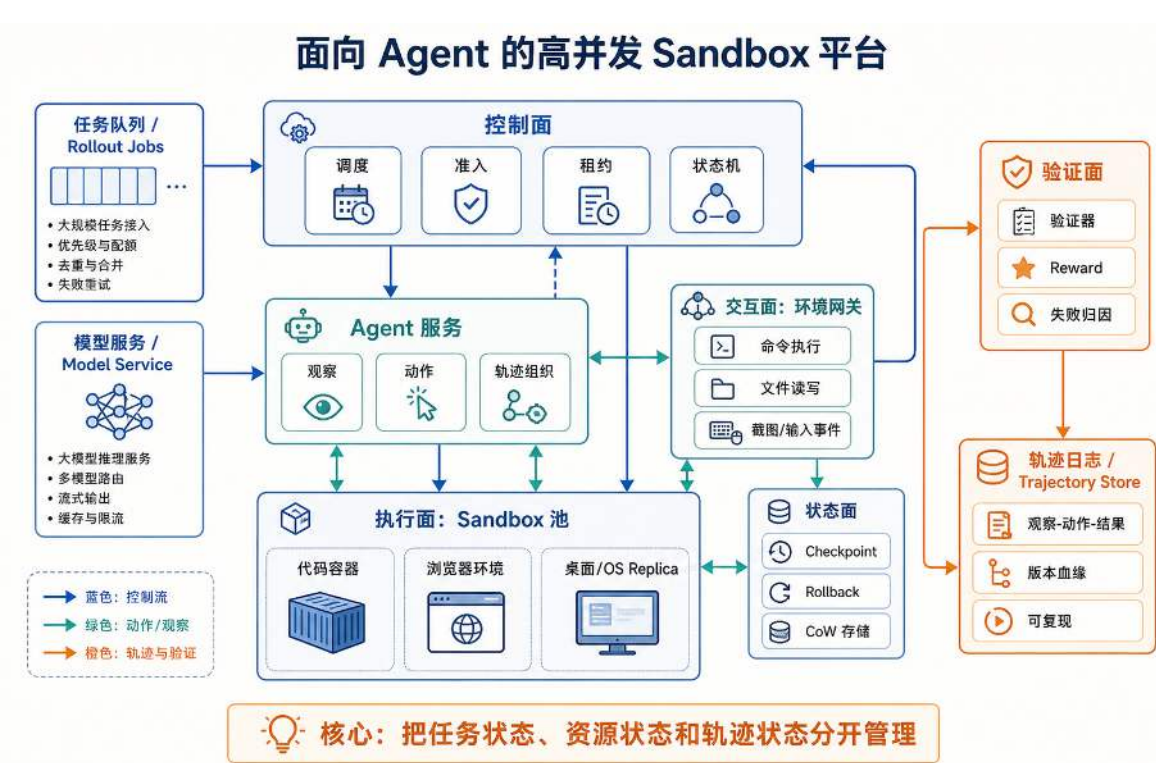


图 88: 高并发 Agent sandbox 平台将任务状态、资源状态和轨迹状态分开管理，并分别组织控制面、执行面、交互面、状态面和验证面。

这五个面向沿用前几节分析在线系统的同一套视角，这次用来观察 Agent 平台内部——控制、执行、交互、状态、验证在不同系统里名字可能不同，但要解决的问题是相通的。Agent sandbox 的特殊之处在于，它把一次请求的生命周期拉长为一条持续变化的执行轨迹：隔离环境要在多步动作之间保持可复位，网络通道要连续传递动作和观察，快照要支持回滚与分叉，日志

³⁵EvoCUA 技术报告: <https://arxiv.org/abs/2601.15876>; 美团技术团队, EvoCUA 官方文章: <https://tech.meituan.com/2026/01/26/EvoCUA.html>。

³⁶DeepSeek-V4-Pro 技术报告: https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro/blob/main/DeepSeek_V4.pdf。

要沉淀为可复查的训练和评测数据。下面先从单机顺序执行的最小闭环开始，再说明它在高并发平台中会被拆成哪些控制、执行和数据路径。

6.5.1 总体架构：先分清任务状态、资源状态和轨迹状态

在讨论十万级 sandbox 平台之前，先把一个 Agent 任务在单机上的完整生命周期拆开：一次只运行一个任务，系统为它启动一个容器，把模型接到这个容器里，让模型观察环境、执行动作、记录反馈，最后用验证器判断任务是否完成，再销毁容器。这个顺序执行的 Agent sandbox 闭环可以写成如下流程。

单机最小实现：顺序运行一次 Agent 任务

```
1 task := loadTask()
2 sandbox := startContainer(task.image)
3 trajectory := newTrajectory()
4
5 for step := 0; step < task.maxSteps; step++ {
6     obs := sandbox.observe()
7     action := model.generate(obs, trajectory)
8     result := sandbox.execute(action)
9     trajectory.append(obs, action, result)
10
11     if verifier.finished(sandbox, task) {
12         break
13     }
14 }
15
16 score := verifier.grade(sandbox, task)
17 saveTrajectory(task, trajectory, score)
18 destroyContainer(sandbox)
```

这段伪代码展示了 Agent 任务的基本闭环：观察环境，选择动作，执行动作，记录反馈，再根据验证器判断任务是否完成。作为单机实验它已经足够，但作为大规模训练和评测平台，它缺少三个系统能力。

第一个缺失是**状态一致性**。模型请求、环境创建、工具执行和验证器运行并不总是按顺序完成。模型返回时环境可能已经超时，环境启动完成时任务可能已经被取消，验证器异常退出时系统也不能简单地把结果记为模型失败。如果平台没有清晰的任务状态机，就很难判断一次失败到底来自模型、环境还是验证器。

状态不一致会进一步导致**资源泄漏**。Agent sandbox 不是普通函数调用。一次任务结束后，环境里可能留下临时文件、后台进程、浏览器缓存、打开的端口和未释放的磁盘写层。某台宿主机如果失联，控制面还要知道上面有哪些 sandbox 正在运行、哪些任务需要重试、哪些资源需要

回收。没有租约、心跳和复位机制，资源泄漏会随着任务数量持续放大。

资源泄漏之上还有**数据可追溯性**问题。一条轨迹不只是最终答案，还包括观察、动作、工具返回、终端输出、截图、文件变化、验证结果和失败原因。如果这些数据分散在不同日志里，缺少统一的任务版本、环境版本和模型版本标记，后续就很难复现这次运行，也很难判断这条轨迹能不能进入训练集。平台运行规模越大，越需要避免把难以解释的运行痕迹误当成高质量训练数据。

因此，设计大型 Agent sandbox 平台时，起点应当是三类事实来源：任务状态、资源状态和轨迹状态。容器、虚拟机或调度框架只有放到这些状态边界之后，才能明确各自负责哪一段问题。如果多个模块各自维护一份状态副本，系统在重试、故障恢复和并发调度时就无法确定哪个副本是权威来源。任务进展由谁记录、资源占用由谁追踪、轨迹数据由谁归档，必须在架构一开始就划清边界。

任务状态记录“一件事从开始到结束经历了什么”。比如一个训练批次有 10 万道任务，每道任务可能被同一个模型尝试多次，也可能被不同模型对比尝试。平台要区分清楚：这是一个批次里的任务，还是某一次具体尝试；这是第一次执行，还是从某个 checkpoint 恢复后的分支。可以把它抽象成 RolloutJob、TaskInstance、Attempt、Step 这样的对象层次。名称可以不同，但层次必须分开：批次、任务、尝试和步骤各自有独立标识，否则系统无法回答“这次失败发生在哪一次尝试的哪一步”。

任务状态还需要状态机。一个 Attempt 可能处在等待调度、环境创建、环境就绪、执行中、等待模型、等待工具返回、验证中、成功、失败、取消、超时等状态。每个状态都应该有进入条件、退出条件和超时规则。比如环境创建超过 30 秒没有成功，应该释放模型预算；等待模型返回超过限制，应该先保留环境一段时间，避免模型返回后找不到上下文；验证器异常退出，不能直接把模型判错，而应该标成验证失败。状态机是大型分布式系统稳定性的骨架：每个状态都有明确的进入条件、退出条件和超时规则。

状态机还需要被持久化，并且转移要有版本约束。否则，两个控制器实例可能同时认为自己有权推进同一个 Attempt：一个把它标记为超时回收，另一个却刚收到模型返回并准备继续执行。生产系统通常会给状态记录增加版本号或租约持有者，更新时检查“我看到的旧状态是否仍然成立”。只有检查通过，状态才能从 Running 转成 Verifying 或 TimedOut。这类 compare-and-swap 式的更新规则，保证控制面在重试、故障切换和并发调度下仍然只有一条被承认的任务时间线。

图 89 将 Attempt 的状态转移约束收束为一条生命周期：成功、任务失败、验证异常、已取消、已超时都是终态，进入终态后不再继续跳转；每次状态写回都必须通过版本号与租约持有者的 CAS 检查，避免并发控制器把验证异常误改为任务失败。

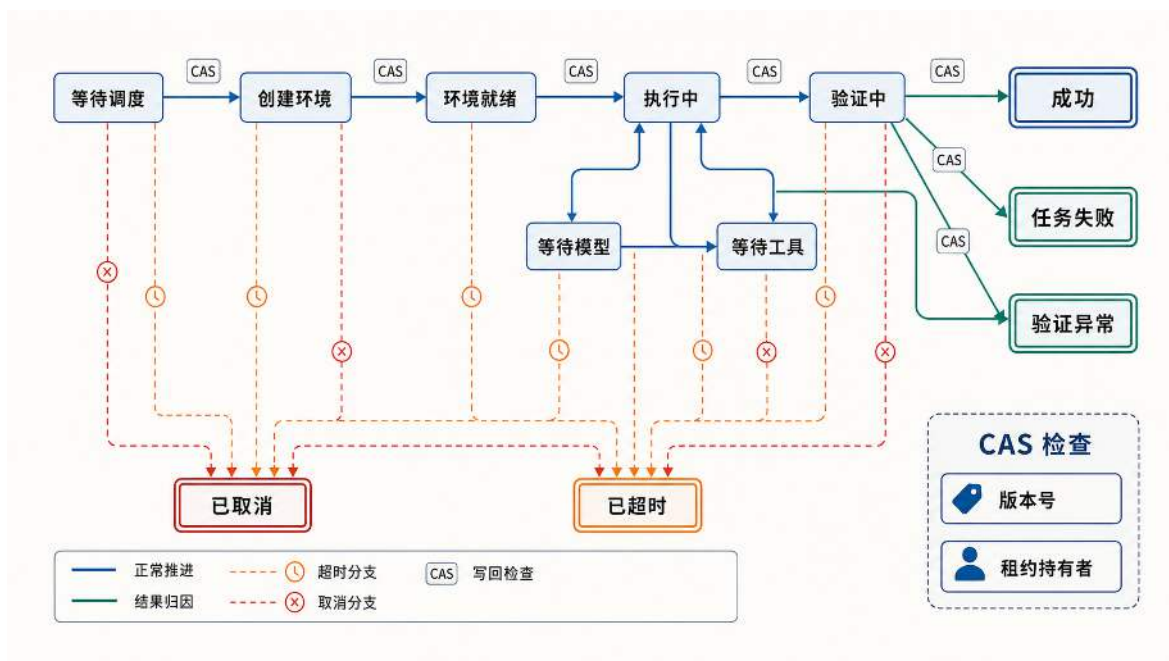


图 89: Attempt 生命周期状态机：状态转移、超时分支与 CAS 版本检查共同保证并发控制器只承认一条任务时间线。

有了这条时间线，后文的准入、租约、超时回收和验证归因就不再是分散规则，而是围绕同一个 Attempt 状态记录展开的控制面动作。

资源状态记录“谁占用了什么”。Agent sandbox 用完不清理，就会留下文件、缓存、后台服务、浏览器 profile、临时端口和进程树。平台需要知道哪些基础镜像可用，哪些 sandbox 在预热池里，哪些 sandbox 正在被某个 Attempt 使用，哪些需要复位，哪些已经损坏。SandboxLease 即租约：某次尝试在一段时间内借走某个环境。有了租约，控制面就能在任务超时、心跳丢失或节点失联时回收资源。

轨迹状态记录“训练样本是怎样产生的”。一条轨迹至少包括任务版本、环境镜像、模型版本、观察、动作、工具返回、终端输出、截图、文件变化、checkpoint 信息、验证结果、reward 和失败原因。这些信息构成一条 TrajectoryRecord，即一份可追溯的执行档案。它们的体积差别很大：状态更新可能只有几百字节，截图和日志可能很大；任务开始和结束不频繁，工具调用却可能每秒发生多次。如果将所有数据集中写入一个中心服务，I/O 压力会随任务规模线性增长，系统很快成为自身数据量的瓶颈。

这三类状态分开后，后面的设计才有落点。控制面负责让任务状态和资源状态保持一致；执行面负责提供真实、隔离、可复位的环境；交互面负责承接高频观察和动作；状态面负责把探索过程变成可回滚、可分叉的过程；验证面负责把运行痕迹变成可以训练和评测的数据。大型系统需要先把责任边界切清楚，后面每一层的扩缩容、恢复和排错才有依据。三类状态中，任务状态将在控制面中继续展开，资源状态在执行面中展开，轨迹状态则在后文的验证面中展开。下面先从控制面入手，说明大型平台如何让任务状态与资源状态保持一致。

6.5.2 控制面：维护全局秩序

Kubernetes 和 HPA 中的控制面职责是：维护期望状态，观察当前状态，再发出调度和回收动作。Agent sandbox 平台沿用同样的思路，只是被管理的对象从普通服务副本变成了大量会被模型反复修改的执行环境。

有了任务状态、资源状态和轨迹状态，下一步就是让这三类状态在运行时保持一致。这正是控制面的职责。它要解决的第一个问题，是让系统在十万级 sandbox 下仍然知道“谁在做什么”。在单机顺序流程里，步骤是线性的：启动环境，调用模型，执行动作，收集结果。工业系统里，这些步骤会同时发生在成千上万个任务上。有的任务在等模型返回，有的任务在等环境启动，有的任务正在跑测试，有的任务已经卡死，还有的任务被用户取消了。控制面如果只提供几个无状态的转发 API，就无法追踪任务进展，也无法在故障时做出正确的回收或重试决策。

控制面的第一项设计，是把模型服务、Agent 服务和环境服务分开管理。三类工作使用的资源不同，运行节奏不同，失败方式也不同。模型服务处理昂贵计算，一次推理可能占用 GPU、显存、KV cache 和推理队列。它关心模型版本、上下文长度、token 预算、批处理和限流。token 预算是对一批任务可消耗的模型计算总量的上限约束。模型服务过载时，控制面应减少新的推理请求，让任务在队列里等待；继续启动更多环境，只会让这些环境空等模型返回。

Agent 服务处理任务推进。它主要维护 **Attempt** 的状态：当前走到第几步，上一轮观察是什么，模型给出的动作能否被解析，工具返回是否超时，轨迹是否已经写入，是否应该进入验证器。Agent 服务的角色是一次尝试的流程控制器：把观察送给模型，把模型输出转化为工具动作，再把环境反馈组织成轨迹。容器创建和 GPU 调度则属于环境层与模型层的职责。

环境服务处理 sandbox 生命周期。它管理镜像、租约、启动、就绪检查、命令执行、文件读写、网络策略、残留进程清理和环境复位。环境服务不评价模型做得好不好，也不决定训练样本是否有价值；它只保证某个 **Attempt** 能拿到一个隔离、可用、可回收的执行环境。

如果把三类工作合并到一个大进程里，小规模时尚可运行，但并发量上升后各类耦合故障会迅速暴露。模型请求慢了，环境可能已经启动却一直空等；环境创建失败了，模型窗口可能已经被占住；Agent 流程重试了一次，环境层可能创建出两个 sandbox；验证器异常退出时，系统无法判定这次失败应该归因于模型、环境还是验证器。服务拆分的作用，是让每一类失败停留在自己的边界内，并且能被控制面明确记录和处理。

一个合理拆分的检验标准是：某一层重启或限流时，其他层仍然能保持自己的状态。模型服务限流，任务可以继续排队，环境不必提前创建；环境服务扩容，模型服务不必改变推理策略；Agent 服务重试，环境服务可以根据同一个 **AttemptId** 返回已有租约，避免重复创建资源。MegaFlow³⁷ 将 agent 训练基础设施拆成 Model Service、Agent Service 和 Environment Service，正是为了让模型计算、任务推进和环境生命周期可以独立扩缩容。

服务边界清楚后，控制面才能分别处理扩缩容和故障恢复。模型服务负载较高时，就限制新任务的模型请求；环境池紧张，就减少新任务准入；轨迹存储变慢，就先把结果写入本地缓冲，再异步归档。控制面维护的是全局秩序：谁在排队，谁拿到了环境，谁占用了模型预算，谁超时

³⁷MegaFlow: Large-Scale Distributed Orchestration System for the Agentic Era, arXiv:2601.07526.

了，谁需要重试，谁应该被回收。

这里的调度不能只按机器空闲程度决策。一次 **Attempt** 正式开始之前，控制面要先确认三件事：环境层能不能提供 sandbox 租约，模型层有没有推理预算，数据层能不能可靠写入轨迹日志。这三个条件对应的是同一条执行链路上的三个瓶颈。只要其中一层已经饱和，继续把新任务放进系统，反而会产生更多无法完成的执行：环境启动了，却一直等不到模型返回；模型窗口占住了，却发现环境还没准备好；执行过程还在产生截图和命令输出，后面的存储系统却无法及时写入。

因此，调度器不能只依据当前还有多少 CPU 和内存，还要把模型预算、环境租约和日志积压一起纳入准入判断。一个入口检查过程可以这样组织：控制面先查看队首任务，依次检查模型、环境和日志三条链路；只有三条链路都能继续承载新任务时，**Attempt** 才进入执行阶段。

准入与反压：一次 Attempt 进入系统前的检查

```
1 attempt := taskQueue.peek()
2
3 if modelBudget.full() {
4     keepWaiting(attempt)
5     return
6 }
7
8 if sandboxPool.noLease(attempt.envSpec) {
9     keepWaiting(attempt)
10    return
11 }
12
13 if trajectoryLog.backlogHigh() {
14     reduceNewAttempts()
15     reduceScreenshotRate()
16    return
17 }
18
19 lease := sandboxPool.acquire(attempt.envSpec)
20 taskQueue.pop()
21 startAttempt(attempt, lease)
```

这种入口减速机制称为**反压** (backpressure)。反压的含义是：当下游的处理能力不足时，上游不要继续全速生产，而要主动减速，把压力挡在入口处。在这个例子里，模型预算满了，任务就留在队列里；环境池没有租约，就不要提前占住模型窗口；轨迹日志积压过高，就降低新任务准入，必要时减少高频截图。这样做会牺牲一部分吞吐，但已经进入系统的任务仍然有环境可用、有模型可调、有日志可写。对大型平台来说，短时间变慢仍可接受；更严重的后果是各层独立推

进而互不协调，最后产生一批无法复现、无法判因、也不适合进入训练集的轨迹。

实际系统里，这个入口检查还要进一步变成“资源预留”。检查只回答“现在看起来能不能跑”，预留则保证“接下来这次尝试确实有资源可以用”。如果调度器只检查不预留，两个并发 Attempt 可能同时看到同一份空闲模型预算或同一个空闲 sandbox，随后发生超分配。更稳妥的做法，是把模型预算、sandbox 租约和轨迹写入通道一起纳入一次准入事务：任何一步失败，都要撤销已经完成的预留，让任务回到等待状态。

图 90 把这次准入排成一条泳道，能看清它其实压着两条边界。一条是服务边界：模型服务、环境服务和轨迹日志各自独立，任一处的故障不会越界扩散。另一条是事务边界：三处预留要么一起确认、一起提交，要么在任一步失败后沿已经完成的预留逐一撤销，让 Attempt 启动前的资源状态始终一致。

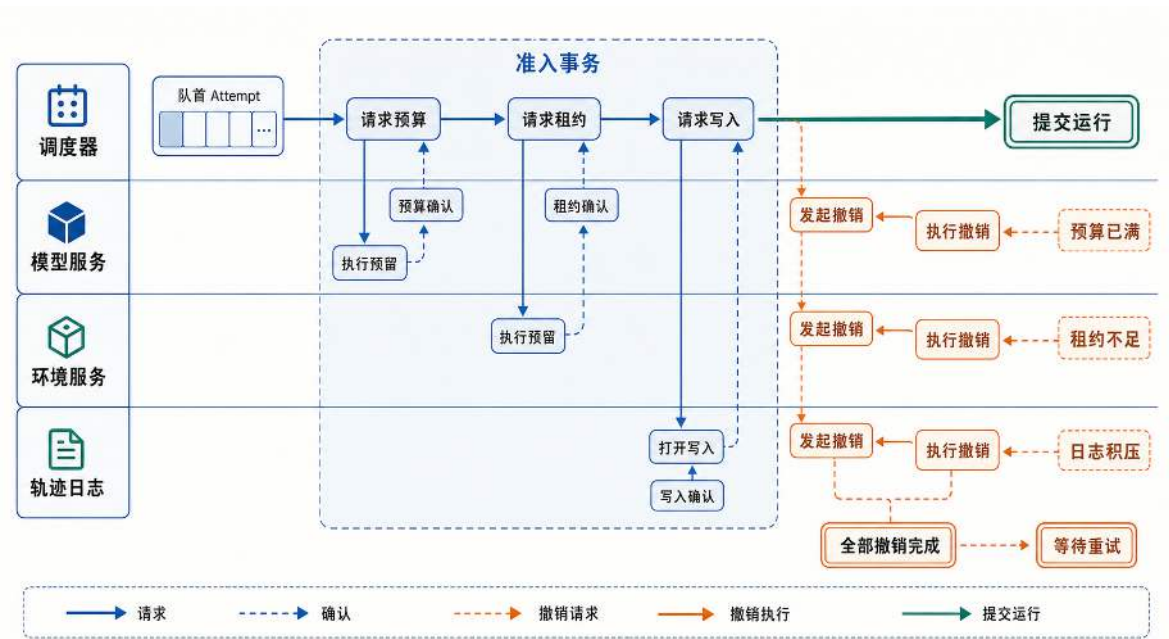


图 90: 控制面准入事务把模型预算、sandbox 租约和轨迹写入通道绑定为一次可撤销的资源承诺，任一环节失败都回滚已预留资源。

把这条事务落成代码，每完成一步预留就登记到一份可回滚的记录里，任一步失败都据此逐条撤销：

准入事务：把检查变成可撤销的资源预留

```
1 attempt := taskQueue.peek()
2
3 reservation := newReservation(attempt.id)
4
5 if !modelBudget.reserve(attempt.id, attempt.maxTokens) {
6     keepWaiting(attempt)
7     return
8 }
9 reservation.add(modelBudget)
10
11 if !sandboxPool.reserve(attempt.id, attempt.envSpec) {
12     reservation.releaseAll()
13     keepWaiting(attempt)
14     return
15 }
16 reservation.add(sandboxPool)
17
18 if !trajectoryLog.openWriter(attempt.id) {
19     reservation.releaseAll()
20     slowDownAdmission()
21     return
22 }
23 reservation.add(trajectoryLog)
24
25 taskQueue.pop()
26 markRunning(attempt, reservation)
```

这段逻辑对应云平台里的 admission controller 与 lease management。控制面在动作开始前建立可追踪的资源承诺；当任务取消、超时或节点失联时，它也能沿着这些承诺释放资源。MegaFlow 的三服务拆分解解决的是“谁负责哪类状态”，这里的准入事务解决的是“这些状态怎样在一次启动中保持一致”。

控制面还必须处理重复请求和部分失败。设想以下场景：Agent 服务准备开始某个 Attempt，于是向环境服务发送“创建 sandbox”的请求。环境服务可能已经在某台宿主机上创建好了容器，并分配了磁盘写层、端口和进程组；但就在它把 sandbox 地址返回给 Agent 服务之前，网络连接断开了。此时 Agent 服务只知道“我没有收到结果”，却不知道环境服务到底有没有把环境创建出来。

如果 Agent 服务直接重发一次创建请求，环境服务可能会再创建一个新的 sandbox。这样，同一个 Attempt 就会对应两个环境：第一个环境已经存在但没有被 Agent 服务记录下来，第二

个环境是重试时新建的。前者会继续占用 CPU、内存、磁盘和端口，却没有任务接管；后者则被 Agent 服务拿来继续执行。任务数量一大，这类“没人认领的环境”会变成严重的资源泄漏。反过来，如果 Agent 服务为避免重复创建而选择不重试，那么它手里没有 sandbox 地址，任务状态只能停在正在创建环境，后续模型调用、工具执行和验证都无法开始。

解决这个问题，需要让创建请求带上稳定的幂等标识，比如 `AttemptId`。幂等 (idempotency) 强调的是“同一逻辑请求应得到一致返回”，常见实现是去重键 + 状态查询：环境服务收到 `AttemptId=A` 时，先查该尝试是否已有历史结果；若已有历史，则返回历史状态（已创建/创建中/失败）并继续推进，而不直接发起第二次创建。如果返回结果是 `Unknown`，需要结合状态确认而不是只依赖单次返回。这样可降低“网络抖动带来的重复重试”的概率，但并不自动等价于通用的 `exactly-once` 副作用语义。

控制面还要把任务准入做成明确规则。队列里还有位置，并不代表任务就可以立刻开始；控制面还要知道这个任务需要什么样的环境。代码修复任务可能只需要一个 Linux 容器、一份仓库、编译器和测试脚本；网页任务可能需要浏览器、固定分辨率、网络访问策略和截图接口；桌面任务还可能还需要窗口系统、剪贴板、输入设备和更大的磁盘写层。它们都叫 sandbox，但资源成本和启动路径完全不同。

因此，任务进入系统之前，控制面需要先生成一张环境需求清单。这里把它记作 `EnvironmentSpec`。这张清单至少要写清楚基础镜像、CPU 和内存配额、磁盘大小、是否需要浏览器或桌面、是否允许访问外网、最多允许执行多少步、验证器从哪里启动。调度器拿到这张清单后，才能判断应该从哪个环境池里申请租约，应该把任务放到哪类宿主机上，应该预留多少模型预算和日志空间。如果没有这张清单，调度器只知道“我要一个 sandbox”，却不知道它到底是轻量代码容器、浏览器环境，还是带 GUI 的桌面环境。结果往往是两种：轻任务被分配到昂贵的桌面环境里，浪费资源；重任务被放入普通容器里，启动后因资源不足而无法正常运行。

生产级平台还会把控制面拆得更细。DeepSeek-V4-Pro 技术报告中的 DSec 使用 API gateway、per-host agent 和 cluster monitor 三类组件：入口层处理请求和鉴权，宿主机侧组件管理本机 sandbox，监控组件维护集群状态。这个设计说明，大规模 sandbox 控制面要持续维护生命周期、租约、故障和恢复关系；“创建容器 API”只是其中很小的一部分。

到这个规模，控制面管理的已经超过了“启动多少个容器”这一层。它要维护每次尝试背后的一组状态记录：环境是否已经分配，模型推理额度是否已经占用，任务是否超过时间限制，资源是否完成回收，轨迹是否已经可靠落盘。只要这些状态有明确的生命周期、超时规则、租约关系和幂等约束，十万级 sandbox 才能作为一个整体系统运行，避免退化成十万个彼此孤立的进程。

6.5.3 执行面：环境保真度与资源密度的取舍

控制面把任务分配出去之后，压力转移至执行环境本身。第 6.1 节中，容器依靠 Namespace、Cgroups 和 OverlayFS 提供隔离、限额和文件系统视图；Serverless 部分则涉及冷启动、预热和轻量沙箱。在 Agent sandbox 场景里，这些机制会同时面对两个要求：环境要足够真实，又要能被大规模创建、复位和回收。

控制面解决“谁该拿到环境”，执行面解决“这个环境到底长什么样”。这里不能简单问“用容器还是虚拟机”，而要先问任务需要多真实的环境。`EnvironmentSpec` 里写的，正是这些问题：操作系统是什么，CPU 和内存给多少，磁盘从哪个镜像启动，能不能联网，允许哪些工具，是否需要浏览器，是否需要桌面。代码任务可能只需要一个 Linux 容器、一个仓库和一套测试工具。网页任务需要浏览器、页面状态、Cookie、截图和网络条件。Computer-use 任务的资源需求更高，它需要窗口系统、输入设备、屏幕流、剪贴板、应用安装状态和后台服务。

环境越真实，轨迹数据越有价值，但成本也越高。纯容器启动快、密度高，适合命令行和代码任务；虚拟机隔离更强，系统行为更完整，但启动慢、占用大；Docker 加 KVM 是一种分层折中：外层容器接入 Kubernetes 等集群调度体系负责编排与生命周期管理，内层通过 KVM 虚拟化提供更完整的操作系统语义和更强的隔离边界。DeepSeek DSec 把 function call、container、microVM 和 fullVM 放到统一接口之后，调用方可以用同一套命令执行、文件传输和 TTY 访问接口使用不同后端。function call 适合无状态工具调用，可以放进预热容器池消除冷启动；container 适合 Docker 兼容的代码任务；microVM 通过 Firecracker 提供更强隔离；fullVM 则面向需要完整客户机操作系统的任务。这些后端体现了分级执行的设计原则：平台必须按任务风险、保真度和成本选择运行时，统一接口只能解决调用方式一致，不能替代运行时选择。

OSGym³⁸ 这类工作把基本单元做成 OS replica，也就是一份可配置、可复位、可操作、可评测的操作系统副本，正是为了保留真实桌面任务需要的窗口、用户目录、后台进程、应用缓存和 GUI 反馈。它进一步说明，完整 OS 环境会把执行面压力推向 CPU、内存、磁盘和内核参数。系统如果想在台宿主机上运行大量 replica，就要考虑硬件感知调度、预热池、资源守护和故障隔离。

OS replica 的成本压力集中在三个方面。磁盘方面，每个 replica 都需要一块可启动磁盘，如果全量复制，几百个环境就会迅速消耗数 TB 存储；OSGym 使用 KVM 虚拟化配合 copy-on-write 磁盘，让多个 VM 共享同一份基础镜像，只为各自修改过的块付费。启动链路方面，按需创建和销毁 VM 容易让训练批次卡在环境准备阶段，因此 OSGym 维护固定大小的预热 runner pool：环境在训练前先准备好，任务来了就分配，任务结束后复位再放回池中。宿主机资源方面，大量 VM 运行在容器中，会触碰文件描述符、inotify watches、AIO context、连接跟踪表等内核限制；资源守护和系统参数调优不是附加细节，而是执行面能否高密度运行的条件。

可以据此得到一个判断标准：如果任务结果主要由命令行输出和文件变化决定，容器通常就够了；如果任务结果依赖系统级行为，比如窗口、剪贴板、桌面应用、后台服务和浏览器状态，就需要更完整的 OS replica。把所有任务都放进 VM，会浪费集群资源；把桌面任务压进普通容器，又会让模型学到失真的环境规律。

从调度角度看，执行环境按保真度可以分级。命令行 sandbox 适合代码补丁、脚本运行、编译测试，它关心的是文件系统、编译器、依赖管理和进程隔离。浏览器 sandbox 适合网页操作和表单任务，它除了文件和进程，还关心浏览器 profile、Cookie、页面加载、截图和网络策略。桌面 sandbox 适合 Computer-use 任务，它需要窗口系统、输入设备、屏幕流、剪贴板、应用安装状态和后台服务。不同级别不应该用同一套成本模型。让桌面 sandbox 执行简单代码任务，是浪

³⁸OSGym: Scalable OS Infra for Computer Use Agents, arXiv:2511.11672。

费；让普通容器去模拟真实桌面，则会失真。

保真度并不是抽象概念。字体缺失会改变页面布局，分辨率不同会改变按钮位置，浏览器版本不同会改变 DOM 行为，键盘映射不同会让输入动作失败，后台服务残留会影响下一轮任务。对人来说，这些都是小问题；对模型来说，这些就是观察空间的一部分。如果训练时环境保真度不足，模型学到的动作在真实系统里会打折。如果环境过于完整，单个 sandbox 的内存、磁盘和冷启动成本又会压垮集群。

执行面的核心工作，是在保真度、隔离强度、启动速度和资源密度之间建立可量化的取舍关系。文件系统可以使用 copy-on-write。逻辑上，每个环境都有自己的完整磁盘；物理上，它们共享同一个只读基础镜像，只为自己修改过的块付费。这样既能保证隔离，又不会为每个环境全量复制几十 GB 的系统盘。镜像也要分层：系统层、语言运行时层、浏览器层、任务依赖层、用户写层。层次分清后，更新某个依赖包不需要重做整个桌面镜像。DeepSeek DSec 中使用 3FS 支撑的只读 EROFS 层和 overlaybd 写层，本质上也是用分层存储和按需加载缓解大规模镜像启动压力。

冷启动路径也要拆开看。一个环境从“没有”到“可用”，中间可能经历拉取镜像、创建磁盘、启动执行环境（容器或 VM）、启动桌面、打开浏览器、加载任务仓库、运行就绪检查。每一步都可能慢。仅知道启动一个 sandbox 需要 3 秒并不充分，平台必须知道慢在哪里。镜像拉取慢，就做镜像预分发；磁盘创建慢，就做预创建；桌面启动慢，就维护 warm pool；任务依赖安装慢，就把常用依赖烘焙进基础层。

预热池是执行面最重要的工程手段之一。训练和评测常常按 batch 运行，如果每来一个任务才从零启动 VM、挂载磁盘、加载桌面、启动应用，模型会一直等环境。预热池提前准备一批干净环境，任务来了就借走，任务结束后复位再放回。它的代价是占用常驻资源，收益是吞吐稳定、尾延迟下降。工业系统里经常要维护多个池：代码池、浏览器池、桌面池，不同池对应不同镜像和不同启动成本。

复位比启动更难。启动只要把环境拉起来，复位却要保证环境回到干净状态。代码仓库有没有残留修改，浏览器缓存有没有污染，后台服务有没有退出，临时端口有没有释放，用户目录里有没有上一次任务留下的文件，都要检查。许多 sandbox 平台表面上可以运行任务，是因为它只验证了能启动；而真正决定训练数据质量的，是任务结束后环境能否可靠地恢复到初始状态。

执行面还要负责安全边界。Agent 会运行模型生成的命令，可能写文件、启动服务、下载依赖、访问网络。平台不能默认这些动作都是安全的。常见做法包括限制出网、限制可挂载目录、限制系统调用、限制进程数量、限制 CPU 和内存、隔离用户身份、记录所有工具调用。越接近真实电脑，攻击面越大；越强调安全，环境越可能受限。因此，安全策略也应当随任务类型和风险等级分级配置。上述保真度、隔离强度、启动速度和资源密度之间的取舍如果处理不当，后续的调度、交互、checkpoint 和训练吞吐都会受到连锁影响。

6.5.4 交互面：分离控制路径与高频数据路径

执行环境准备好之后，系统还要把模型和环境之间的高频交互稳定传输。Agent sandbox 的交互面承担的正是这一职责：将模型发出的动作送达执行环境，将环境产生的观察送回模型。这

需要回到云平台中常见的控制面与数据面分工：控制面决定谁可以使用哪个环境，数据面承载具体的数据传输。容器网络和 Serverless 中已有类似分工：控制逻辑要稳定，数据路径要短、快、可扩展。Agent sandbox 的数据面还要传输截图、终端输出、文件片段、工具调用结果和鼠标键盘事件，压力比普通请求响应更复杂。

普通批处理任务启动后独立执行直到结束，Agent sandbox 却要 and 模型持续通信。模型每走一步，都可能产生观察和动作。观察可能是一张截图、一段终端输出、一个网页 DOM、一个文件片段；动作可能是鼠标点击、键盘输入、命令执行、文件写入、浏览器跳转。一次任务可能几十步、上百步；一批任务可能成千上万条轨迹同时执行。

最朴素的设计，是让所有交互都经过中心控制器。模型要执行命令，需先请求控制器；控制器转发给 sandbox；sandbox 返回输出；控制器再转给 Agent 服务。这个设计在小规模下可行，但在高并发下会迅速成为瓶颈。控制器原本只需要管理任务状态和资源租约，现在却要转发截图、终端输出、文件内容和工具调用结果。请求量增大后，控制器的带宽和 CPU 被数据转发占满，调度、回收和健康检查的响应时间随之恶化。

更合理的做法，是把控制面和数据面分开。控制面管“谁可以使用哪个环境”，数据面管“这个环境里的每一步动作和观察怎样传输”。环境创建、任务分配、状态更新、租约回收走控制面；截图传输、命令执行、文件读写、鼠标键盘事件走更贴近环境的数据通道。EvoCUA 在高并发下使用异步网关承接大量交互请求，体现了这种分工：控制面维持生命周期秩序，高频交互由数据通道处理。

分开之后还要回答一个问题：数据既然不再绕经中心控制器，控制会不会随之丢失？图 91 表明，控制并未随之丢失：低频的创建、删除、策略、心跳仍归控制面，高频的动作与观察改走 Agent 服务到 I/O 网关再到 sandbox 的短路径；而网关上的 SandboxId 路由、租约版本校验和沙箱锁，恰好把控制面的所有权约束钉在数据面的入口上。

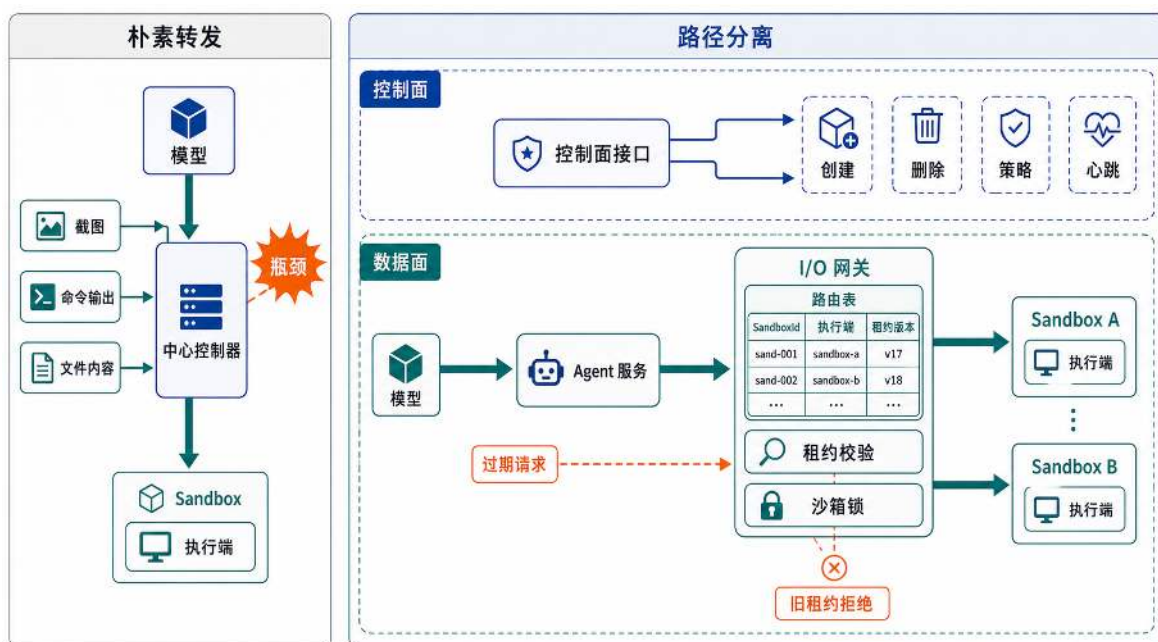


图 91: 交互面将低频生命周期控制路径与高频动作观察数据路径分离，数据面经 I/O 网关直达 sandbox 执行端，并用租约校验阻止旧请求误送。

顺着这条数据面路径，一次工具调用会这样走。模型输出一个动作，比如“运行测试”。Agent 服务先做语义检查：这个动作是否属于允许工具，参数是否合法，是否超过任务预算。然后它把动作发送给环境网关，网关根据 `SandboxId` 找到对应的执行端。执行端在 sandbox 内运行命令，收集 `stdout`、`stderr`、退出码和运行时间，再把观察返回。控制面只需要知道这一步开始了、结束了、是否超时，不应该亲自搬运所有输出内容。

这条热路径需要一张稳定的路由表：`SandboxId` 对应哪个网关、哪个宿主机、哪个执行端，以及当前租约版本是多少。租约版本很重要，因为 sandbox 可能已经被回收后重新分配给另一个 `Attempt`。如果旧请求带着过期版本号到达，网关必须拒绝它，而不能把旧动作误送进新环境。高并发交互面还会给每次工具调用分配 `ToolCallId`，用于重试去重和结果归档。它帮助识别“同一条调用的重复重试”，但在返回状态未知或执行超时，网关仍需查阅状态并做幂等确认，否则不能简单用 ID 就宣称“只执行一次”。

环境网关是 sandbox 对外暴露的唯一入口，承担动作转发和权限校验双重职责。比如同样是文件读取，读任务目录可以，读宿主机目录不行；同样是网络请求，访问任务提供的本地服务可以，访问公网可能不行；同样是命令执行，普通用户权限可以，提权操作不行。交互面如果只追求吞吐，不做权限检查，就会把 sandbox 变成一个高并发的风险入口。

交互面要承载高频 I/O，就需要异步 I/O 网关、连接复用、分片路由和限流。异步网关让大量慢请求不阻塞工作线程；连接复用减少频繁建连的开销；分片路由把不同 sandbox 的流量分散到不同网关；限流和背压保护环境、模型服务和日志系统。Orchard Env³⁹ 的环境服务把生命周期管理放在 `orchestrator` 中，而把命令执行、文件读写和健康检查直接路由到 sandbox pod 内

³⁹Orchard: An Open-Source Agentic Modeling Framework, arXiv:2605.15040.

部的轻量执行 agent，并绕开 Kubernetes API server 的热路径。这个例子说明，云平台里的控制面稳定性和数据面吞吐不能混在一条路径上解决。

Orchard Env 的做法可以抽象成冷路径和热路径分离。冷路径包括创建 Pod、删除 Pod、设置资源限制、下发 NetworkPolicy、等待 readiness probe，这些操作低频但需要一致性，适合由 orchestrator 通过 Kubernetes API 管理。热路径包括 `/exec`、文件上传下载、目录列举、执行端状态查询和命令超时处理，这些操作高频且延迟敏感，适合直接路由到 sandbox 内部的执行 agent。为了避免同一个 sandbox 内部多个命令并发冲突，执行端还需要 per-sandbox lock；为了避免长时间无人使用的环境泄漏，控制面还要通过 heartbeat 清理过期 sandbox。这里的云计算概念仍然是控制面和数据面分离，只是数据面承载的不再是普通 HTTP 请求，而是 Agent 的动作和观察。

交互面还需要处理观察数据分级。终端输出可能需要完整保存，给训练和排错使用；屏幕截图可能只需要在轨迹中保存关键帧，连续帧可以压缩或降采样；大型文件内容不应该直接写入轨迹，可以保存引用和摘要；调试日志可以短期保留，验证结果要长期保留。数据分级的目标是保证训练数据可用，同时避免高频交互请求在存储和网络带宽上形成新的瓶颈。

因此，交互面的指标不能只看平均延迟。它还要看尾延迟，也就是最慢的那一小部分请求。如果一次截图传输偶尔延迟 5 秒，模型就会在错误的时间点上做判断；如果命令输出偶尔丢失部分内容，轨迹就可能失去关键证据。Agent 任务是长链路任务，单步小抖动会沿着轨迹累积，最后表现为任务失败或数据不可用。

在高频、多并发的交互负载下，瓶颈可能首先出现在 I/O 网关、日志管道、截图传输和连接管理上。没有稳定的交互面，环境即使已经成功启动，模型也无法及时获得可靠反馈，长链路任务会在延迟、丢包和观察缺失中逐步失真。

6.5.5 状态面：把回滚和分叉放进训练循环

交互过程一旦变长，环境的中间状态就会成为一种资源。前面讨论 Serverless 冷启动时，快照主要用于加快环境恢复；在 Agent sandbox 中，快照还要支持探索分叉。模型走到某个中间状态后，平台可能需要从这里保存、回滚，再尝试另一条路径。因此，这里讨论的仍然是云计算里的快照、恢复和存储一致性问题，只是它们被放进了训练与评测循环里。

如果 Agent 只线性执行一次，环境坏了可以从头重跑，虽然慢，但还能接受。训练和评测很快会走到更复杂的形态：同一道题要跑多条候选轨迹；搜索算法要从中间状态分叉；调试代码时，模型会尝试一个 patch、跑测试、失败后回到修改前；抢占式云资源被回收时，平台希望换一台机器继续跑。

这些场景都要求 sandbox 支持 checkpoint 和 rollback。在这里，checkpoint 已经进入训练循环本身。如果第 20 步失败，每次都从第 1 步重放，会浪费模型调用、工具执行和环境等待时间。系统需要从关键分岔点恢复，再尝试另一条路径。恢复越快，同样的模型预算就能覆盖更多分支。

checkpoint 的难点不只是保存一份文件。平台要先找到一个一致性边界：上一条工具调用已经结束，文件系统写入已经落到可追踪层，后台进程状态已经可枚举，Agent 侧也记录了当前 turn 的位置。否则，系统可能保存到一半状态：磁盘上已经有新文件，进程内存还停留在旧逻辑；

或者 Agent 记录已经进入下一步，环境却还没完成上一条命令。这样的快照恢复出来后看似可用，实际会把轨迹带到一个从未真实出现过的中间状态。

问题是，Agent 的状态到底在哪里？只保存聊天记录不够。模型可能记得自己执行过 `pip install`，但环境里的依赖树并没有恢复。只保存文件系统也不够。Python 解释器堆、后台进程、打开的文件描述符、临时服务都可能影响下一步。只保存进程也不够，因为磁盘可能已经被另一个分支改过。对 Agent sandbox 来说，一个可恢复点至少要把文件系统状态、进程状态、GUI 状态、后台服务和 Agent 的 turn 位置对齐。

可以把状态分成四层。第一层是任务状态，也就是 Agent 已经走到第几轮、上下文里有哪些观察、当前预算还剩多少。第二层是文件系统状态，包括代码修改、依赖安装、临时文件和浏览器 profile。第三层是进程状态，包括正在运行的服务、打开的连接、解释器内存和文件描述符。第四层是外部状态，比如远端网站、数据库、消息队列和网络服务。前三层平台还能控制，第四层最麻烦，因为它可能已经产生不可逆副作用。所以训练 sandbox 通常要尽量把外部依赖也模拟进环境，或者把外部调用限制在可回放、可清理的范围内。

DeltaBox⁴⁰ 的设计重点，是让保存和回滚足够快。它把文件系统做成可切换的层，每次 checkpoint 冻结当前写层，再插入新的写层；回滚时切回对应层。进程状态则用增量 dump 和 template fork 加速恢复。这样做的目标很明确：checkpoint/rollback 不能是几秒钟一次的灾备动作，而要快到可以放进 Agent 的搜索循环里。如果保存一次状态本身就很慢，模型宁可从头跑，checkpoint 就失去了训练价值。

Crab⁴¹ 的设计重点，是不要每一步都存。Agent 的一轮动作可能只是读取文件，也可能修改源码、安装依赖、启动服务。OS 能看到系统调用和状态变化，却不知道这一轮对 Agent 有没有意义；Agent 框架知道工具调用，却看不到底层 OS 副作用。这就是 Agent 层和 OS 层之间的语义断层。Crab 在每一轮动作结束时观察操作系统可见的影响，再决定这一轮不存、只存文件系统、只存进程，还是做完整 checkpoint。这样可以避免把每一步都做成重型快照。

checkpoint 的位置也要有策略。任务刚开始时，环境很干净，保存一次基础点有价值；模型完成安装依赖后，保存一次可以避免重复安装；修改关键文件前，保存一次便于分支；运行大型测试前，保存一次可以从测试前恢复。相反，连续读取文件时没有必要频繁保存。好的状态面要回答四个问题：什么时候保存，保存到什么粒度，保存多久，谁来清理。

checkpoint 策略可以写成一个简单规则：在语义上可能分叉、在计算上重跑很贵、在资源上还能承受时保存。三者缺一不可。只是语义上重要但保存太贵，系统会被快照拖慢；只是保存很便宜但没有分叉意义，快照会变成垃圾数据；只是重跑很贵但状态不完整，恢复出来的环境也不可信。一种简化的粒度选择逻辑如下。

⁴⁰DeltaBox: Scaling Stateful AI Agents with Millisecond-Level Sandbox Checkpoint/Rollback, arXiv:2605.22781.

⁴¹Crab: A Semantics-Aware Checkpoint/Restore Runtime for Agent Sandboxes, arXiv:2604.28138.

语义感知 checkpoint：按 OS 可见影响选择保存粒度

```
1 effect := inspectOSEffects(lastStep)
2
3 if effect.readOnly() {
4     skipCheckpoint()
5 } else if effect.onlyFileChanged() {
6     checkpointFileSystemLayer()
7 } else if effect.onlyProcessChanged() {
8     checkpointProcessState()
9 } else if effect.fileAndProcessChanged() {
10    checkpointFileSystemAndProcess()
11 }
12
13 if isBranchPoint(lastStep) && replayCostHigh(lastStep) {
14    pinCheckpointForSearch()
15 }
```

这段逻辑说明，checkpoint 不是越多越好，也不是越完整越好。状态面真正要做的是把 Agent 层的轮次语义和 OS 层的实际副作用对齐：读文件通常不需要保存，修改源码至少要保存文件系统，安装依赖或启动后台服务可能需要同时保存文件系统和进程状态。这样的选择既保证恢复正确，又避免把宿主机 I/O 消耗在无意义的快照上。

一次失败恢复可以分成两个阶段。第 i 步执行前后，平台先根据这一轮动作的副作用决定保存哪些状态：只改文件时冻结文件写层，改变运行中进程时保存进程状态，两类副作用同时出现时则把文件层和进程状态纳入同一个 checkpoint。只有当任务轮次、文件写层和进程状态彼此对齐时，这个 checkpoint 才能成为后续恢复继续执行（resume）的起点。图 92 将这一过程展开到同一条时间线上，显示第 i 步失败后系统为什么要回到最近的一致状态点，而不是只回退其中某一层。

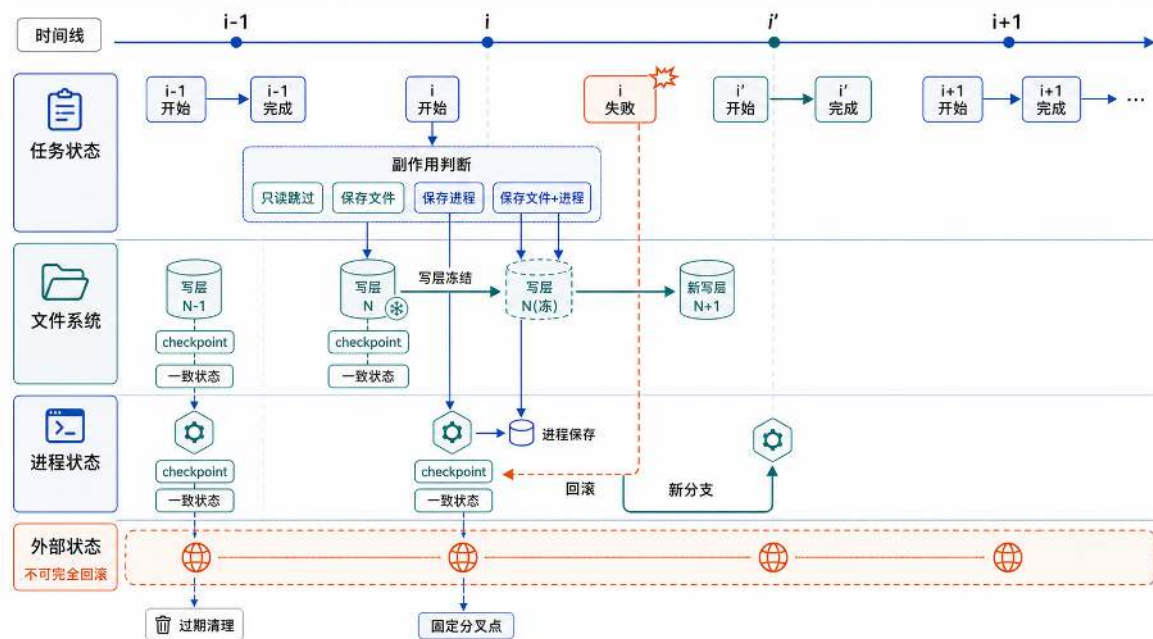


图 92: 语义感知 checkpoint 的失败恢复过程：平台在第 i 步按副作用选择保存文件层、进程状态或二者；失败后回滚到最近的一致状态点，从该状态点启动新分支 i' 。

第 i 步失败后，恢复动作本质上是在重新建立一个可继续执行的环境，而不是把某个日志指针往回拨。系统不能只把 Agent 的会话上下文退回去，也不能只把磁盘文件退回去；它必须恢复到最近一个文件系统、进程状态和任务轮次同时成立的 checkpoint，再从那里启动新分支 i' 。外部接口、时钟和随机源位于平台难以完全控制的边界外，因此它们通常不能像文件写层和进程状态那样可靠回滚，只能通过模拟、录制回放或访问限制来降低不一致风险。这样，checkpoint 才真正成为恢复继续执行的入口，而不是一份看似完整、恢复后却无法继续执行的快照。

高并发时，checkpoint 还会变成资源调度问题。几十个环境同时 dump 内存或写快照，会使宿主机 I/O 饱和。状态面需要一个主机级调度器，把 checkpoint 当作资源来排队。正在等模型返回的任务，可以把 checkpoint 工作安排在等待时间里；模型已经返回、任务正被恢复阻塞，就要提高优先级。快照还要有垃圾回收策略，过期分支、失败分支和低价值中间点不能无限保留。否则，状态面会从“提升探索效率的工具”变成新的性能瓶颈。

checkpoint 的价值不只在节省重跑时间。它也为后面的失败轨迹分析提供了可操作的入口：如果系统能把环境恢复到某个关键错误发生前，就可以从同一个中间状态重新探索。状态面保存的不只是执行现场，也是在为训练数据的再加工保留分叉点。

6.5.6 验证面：让轨迹能被复现、筛选和归因

环境能启动、能交互、能回滚之后，还必须回答最后一个云平台问题：运行过程是否可观察、可审计、可追溯。扩缩容和 Serverless 场景中，监控指标用来判断系统是否健康；在 Agent sandbox 中，日志和指标还要进一步变成训练数据的证据链。一次任务是否成功，不能只看最终分数，还要能追溯它使用了哪个镜像、哪个模型、哪个验证器，以及中间是否发生环境异常。

完成调度、交互和状态保存之后，一个大型 sandbox 平台已经具备执行能力。但训练和评测还需要最后一层保证：轨迹必须能被解释。否则，平台运行规模越大，越可能积累大量无法用于训练和评测的数据。对训练系统来说，错误归因的轨迹比没有轨迹更危险，因为它会把模型带向错误经验。

验证面承担三项职责：判断轨迹能否复现，判断轨迹能否进入训练或评测数据，以及在失败时将原因归属到模型、环境、工具或验证器。只有这三项职责得到满足，前面生产出来的轨迹才有价值。

一条可用轨迹至少要回答这些问题：这次任务从哪个任务版本开始？用了哪个环境镜像？允许哪些工具？网络是否开放？模型是哪一个版本？中间有没有从 checkpoint 恢复？验证器是哪一版？reward 是怎样计算的？失败前是否重试过？如果这些信息丢失，轨迹数量虽大，训练价值却会下降。百万条轨迹如果来自不同版本的任务和评测器，却没有记录版本对应关系，数据的统计意义就无法保证。

验证面首先要追求可复现。这里的可复现，并不要求模型每次都走出完全相同的动作序列；它要求给定任务版本、环境版本、工具权限、随机种子、模型版本和验证器版本后，工程师能重新构造出当时的评测条件。代码任务要能重新拉起仓库和依赖，桌面任务要能恢复屏幕与应用状态，网页任务要能恢复页面数据或模拟服务。没有复现能力，失败分析就缺乏可验证的基础。

这也是为什么轨迹要记录来源与派生关系。平台要知道一条轨迹从哪个任务模板开始，运行在哪个环境镜像上，由哪个模型产生动作，调用过哪些工具，最后由哪个验证器给出分数。如果一条恢复样本是从失败轨迹中分叉出来的，平台还要记录它来自哪条失败轨迹、从哪个 checkpoint 继续执行。缺少这些来源记录，轨迹就像没有实验条件的实验结果，看起来有数字，实际上很难使用。

reward 也不能只靠模型自评。代码任务要跑测试，桌面任务要检查窗口或文件状态，网页任务要验证页面结果。EvoCUA 中把任务和可执行验证器一起生成，强调的正是这一点：环境交互轨迹只有配上可执行、可复查的验证规则，才能成为训练和评测数据。更好的做法是把验证器也纳入版本管理：任务版本、环境版本、验证器版本和模型版本一起记录。这样，当某批轨迹分数异常时，工程师才能判断是模型退化、环境漂移，还是验证器改了规则。

失败归因同样重要。评测平台真正难处理的是失败原因无法明确归属：模型决策错误、浏览器响应超时、环境复位不完整、reward 脚本版本变更，这些因素可能同时存在。模型错误应该进入模型分析；环境错误应该进入基础设施告警；评测器错误应该冻结结果，等待修复后重跑。把这些混在一起，训练系统就会把基础设施噪声当成模型经验。

失败轨迹还可以进一步转化为训练资源。一次 Agent 任务最终失败，并不意味着整条轨迹上的每一步都是错误的。更常见的情况是：前面的观察、定位和部分操作都可能是合理的，真正把任务带偏的只是某一个关键步骤——在一条数十步的轨迹中，模型可能前几步的观察和操作都正确，失败仅源于某一步的错误判断或错误分支选择。

因此，验证面可以尝试从失败轨迹中定位关键错误步骤。设一条轨迹在第 n 步失败，系统通过验证器、日志和状态差异分析判断，第 i 步可能是导致失败的动作。此时，如果状态面保存了第 $i-1$ 步之前的 checkpoint，平台就可以把 sandbox 恢复到错误发生前的环境状态，再从这一

点重新引导模型选择动作。若新的分支最终通过验证，系统就得到了一条很有价值的恢复样本：它不只告诉模型“任务怎样做成”，还告诉模型“在一个复杂中间状态下，怎样从即将走错的位置恢复回来”。

这类恢复样本对 Agent 能力训练尤其重要。普通成功轨迹提供的是一条可行路径；失败后的恢复轨迹提供的是纠错经验。模型在真实环境中执行任务时，难免会遇到命令超时、测试失败、页面状态异常、文件修改走偏等情况。它需要学习的不是永远不犯错，而是在环境已经发生变化之后，仍然能够识别偏差、回到合理路径，并继续推进任务。

上述机制说明，高质量 Agent 训练会反过来要求更强的 sandbox 云基建。轨迹日志负责记录每一步发生了什么，验证器负责判断哪一条分支真正成功，checkpoint/rollback 负责把环境恢复到关键步骤之前，轨迹来源记录负责说明恢复样本来自哪条失败轨迹、从哪个 checkpoint 分叉出来。只有这些能力同时成立，失败轨迹才能从一次运行结果，变成可以复现、可以分叉、可以筛选的训练数据。

图 93 把前面这些环节串联成一条完整的派生链：验证器先在失败轨迹上定位第 i 步，状态面据此把 sandbox 回滚到第 $i-1$ 步的 checkpoint，Agent 服务从这个环境状态重新探索。这条链上有一道不能省的关卡：新分支跑完必须重新过一遍验证，只有验证通过的分支，才带着来源字段写回轨迹库。

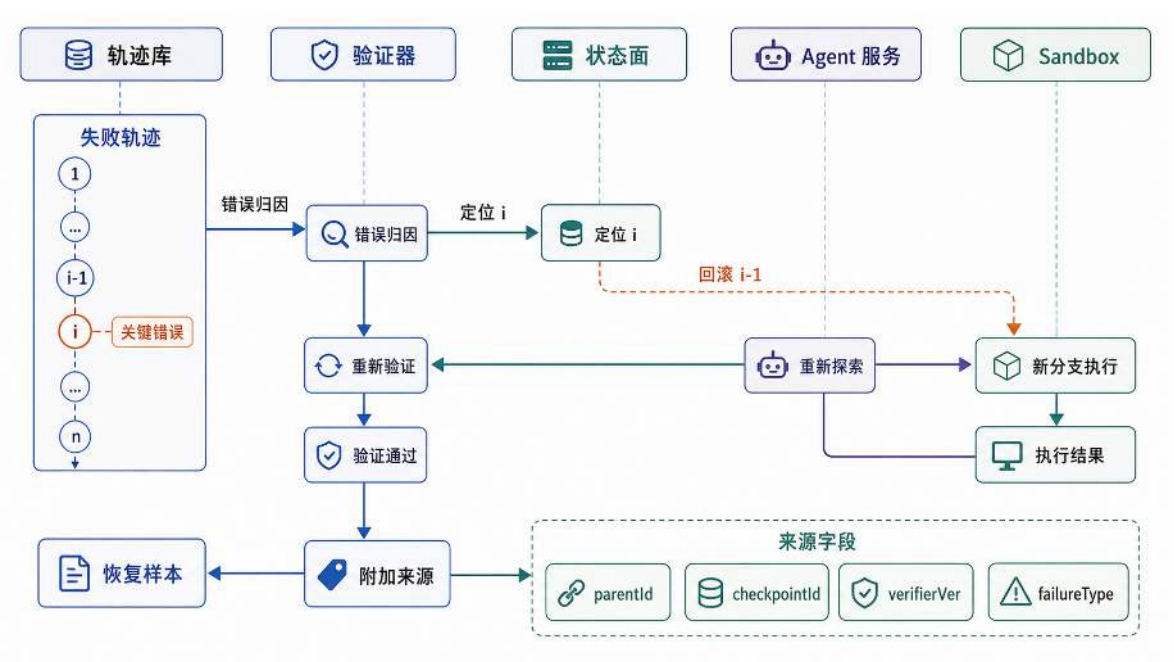


图 93: 失败轨迹可以经由错误归因、checkpoint 回滚、Agent 重新探索和验证器重新验证派生为恢复样本，并在轨迹记录中保留来源关系。

为了让上述过程产出的恢复样本能够被复现、筛选或用于训练，验证面必须在轨迹记录中写入来源字段：父轨迹、checkpoint、验证器版本和失败类型分别说明样本从哪里来、从哪个状态分叉、由哪个验证逻辑判定，以及原始失败属于哪一类。

训练数据还需要筛选。成功轨迹有价值，但失败轨迹也可能具有训练价值。一次失败可能暴露了模型不会恢复的分支，也可能只是 sandbox 网络抖动。平台要把失败分成几类：模型决策错误、工具使用错误、环境异常、验证器异常、超预算、用户任务本身有问题。只有分类清楚，失败轨迹才能用于训练模型的纠错能力，而不是把系统故障教给模型。

为了支撑这些判断，轨迹记录需要从一开始就结构化，而不是事后从日志里拼。一条轨迹的最小结构可以组织为如下记录：

TrajectoryRecord：一条可复查轨迹的最小结构

```
1 TrajectoryRecord {
2     taskVersion
3     environmentSpec
4     imageDigest
5     modelVersion
6     toolPolicy
7     steps[] {
8         observationRef
9         action
10        toolResult
11        timestamp
12        checkpointId
13    }
14    verifierVersion
15    reward
16    failureType
17 }
```

这里的 `imageDigest` 用来确认环境镜像没有漂移，`toolPolicy` 用来确认工具权限一致，`checkpointId` 把轨迹和状态面连接起来，`verifierVersion` 则保证 `reward` 的含义可追溯。这样，验证面就不再只是保存日志，而是在为后续训练、评测、审计和复现实验建立统一的轨迹来源与派生记录。

最后，验证面要把轨迹变成数据资产，而不是一次性日志。DeepSeek DSec 维护每个 sandbox 的全局有序轨迹日志，记录命令调用及其结果，用于抢占后的快速续跑、细粒度溯源和确定性重放。这个设计说明，日志在 Agent sandbox 中不只是排错材料，也是一种恢复和审计机制。轨迹可以被重放，可以被抽样审查，可以按任务类型、工具类型、失败类型、环境版本检索。这样，研究人员才能提出有意义的问题：模型是不是在安装依赖时经常走错？是不是浏览器任务里的截图延迟导致失败？是不是某个基础镜像升级后通过率下降？在这一层面上，sandbox 平台承担的职责已经超过任务执行本身，还包括为模型能力改进提供可复查的实验记录。

6.5.7 小结：Agent sandbox 的五个面向

从整个 Agent sandbox 子系统来看，存在一条清楚的设计链路：先把任务状态、资源状态和轨迹状态分清；再用控制面维持全局秩序；用执行面提供真实而可承受的环境；用交互面承接高频动作和观察；用状态面支持回滚与分叉；最后用验证面把轨迹变成可信数据。只要这条链路中任何一环不稳，问题就可能传导到训练数据的质量中。

这些设计并没有脱离云原生的基础机制。容器和 VM 提供隔离边界，OverlayFS 和 CoW 磁盘降低复制成本，HPA 和 Serverless 提供弹性调度思路，快照与恢复把冷启动和状态迁移变成可优化对象，日志和监控让失败原因有迹可循。Agent 训练与评测只是提出了一种更难的新工作负载：它不只要运行服务，还要批量运行一个个会被模型改变的环境。

因此，面向 Agent 的高并发 sandbox 是云计算问题在新工作负载下的延伸。云原生让普通应用可以被批量部署和伸缩；Agent sandbox 则让可交互环境本身也能被批量创建、隔离、复位、分叉、验证和回收。支撑这些能力的仍然是进程、文件系统、网络、调度和控制循环，只是这一次，被管理的不再只是服务副本，而是一条条正在试错的执行轨迹。

6.6 小结：从基石机制到系统工程方法

回到本章开头的问题：当系统不再只是“运行一个服务”，而是要批量创建隔离环境、接入网络、调度资源、快速启动、保存状态，并在失败后复现与验证时，云原生平台到底承担了哪些工作？学完这一章，读者应当看到，云原生不是一组产品名，而是一套围绕进程、文件系统、网络、资源、状态和日志展开的系统工程方法。

本章先拆开这些能力的基础机制。容器用 Namespace 改变进程看到的系统视图，用 Cgroups 约束资源使用，用 rootfs 构造文件系统环境，并以 OverlayFS 说明镜像层共享和容器独立写入的一种常见实现；容器网络用 veth、bridge、NAT、VXLAN 和端口映射，把隔离的网络命名空间重新接入服务系统，并在高频交互场景下面对 Overlay 封装开销。应用要接受自动伸缩，无状态服务必须做到状态外置、稳定接入和安全下线，有状态服务则必须先划分状态类型，再明确状态归属、分片方式和重平衡协议。平台内部用指标、目标、动作和校验组成反馈闭环，并让 HPA、调度器和节点扩容器按不同时间尺度分层协作；调度器先用硬约束形成可行集合，再用多目标评分选择放置位置。Serverless 把容量管理继续下沉到按需执行环境，但冷启动、预热池、快照恢复和启动路径拆分会决定它能否服务低延迟场景。

面向 Agent 的高并发 sandbox 把这些基础机制重新组合到更长的任务生命周期中。同一个反馈闭环，从弹性伸缩的副本管理延伸到 Agent 任务的尝试、保存、回滚与验证；同一套隔离与网络机制，从承载服务请求延伸到承载模型对环境的持续修改；同一种存储与日志思路，从记录系统健康延伸到记录可复现的训练数据轨迹。Agent sandbox 并没有脱离云计算主线，它只是把隔离、网络、调度、状态和可观测性推到更高强度：平台管理的不再只是服务副本，也是一条条会改变环境、会失败、会恢复、会被重新用于训练的数据轨迹。

6.6.1 未来展望：云原生的下一站

云原生技术不会停在 Kubernetes、Serverless 或某一种 sandbox 形态上。更值得关注的是，新的工作负载会不断把旧问题推到更高强度：更多实例、更短启动、更复杂状态、更长执行过程，以及更严格的复现和审计要求。沿着本章的主线看，未来的云平台至少有四个方向值得持续关注。

第一，云平台会从托管服务副本，走向托管执行轨迹 传统云平台主要管理服务副本：一个 Pod 是否健康，一个函数实例是否就绪，一个服务是否需要扩容。Agent sandbox、自动化评测、CI/CD 流水线 and 数据处理任务提出了更复杂的要求：平台不仅要知道一个环境是否在运行，还要知道一段执行过程走到了哪里，中间产生了哪些状态，能不能从某一步恢复，失败后能不能复现。也就是说，平台管理的对象正在从“运行中的实例”扩展到“可记录、可恢复、可验证的执行轨迹”。这意味着任务状态机、租约、checkpoint、日志和验证器需要成为更基础的平台能力。

第二，控制面会从阈值响应，走向半自治闭环 HPA 已经展示了控制闭环的基本形态：观察指标，计算目标，执行动作，再等待结果生效。未来的控制面仍然会遵循这个框架，但观测信息会更丰富，决策过程也会更接近半自治。平台可能结合历史流量、活动日历、实时指标和故障信号提前扩容；也可能在延迟异常时自动收集日志、隔离实例、触发重试，并把处理过程交给工程师复核。关键在于让控制面具备更好的异常归因、动作建议和结果验证能力。无论形式怎样变化，云平台仍然离不开清晰的观测、决策、执行和回滚边界。

第三，执行环境会走向分层沙箱 未来不会只有一种标准沙箱。容器启动快、密度高，适合命令行和普通服务；microVM 提供更强隔离，适合多租户和不可信代码；完整 VM 或 OS replica 保留更多系统语义，适合桌面、浏览器和复杂工具链任务；Wasm 则把隔离边界下沉到指令集与运行时层，适合短、轻、系统调用依赖少的计算单元。平台要做的不是把所有任务塞进同一种运行时，而是根据隔离强度、环境保真度、启动速度和资源成本选择合适的执行后端。第 6.1 节讲的 Namespace、Cgroups、OverlayFS，以及 Serverless 中的冷启动、预热和快照，会继续以不同组合出现在这些沙箱形态中。

第四，状态和日志会成为云平台的核心资产 过去，很多系统把状态管理留给应用，把日志主要用于排错。随着 Serverless、Agent sandbox 和自动化评测的发展，状态与日志会越来越靠近平台本身。状态需要被保存、迁移、回滚和分叉；日志需要记录任务版本、环境版本、模型版本、工具调用和验证结果。它们不再只是故障发生后的辅助材料，而是恢复执行、复现实验、筛选训练数据和审计系统行为的依据。未来的云平台如果只会启动实例，却不能解释实例里发生过什么，就很难支撑更复杂的自动化工作负载。

这些方向看起来指向不同技术，背后仍然是本章反复出现的基础问题：执行环境怎样隔离，资源怎样调度，状态怎样保存，失败怎样恢复，运行过程怎样被记录和验证。理解这些问题，比记住某个具体平台的名字更重要。

6.6.2 写给读者的寄语：不畏浮云遮望眼

作为这本教材云原生核心原理篇的收尾，我们想留下一个朴素但重要的工程信念。

今天的开源工具和云平台已经足够丰富，许多问题看起来只需要几行 YAML、几个 API 就能解决。可是，一旦系统进入真实生产环境，排队、卡顿、断连、OOM、冷启动、状态漂移和日志缺失都会重新出现。真正决定工程师能否处理这些问题的，往往是沿着系统边界向下追问的能力：进程在哪里运行，资源在哪里受限，网络路径如何转发，状态怎样保存，失败又被记录到了哪里。

这也是本章坚持拆解底层机制的原因。在计算机科学中，越是复杂的上层系统，越离不开那些看似朴素的基础构件：进程、内存、文件系统、网络 I/O、调度循环、状态恢复和日志记录。容器、Kubernetes、Serverless，乃至面向 Agent 的 sandbox 平台，都只是这些构件在不同压力下的重新组合。

希望本章能成为读者心中的一个起点。以后面对新的微服务框架、调度算法、云平台形态，甚至下一代由 AI 驱动的基础设施时，不必停在黑盒外面反复试错，而是敢于打开它，沿着隔离、网络、调度、状态和可观测性一层层看下去。只要能把复杂系统拆回这些基本问题，就已经抓住了理解它的入口。

云计算的技术形态还会继续变化，但底层问题不会消失。应用仍然要被隔离，资源仍然要被调度，网络仍然要被连通，状态仍然要被恢复，失败仍然要被记录。愿读者在之后的学习和工程实践中，始终保留这种追问底层的勇气：不止会使用工具，也能理解工具；不止能跑通系统，也能解释系统；不止在黑盒外等待答案，也能亲手打开下一个黑盒。

6.7 思考题

1. 容器 A 内部运行 `ps` 时只能看到自己的进程，但它和宿主机共享同一个 Linux 内核。请解释 PID Namespace 如何让容器看不到宿主机进程，并说明为什么共享内核使容器隔离弱于虚拟机隔离。进一步思考：如果宿主机内核存在漏洞，容器共享内核会带来什么风险？为什么虚拟机的独立内核边界更强？
2. 战斗服容器的 CPU limit 设为 2 核，某一帧的物理计算突然需要超过 2 核的计算能力。请解释 Cgroups 会如何限制 CPU 使用，玩家侧可能看到什么现象。注意这里的“2 核”通常指在一个配额周期内最多使用相当于 2 个 CPU 的累计运行时间，并不意味着进程被固定绑定到某两个物理核心（只有配合 CPU Manager、cpuset 等绑核机制时才涉及固定核心集合）。再比较 CPU throttling 与内存超限后的 OOM kill：它们分别在什么资源压力下发生，对进程生命周期有什么不同影响？
3. 容器 A (`lobby-api`) 要访问同一台宿主机上的容器 B (`redis`)。请画出数据包从 A 的 `eth0` 到 B 的 `eth0` 经过的完整路径，包括 veth、bridge 和对端 veth。然后解释为什么容器内的 `127.0.0.1` 只指向容器自己的网络命名空间，而不是宿主机或另一个容器。

4. 活动刚开始, QPS 在 30 秒内翻倍。HPA 的采样周期是 15 秒, 新 Pod 从创建到 Ready 需要 45 秒。请估算从负载上升到新容量真正生效, 系统至少会经历哪些延迟阶段。进一步思考: 最小副本数和预热池分别怎样缩短玩家排队时间, 它们又会带来什么成本?
5. 一个 Serverless 函数的冷启动耗时 1.2 秒, 其中沙箱创建约 200ms、运行时启动约 400ms、应用初始化约 600ms。请判断 MicroVM、Wasm、Snapshot 和暖池分别属于“减少工作”“复用结果”还是“预留容量”, 并说明它们主要影响哪些阶段。进一步比较: 快照按需恢复需要支付哪些补齐成本, 暖池为什么会产生持续的资源成本, Wasm 又为什么不适合强依赖完整 OS 语义的函数?
6. 某匹配系统需要扩容, 其中包含可从数据库重建的排行榜缓存、玩家 WebSocket 连接、全局匹配队列和账户余额。请分别判断它们属于哪类状态, 并说明哪些状态可以直接重建、哪些需要排空或迁移、哪些必须由协调或持久化机制保护。
7. 一个 Pod 需要 4 核 CPU、GPU 标签并要求与同类副本分散部署。请说明调度器怎样先用硬约束形成可行节点集合, 再怎样按资源均衡、故障域分散和数据本地性评分。如果当前没有可行节点, 为什么 Cluster Autoscaler 也不一定能够通过增加任意 Node 解决问题?
8. lobby-api 的平均停留时间为 200ms, 当前请求到达率为每秒 500 个。请用 Little 定律估算系统内的平均请求数。现在假设到达率突然从 500 升到每秒 1000 个, 系统仍能持续完成每秒 800 个请求, 并且不拒绝、不丢弃请求。请计算队列每秒净增加多少请求, 以及 t 秒后比负载上升前多积压多少请求; 再解释为什么系统失去稳态后, 不能继续用一个固定的平均停留时间和平均请求数描述它。进一步思考: 为什么 P99 往往会比平均值更早、更明显地恶化?
9. 一个 Agent 任务启动前需要同时获得 sandbox、GPU 配额和网络出口。平台先后完成前两项预留, 却在申请网络出口时失败。若平台直接重试整个流程, 可能出现哪些资源泄漏或重复分配问题? 请说明幂等预留、超时释放和后台对账分别怎样处理这类部分失败, 并解释为什么“检查时资源充足”不能保证后续申请一定成功。
10. 控制器故障重启后, 把同一个 Attempt 分配给了新的执行节点, 但旧节点尚未停止, 并继续提交工具调用。请说明租约版本或 fencing token 怎样阻止旧节点继续写入状态, 再说明 ToolCallId 幂等键怎样避免同一次工具调用被重复执行。为什么只使用其中一种机制仍然不够?
11. Agent 在第 i 步修改了 sandbox 内的代码文件, 同时向外部服务发送了一条消息。平台随后回滚到第 $i-1$ 步的 checkpoint。哪些状态能够随 checkpoint 恢复, 哪些外部副作用不会自动撤销? 为了判断回滚后的新分支是否可信, 轨迹记录和验证器至少还应保留哪些版本与来源信息?